



# 媒体与认知

Media and Cognition

清华大学电子工程系

王生进 李亚利

Email: [liyali13@tsinghua.edu.cn](mailto:liyali13@tsinghua.edu.cn)

# 第6讲 深度学习之序列建模

6.1 序列建模概述

6.2 循环神经网络

6.3 TRANSFORMER

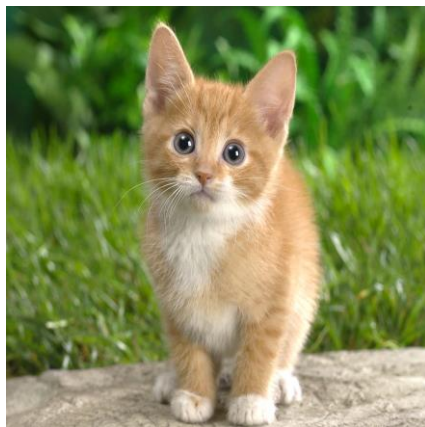
6.4 应用与展望

# 6.1 概述

## ➤ 基于神经网络的模式识别

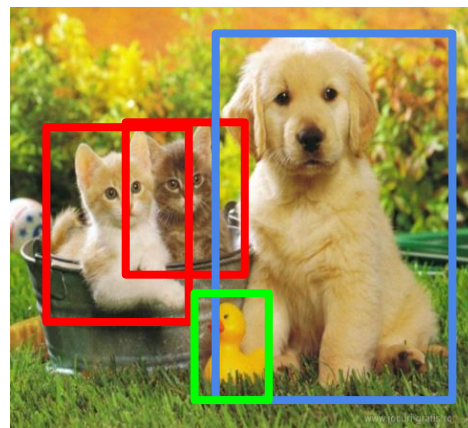
- ◆ 模式识别：图像分类、目标检测、手写数字识别等

**Classification**



CAT

**Object Detection**



CAT, DOG, DUCK

**Instance Segmentation**



CAT, DOG, DUCK

# 6.1 概述

## ➤ 基于神经网络的模式识别

◆ 序列识别：自动语音识别ASR、手写文字识别等

手写文字识别

*Foreign Minister.*

FOREIGN MINISTER.

自动语音识别ASR  
Speech recognition



THE SOUND OF

机器翻译

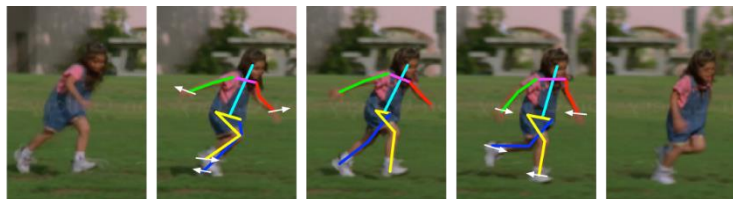
Machine translation

我是电子工程  
系的学生。

I am a student from  
the Department of  
Electronic Engineering.

行为识别

Activity recognition



Running

# 第6讲 序列建模：

## 深度学习方法

### 6.1 序列建模概述

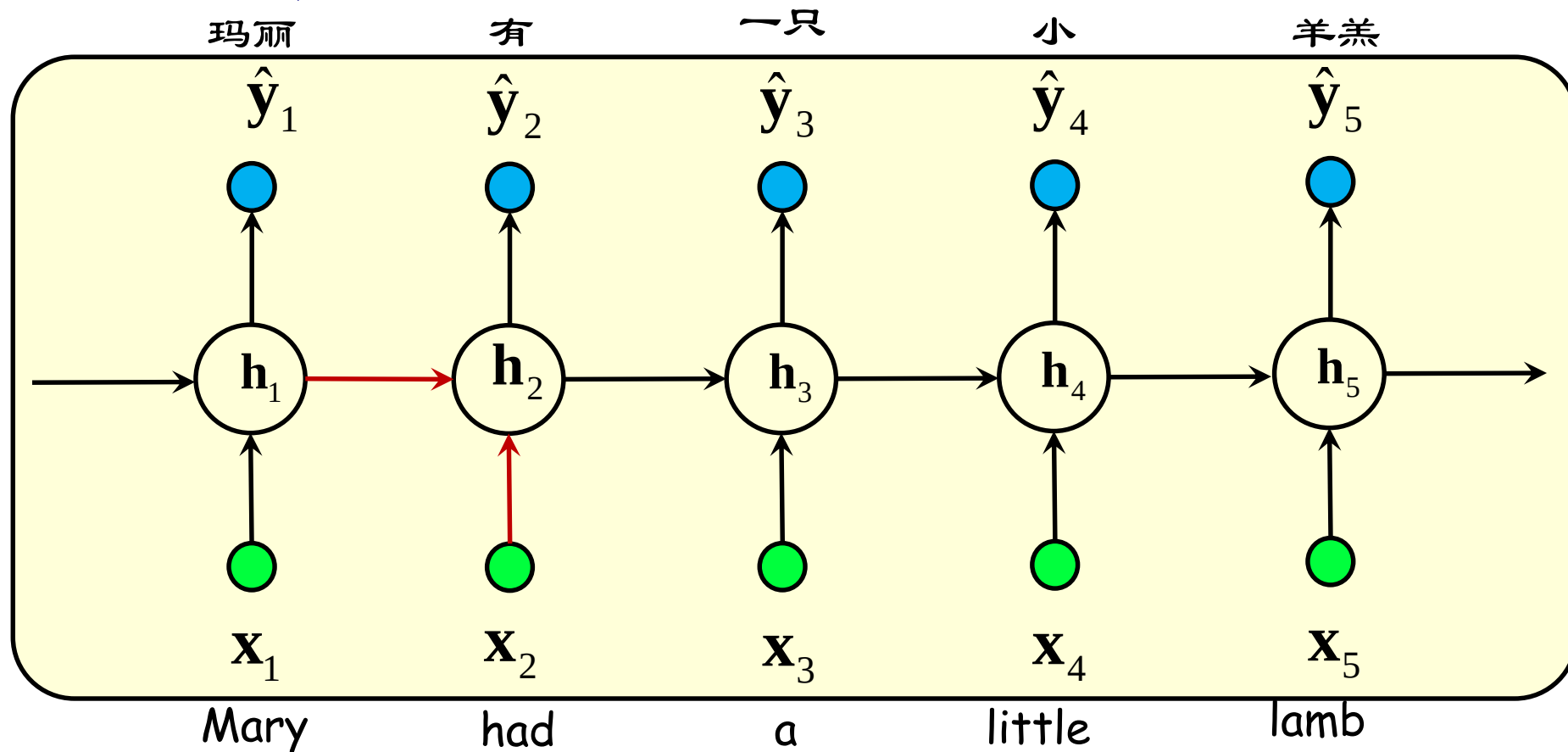
### 6.2 循环神经网络

### 6.3 TRANSFORMER

### 6.4 应用与展望

# 引子

## ► 回顾：循环神经网络RNN

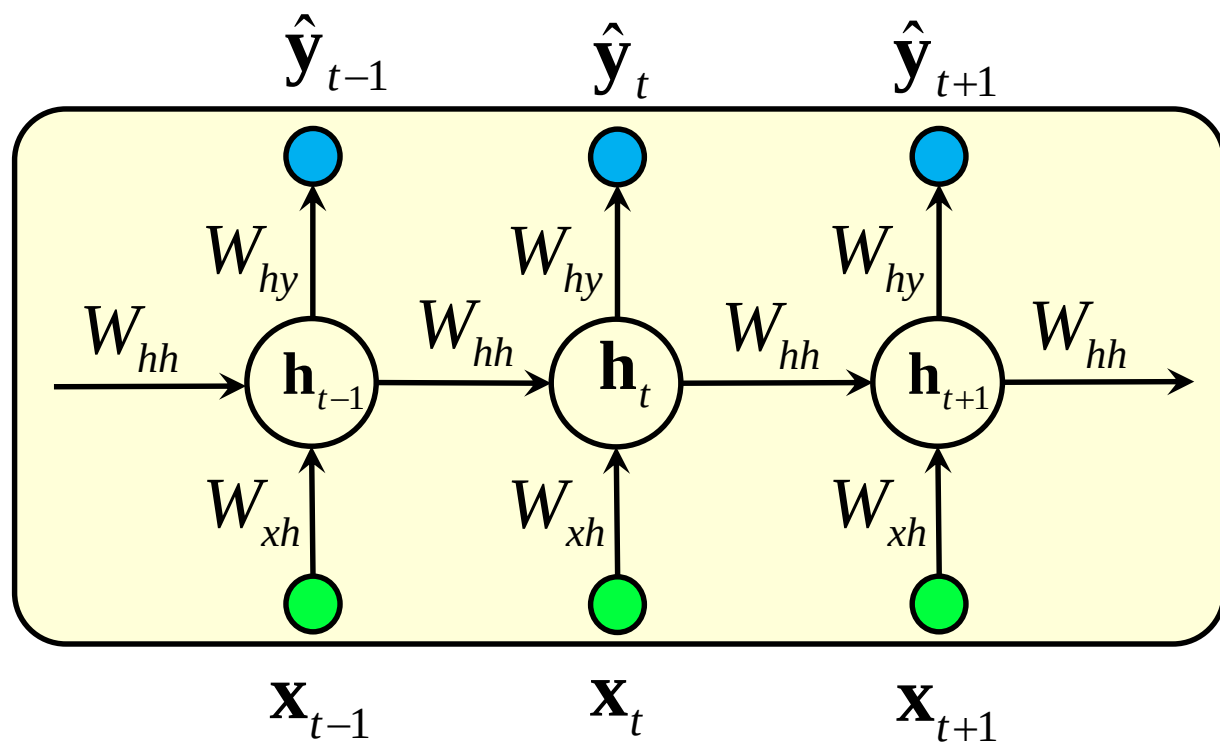


思考：RNN进行序列建模  
在计算上有什么缺点？

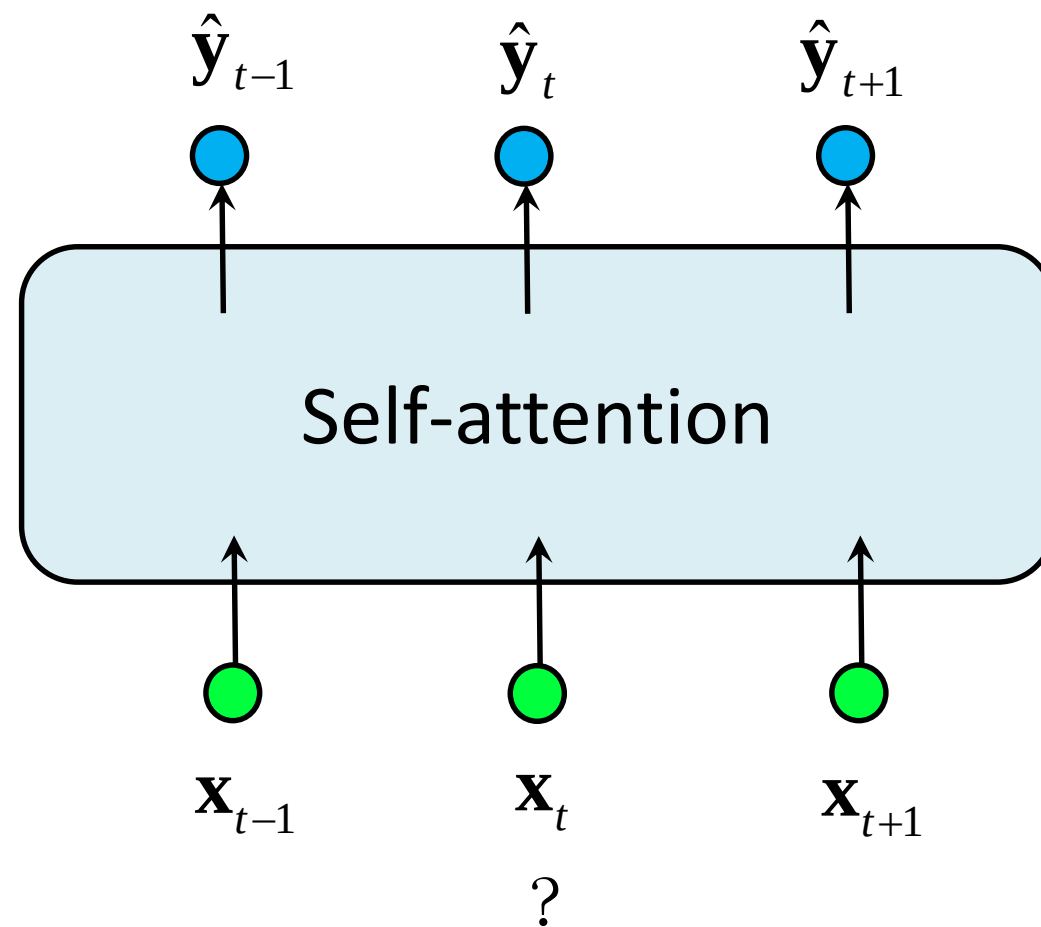
$$\mathbf{h}_t = \tanh(\mathbf{W}_h [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_h)$$

$$\hat{\mathbf{y}}_t = \mathbf{W}_{hy} \mathbf{h}_t + \mathbf{b}_y$$

# 引子

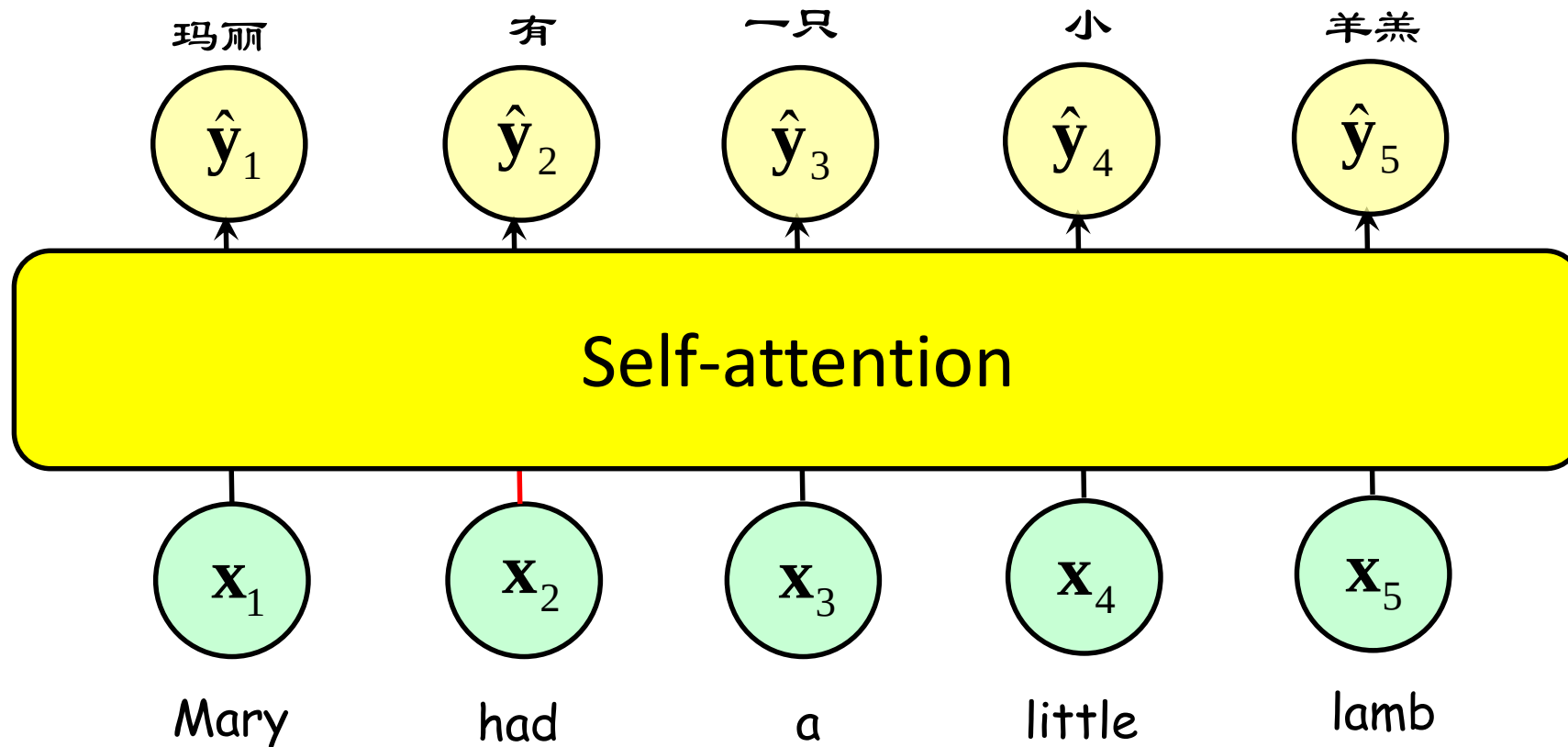


循环神经网络



## 6.3.1 自注意力机制 Self-attention

### ➤ Self-attention

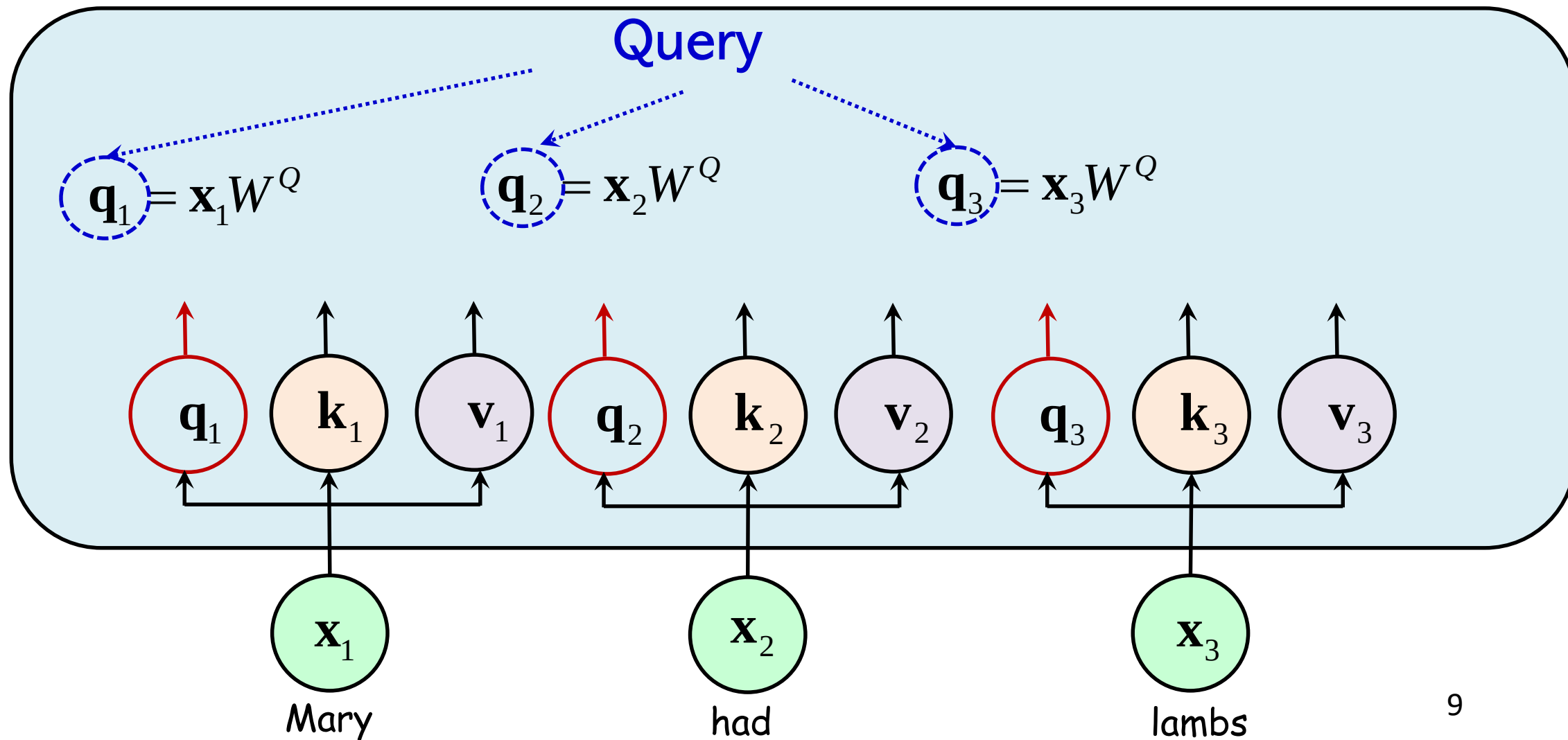


$$Q = XW^Q, K = XW^K, V = XW^V \quad \text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



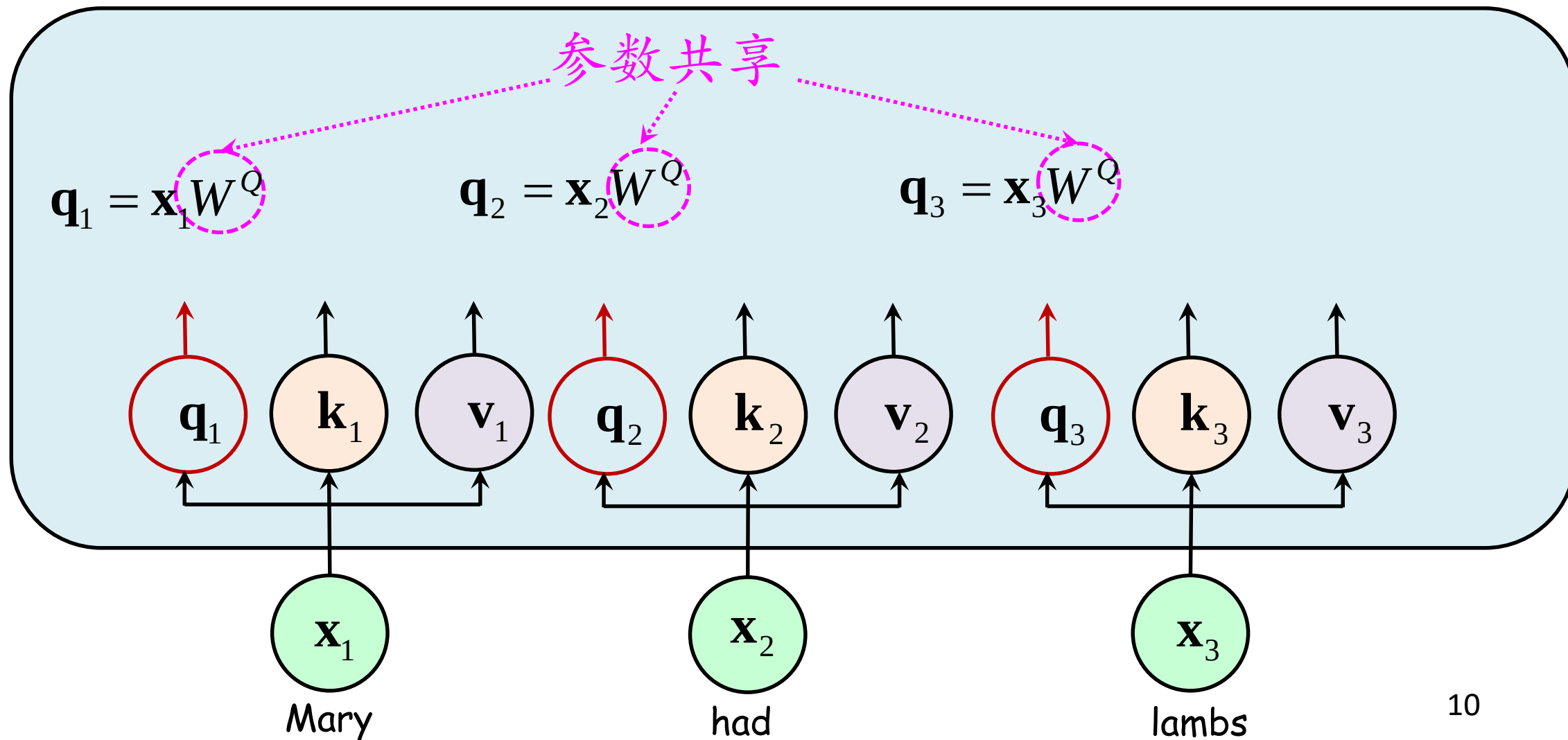
## 6.3.1 自注意力机制 Self-attention

### ➤ Self-attention



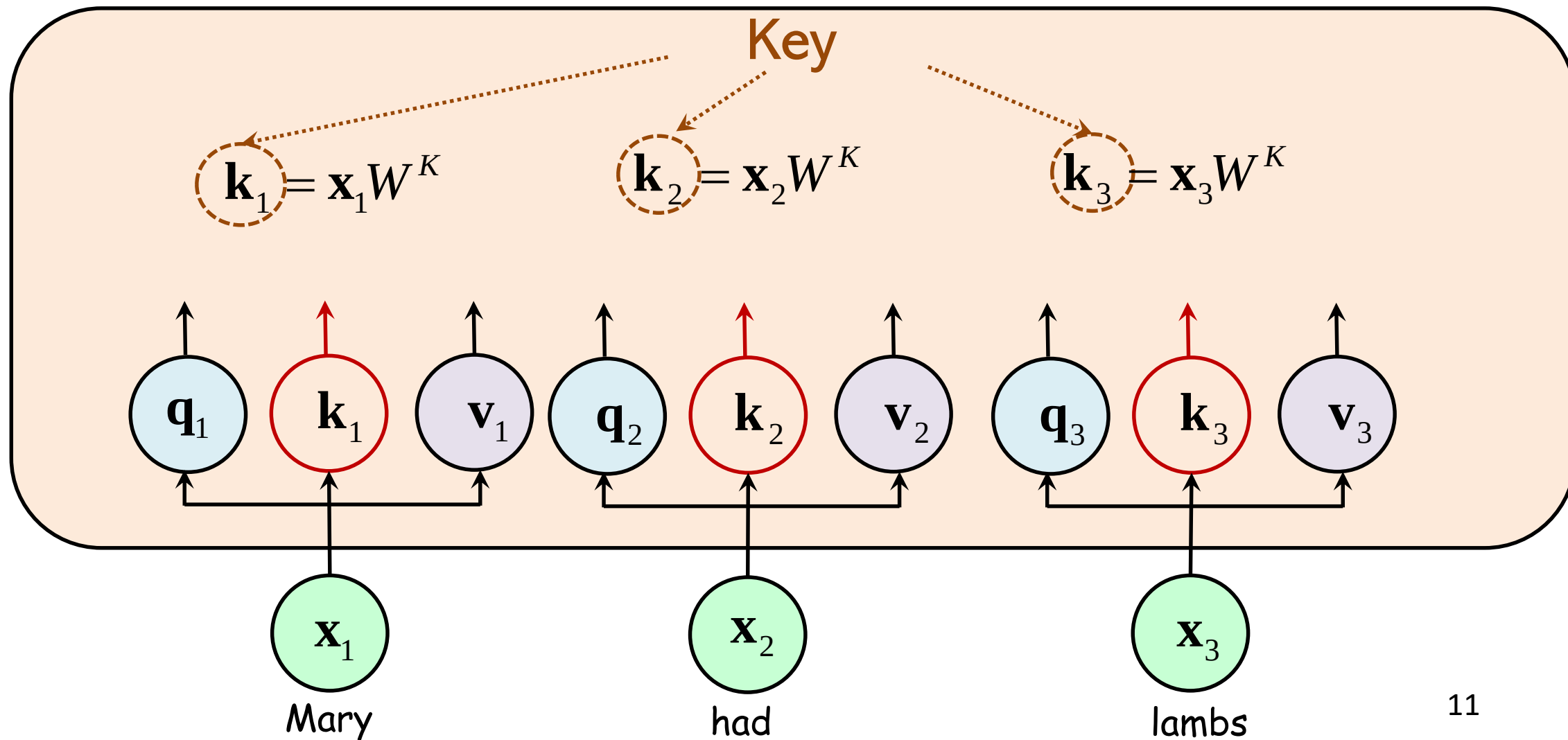
## 6.3.1 自注意力机制 Self-attention

### ➤ Self-attention



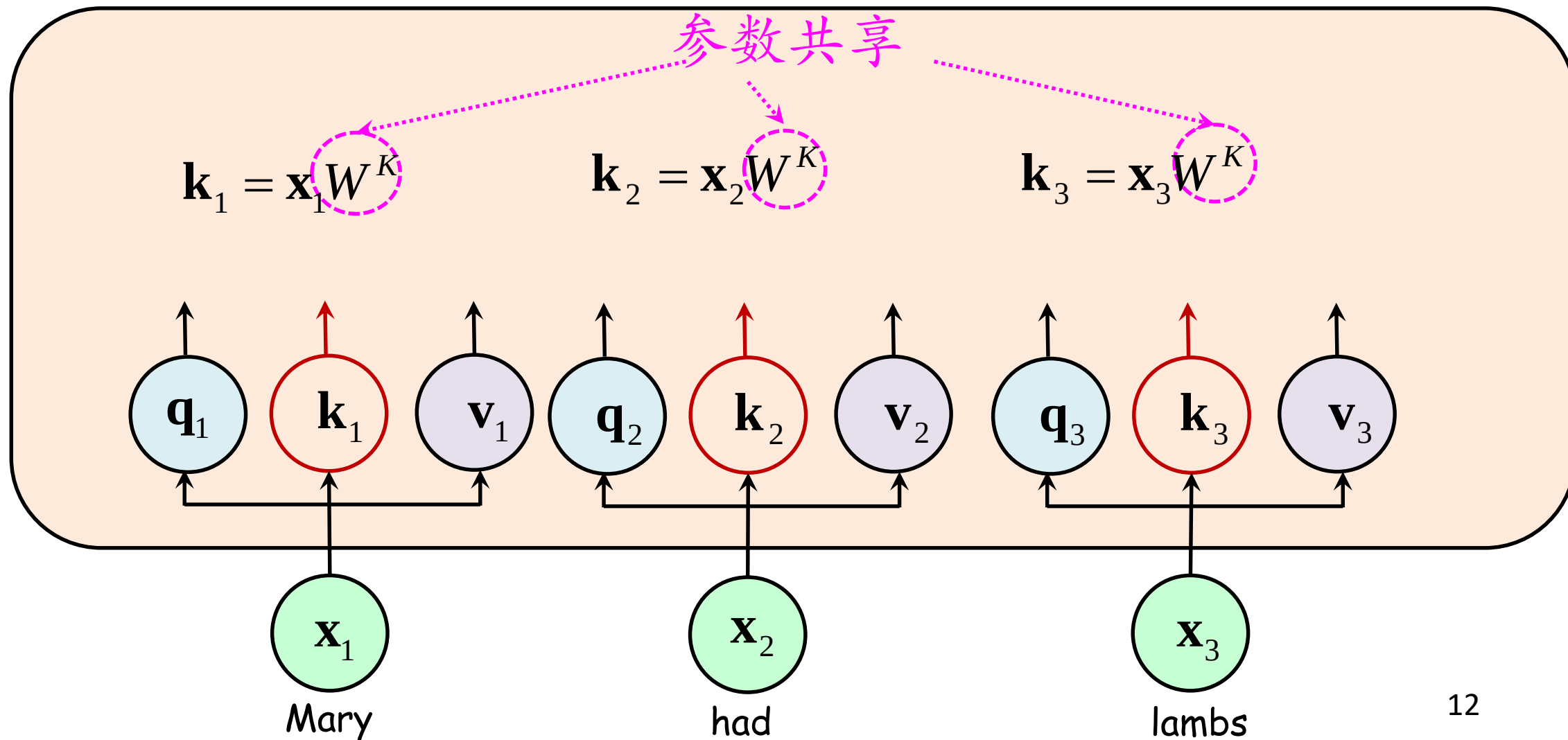
## 6.3.1 自注意力机制 Self-attention

### ➤ Self-attention



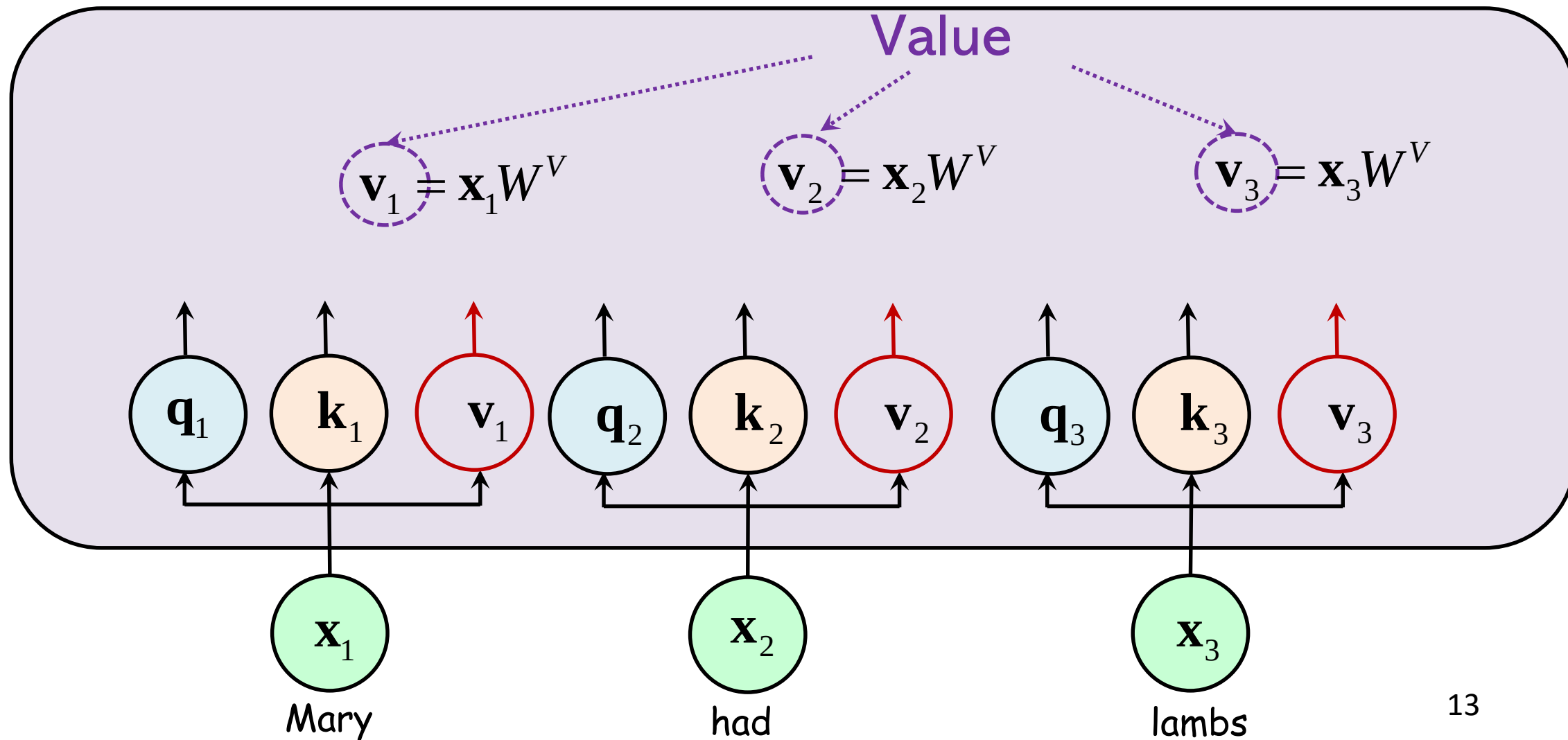
## 6.3.1 自注意力机制 Self-attention

### ➤ Self-attention



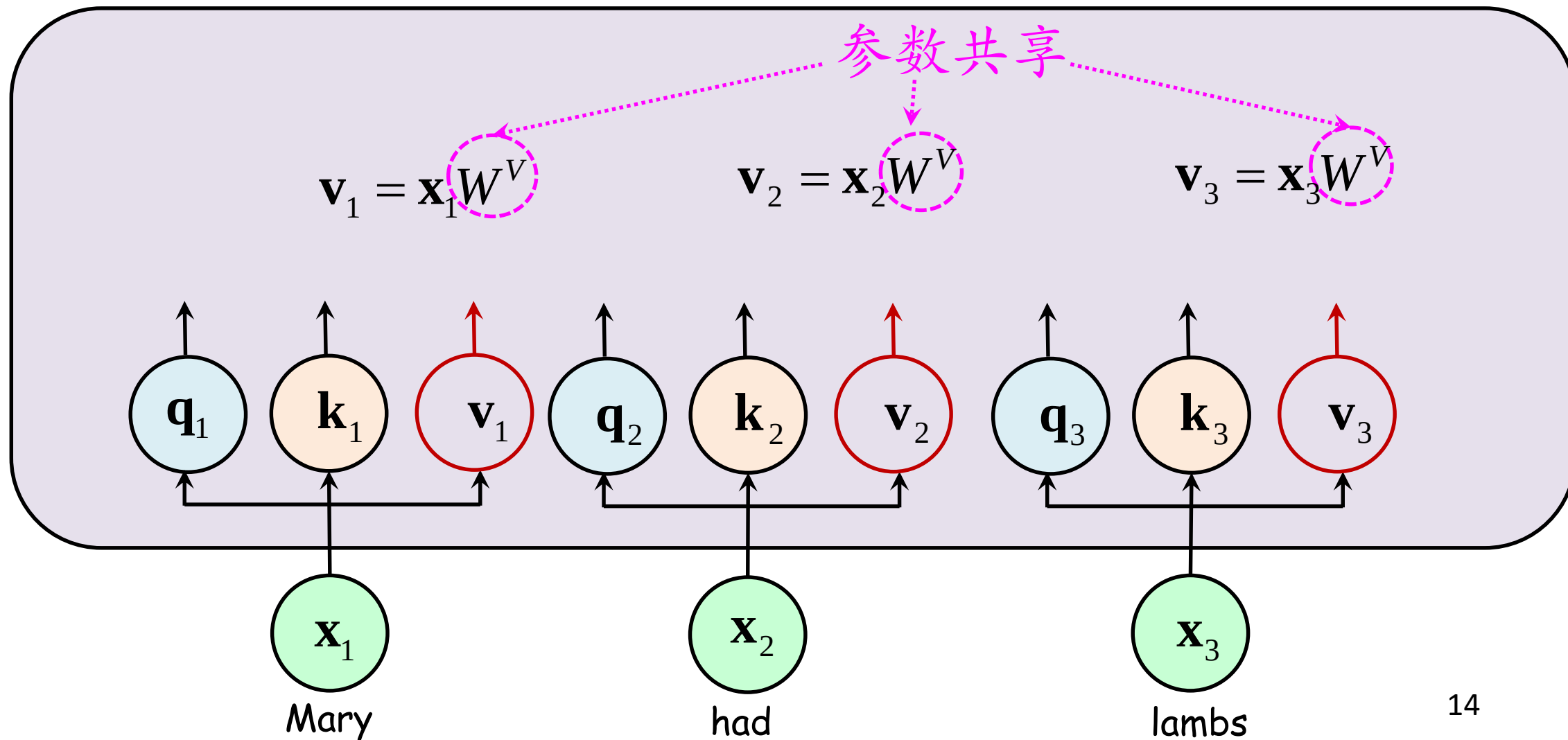
## 6.3.1 自注意力机制 Self-attention

### ➤ Self-attention



## 6.3.1 自注意力机制 Self-attention

### ➤ Self-attention



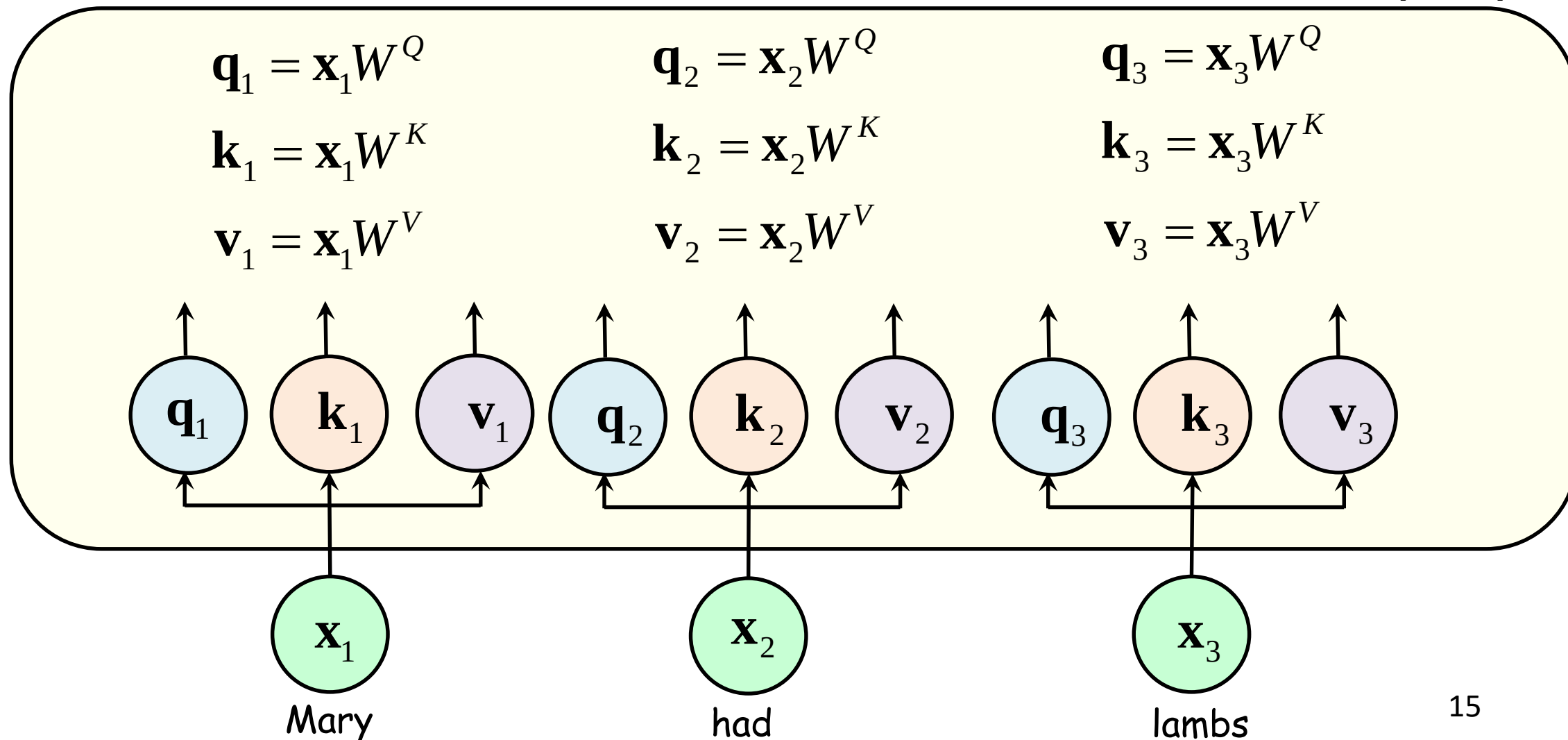
## 6.3.1 自注意力机制 Self-attention

### ➤ Self-attention

$$\mathbf{q}_i = \mathbf{x}_i W^Q$$

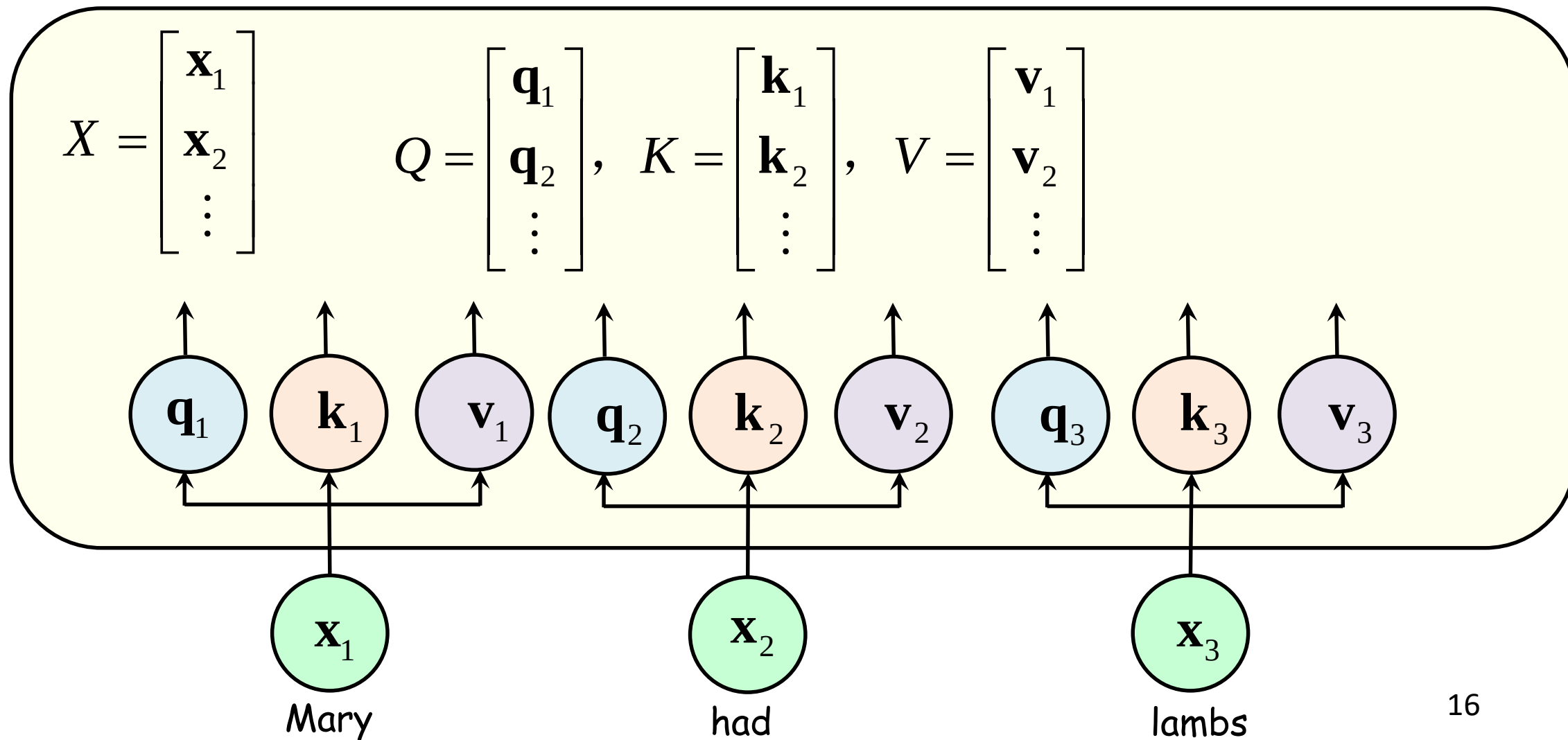
$$\mathbf{k}_i = \mathbf{x}_i W^K$$

$$\mathbf{v}_i = \mathbf{x}_i W^V$$



## 6.3.1 自注意力机制 Self-attention

### ➤ Self-attention

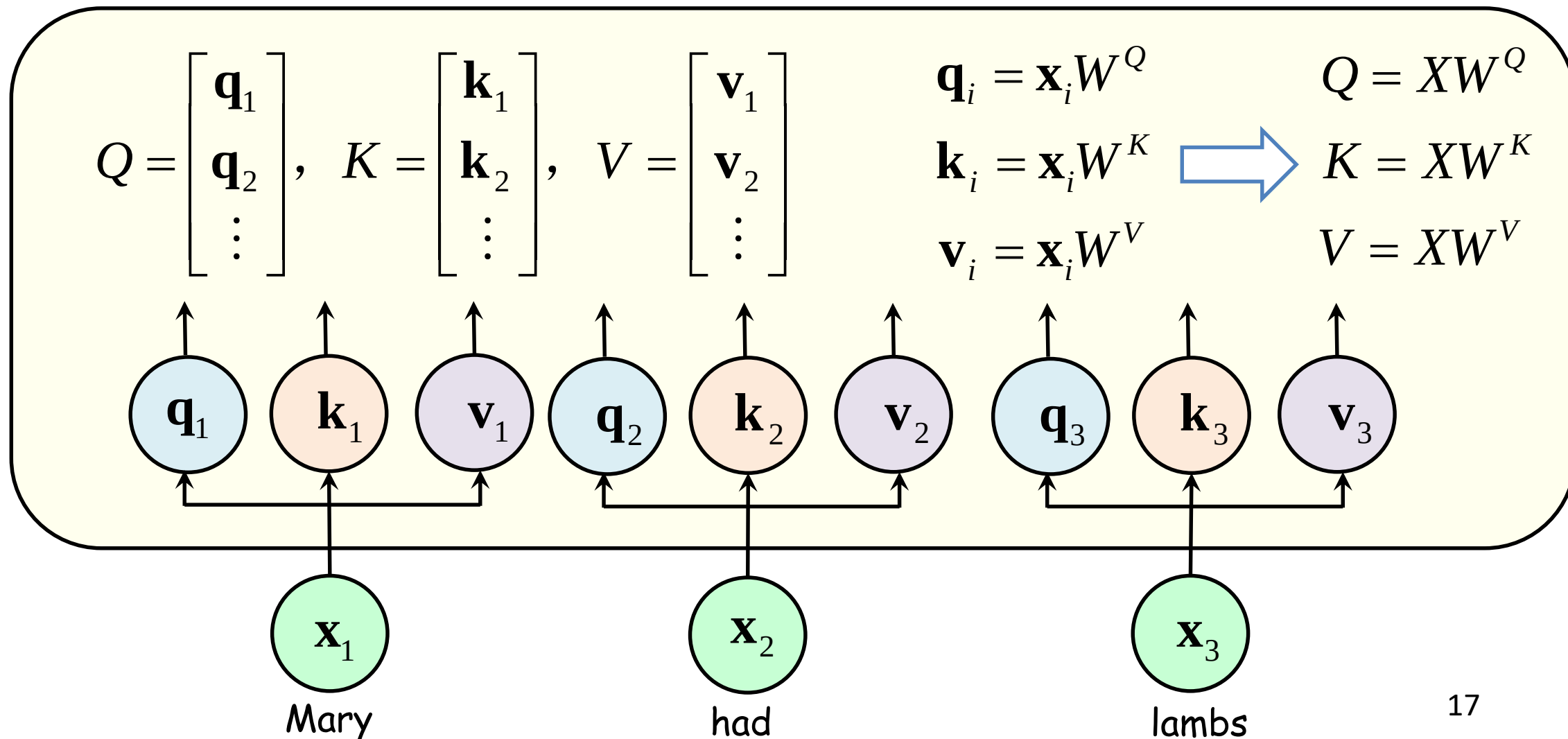




## 6.3.1 自注意力机制 Self-attention

### ➤ Self-attention

$$X = \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \vdots \end{bmatrix}$$



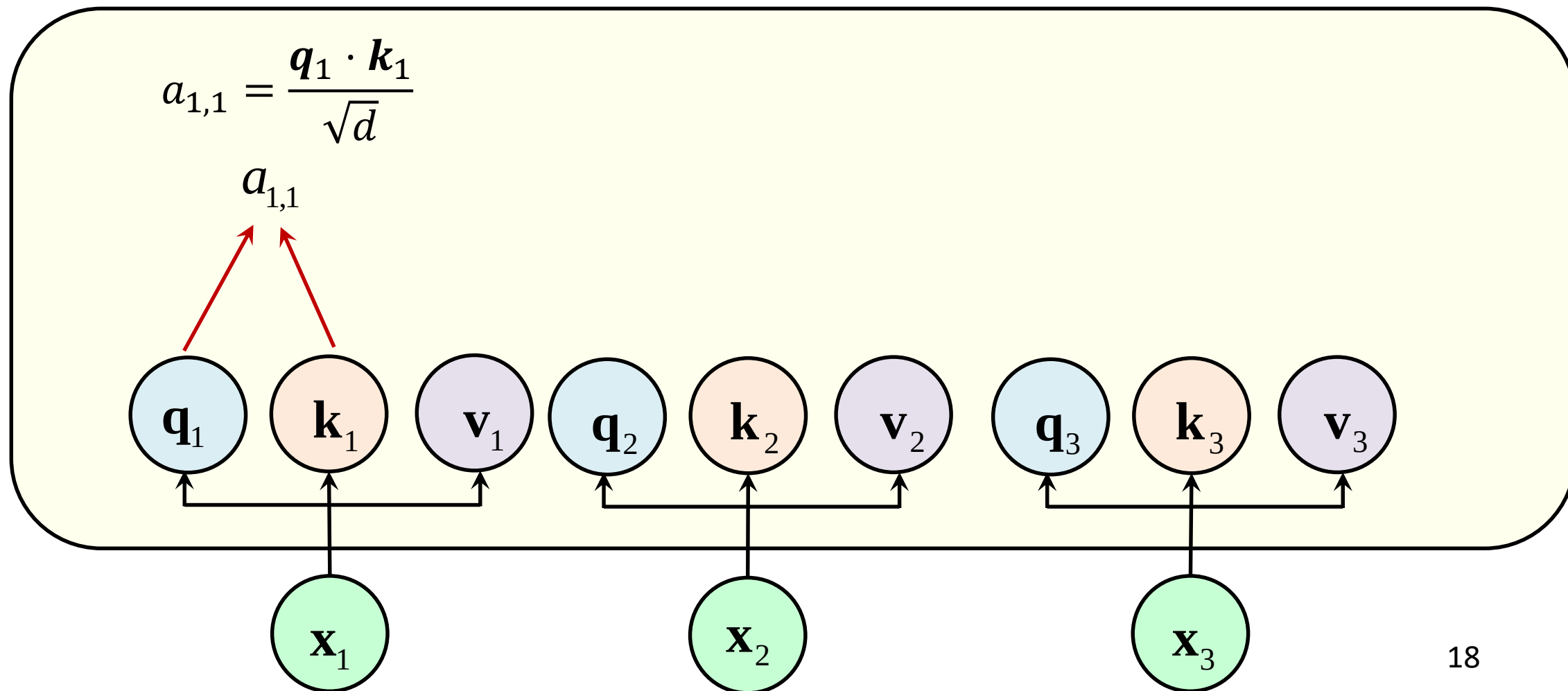
## 6.3.1 自注意力机制 Self-attention

$$Q = XW^Q$$

$$K = XW^K$$

$$V = XW^V$$

### ➤ Self-attention



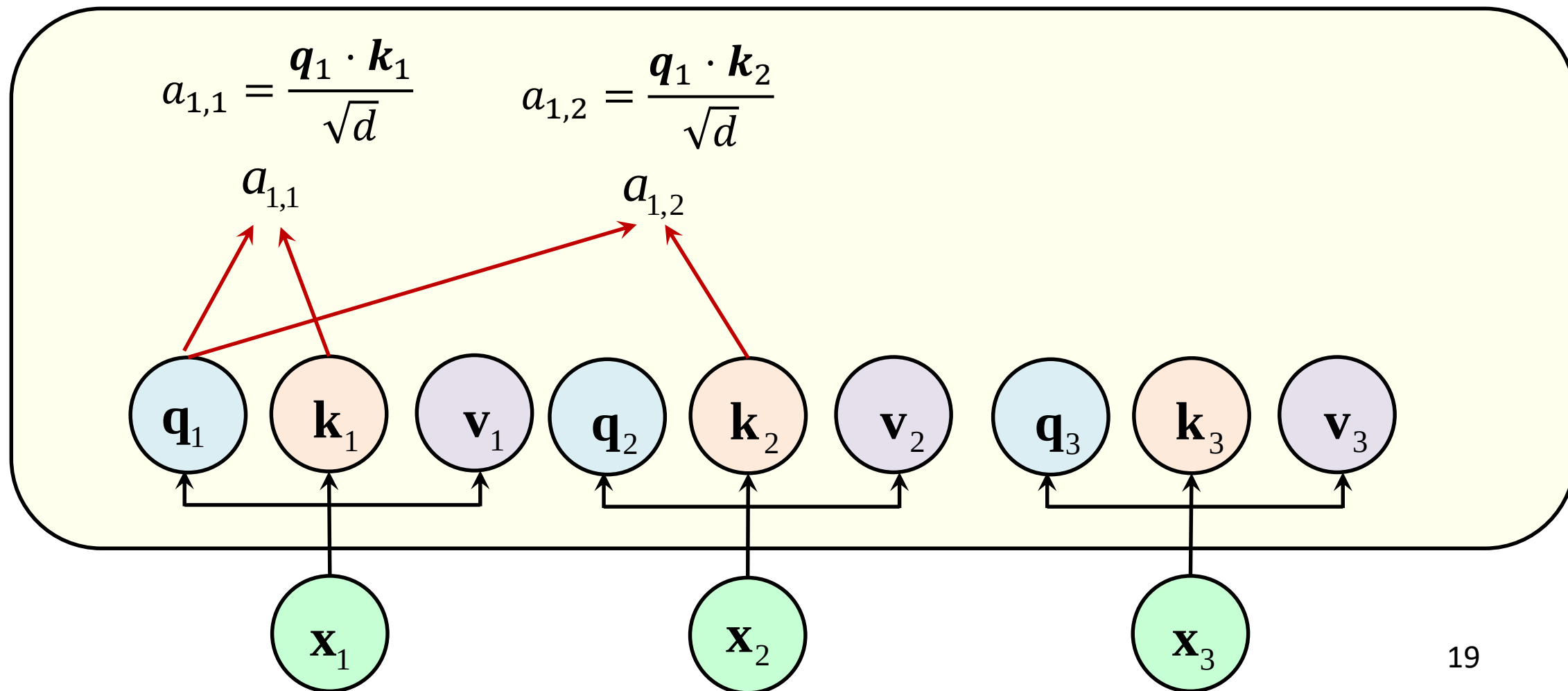
## 6.3.1 自注意力机制 Self-attention

$$Q = XW^Q$$

$$K = XW^K$$

$$V = XW^V$$

### ➤ Self-attention



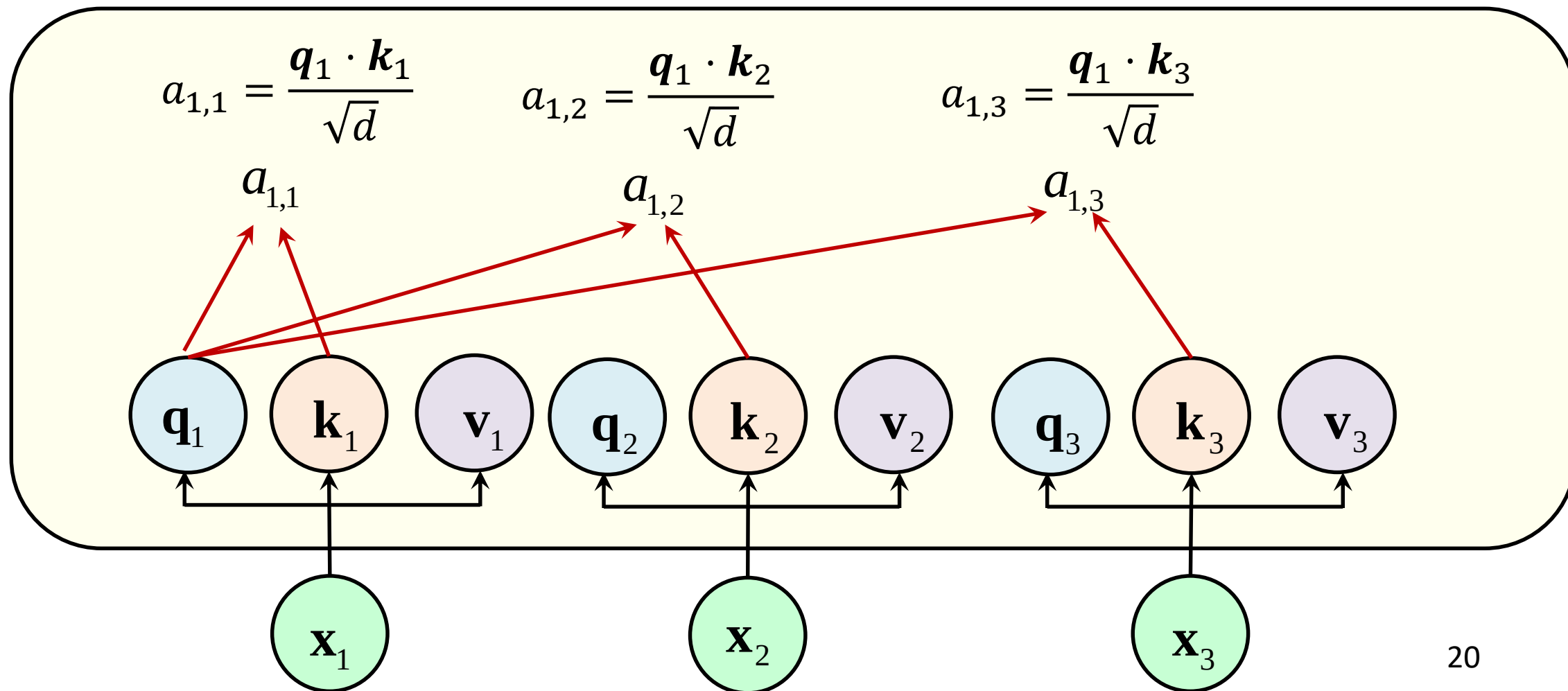
## 6.3.1 自注意力机制 Self-attention

### ➤ Self-attention

$$Q = XW^Q$$

$$K = XW^K$$

$$V = XW^V$$



## 6.3.1 自注意力机制 Self-attention

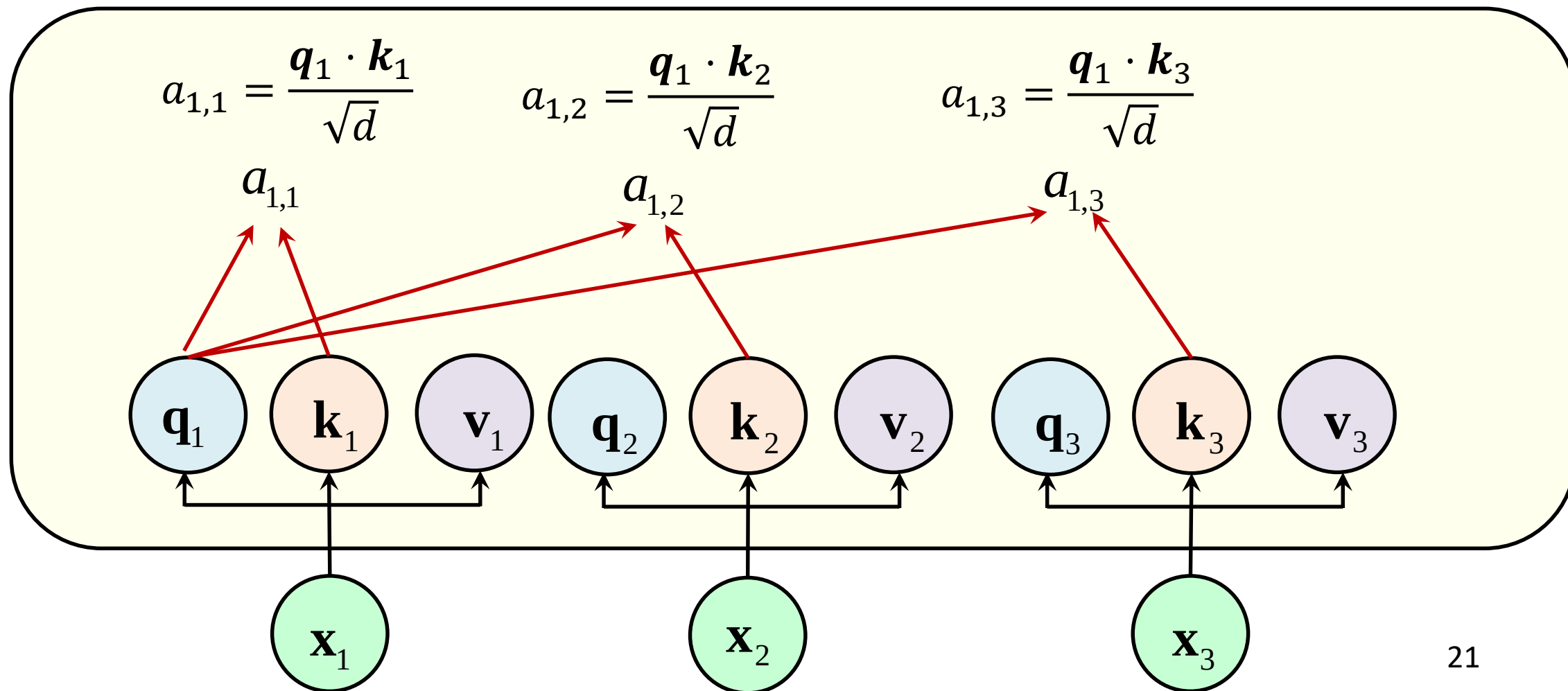
$$Q = XW^Q$$

$$K = XW^K$$

$$V = XW^V$$

### ➤ Self-attention

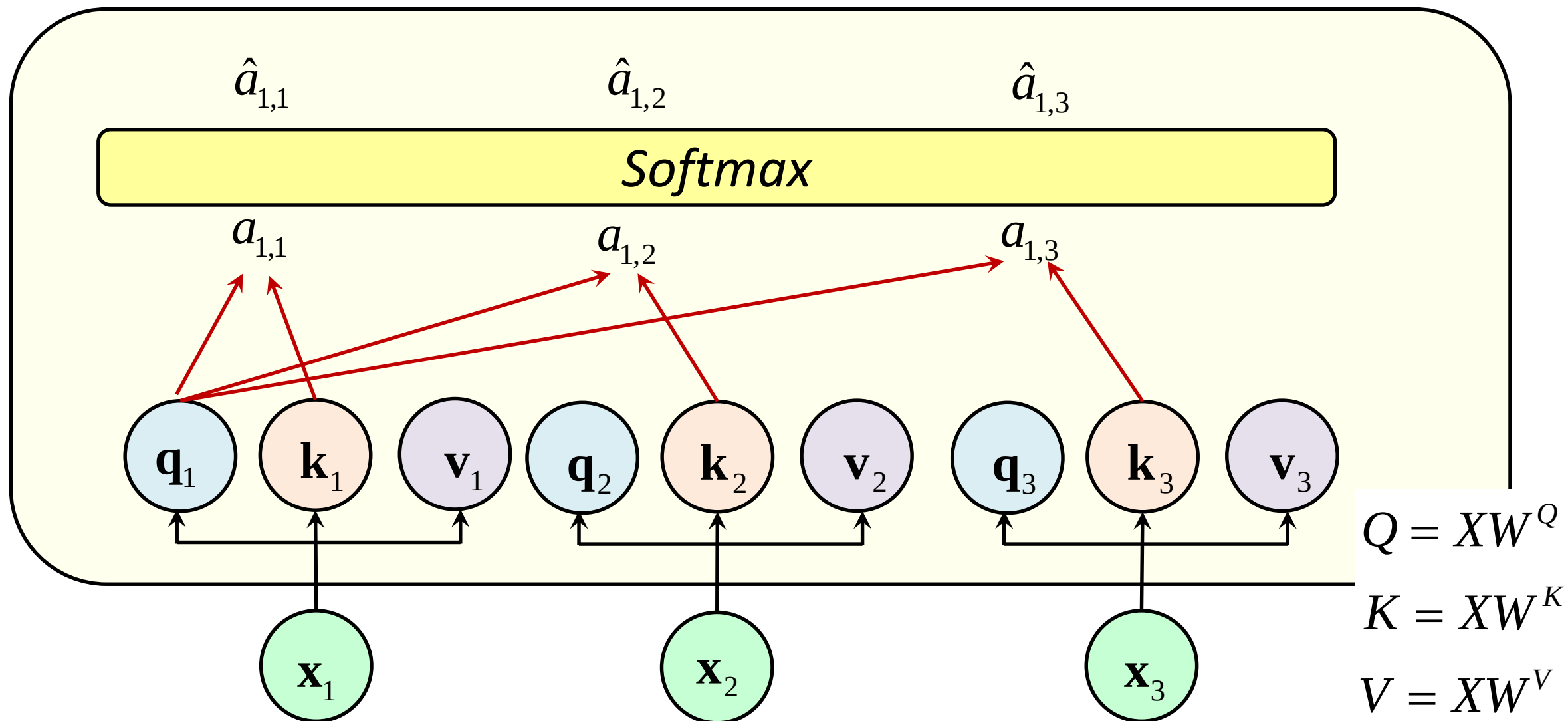
$$a_{1,j} = \frac{\mathbf{q}_1 \cdot \mathbf{k}_j}{\sqrt{d}}, j = 1, 2, \dots$$



## 6.3.1 自注意力机制 Self-attention

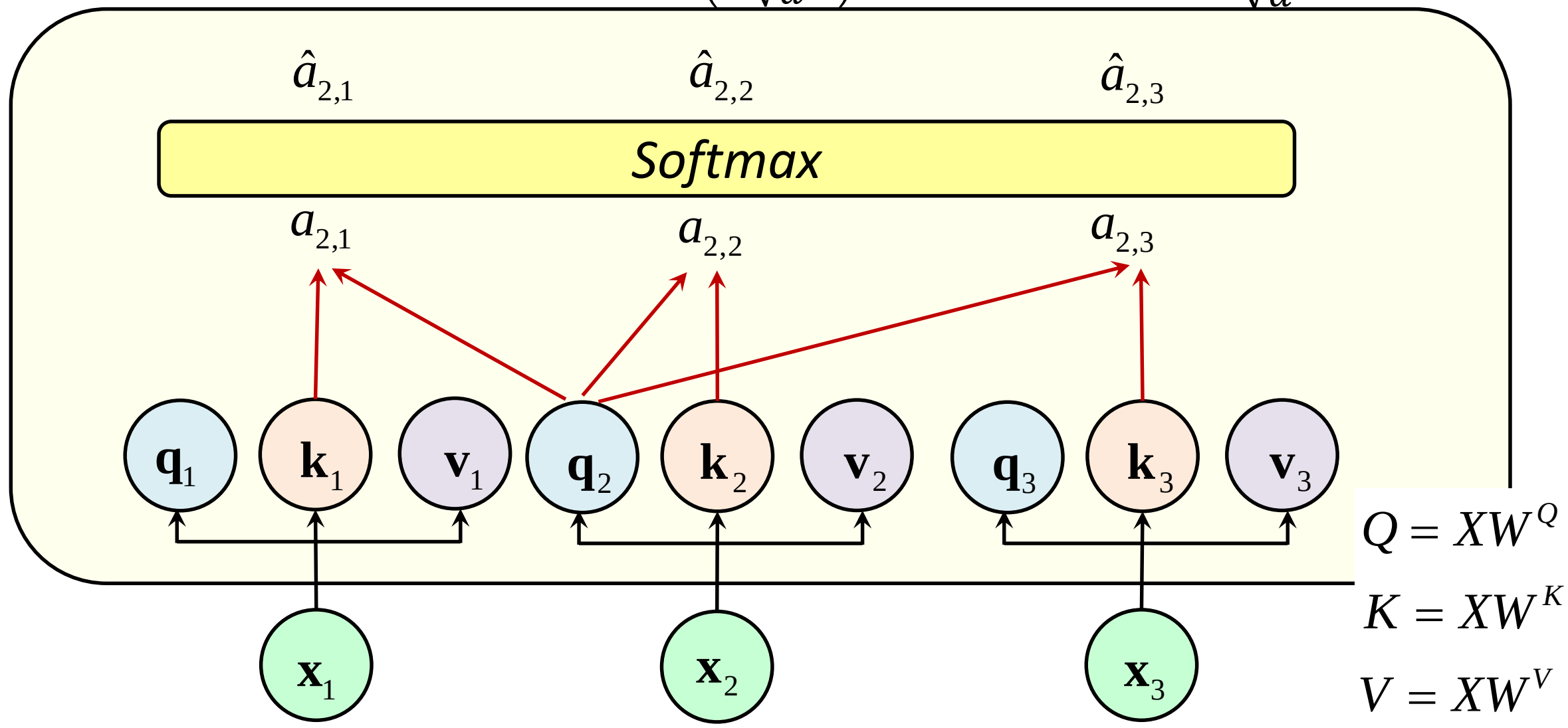
➤ Self-attention

$$\hat{a}_{1,j} = \text{Softmax}\left(\frac{\mathbf{q}_1 \cdot \mathbf{k}_j}{\sqrt{d}}\right), j = 1, 2, \dots \quad a_{1,j} = \frac{\mathbf{q}_1 \cdot \mathbf{k}_j}{\sqrt{d}}, j = 1, 2, \dots$$



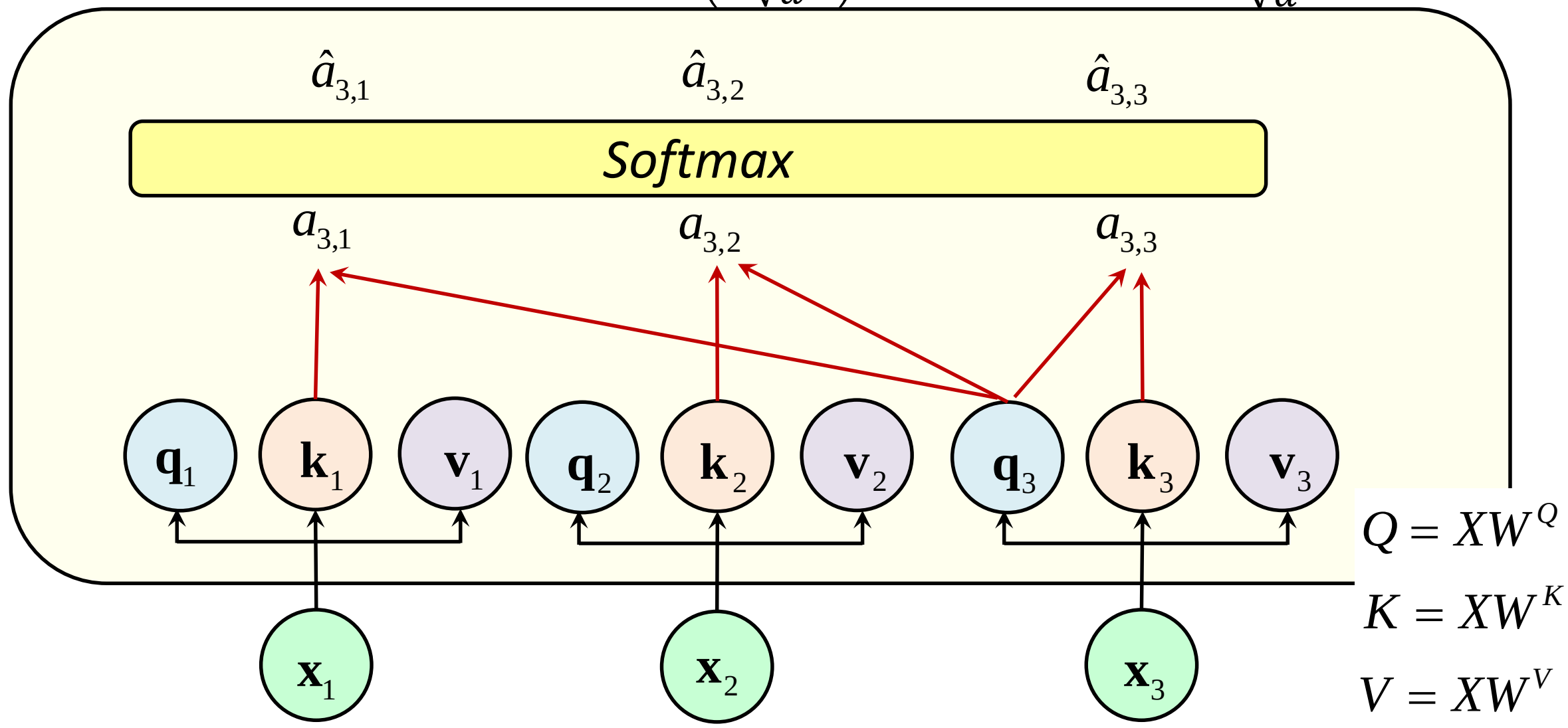
## 6.3.1 自注意力机制 Self-attention

➤ Self-attention  $\hat{a}_{2,j} = \text{Softmax}\left(\frac{\mathbf{q}_2 \cdot \mathbf{k}_j}{\sqrt{d}}\right), j = 1, 2, \dots$   $a_{2,j} = \frac{\mathbf{q}_2 \cdot \mathbf{k}_j}{\sqrt{d}}, j = 1, 2, \dots$



## 6.3.1 自注意力机制 Self-attention

➤ Self-attention  $\hat{a}_{3,j} = \text{Softmax}\left(\frac{\mathbf{q}_3 \cdot \mathbf{k}_j}{\sqrt{d}}\right), j = 1, 2, \dots$   $a_{3,j} = \frac{\mathbf{q}_3 \cdot \mathbf{k}_j}{\sqrt{d}}, j = 1, 2, \dots$

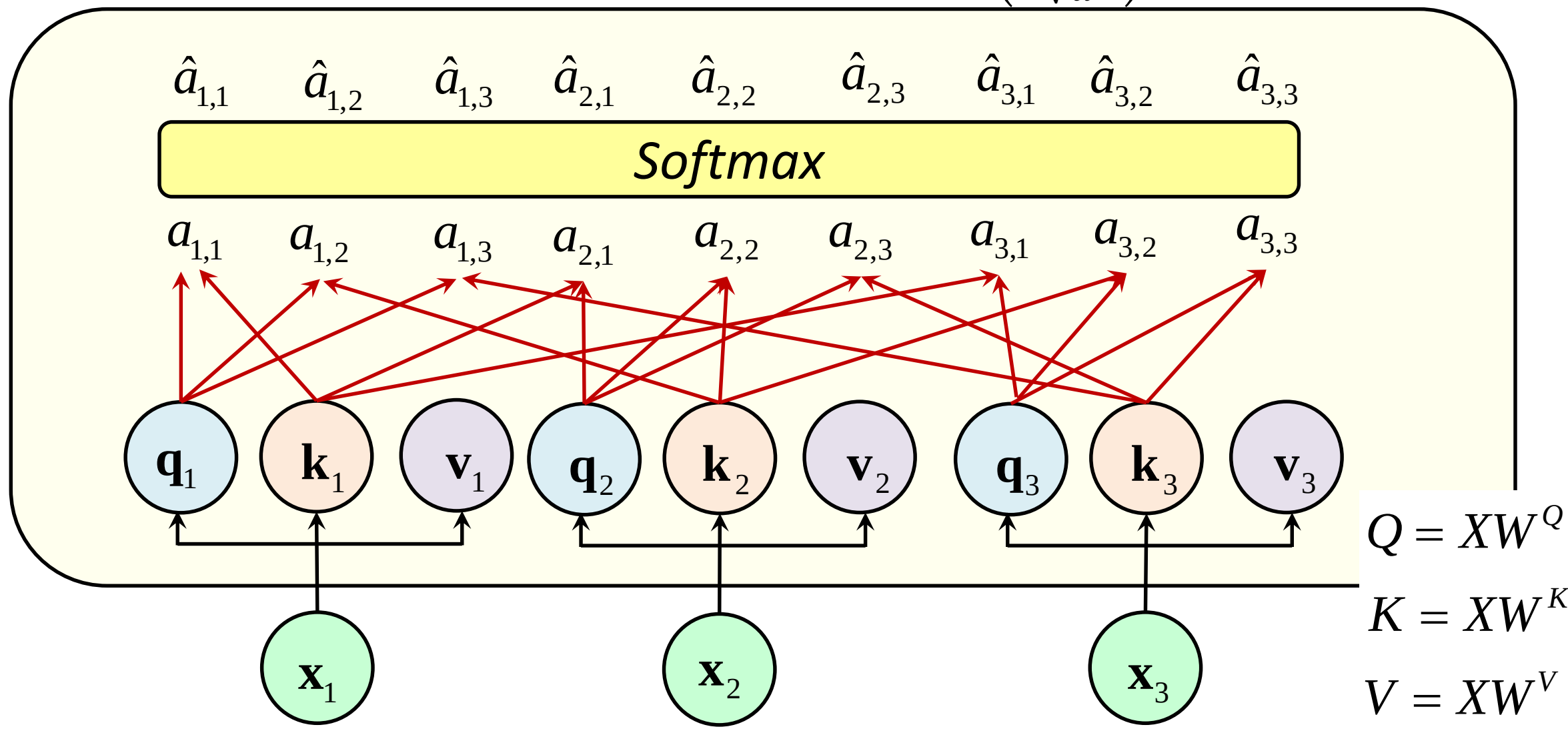




## 6.3.1 自注意力机制 Self-attention

### ➤ Self-attention

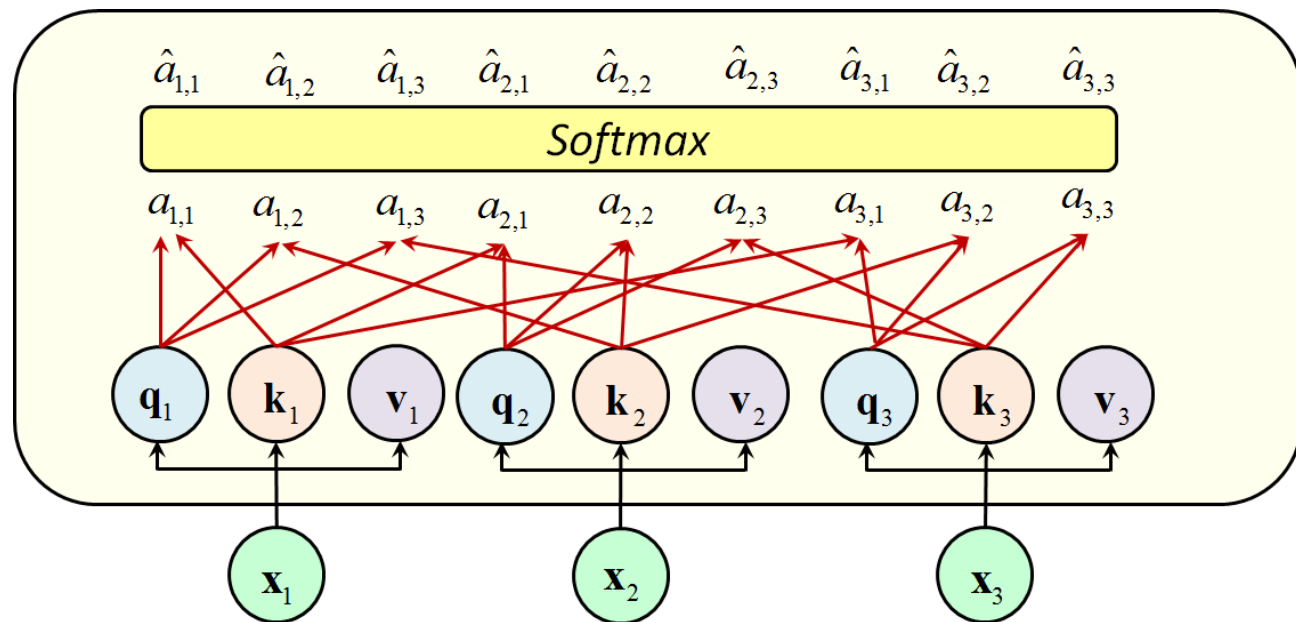
$$\hat{a}_{i,j} = \text{Softmax} \left( \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d}} \right), i, j = 1, 2, \dots$$



## 6.3.1 自注意力机制 Self-attention

➤ Self-attention  $\hat{a}_{i,j} = \text{Softmax}\left(\frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d}}\right), i, j = 1, 2, \dots$   $a_{i,j} = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d}}, i, j = 1, 2, \dots$

$$Q = \begin{bmatrix} \mathbf{q}_1 \\ \mathbf{q}_2 \\ \vdots \end{bmatrix}, \quad K = \begin{bmatrix} \mathbf{k}_1 \\ \mathbf{k}_2 \\ \vdots \end{bmatrix}, \quad V = \begin{bmatrix} \mathbf{v}_1 \\ \mathbf{v}_2 \\ \vdots \end{bmatrix}$$



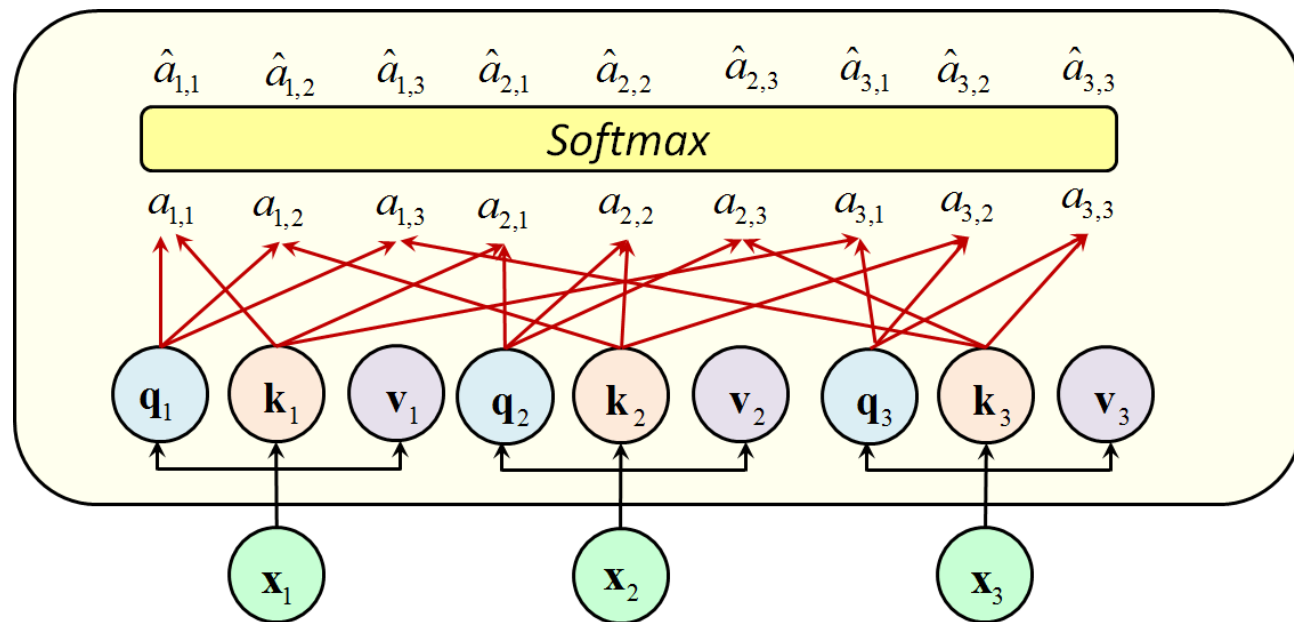
$$A = \{a_{i,j}\} = \begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,1} & a_{2,2} & a_{2,3} \\ a_{3,1} & a_{3,2} & a_{3,3} \end{bmatrix}$$

$$= \frac{1}{\sqrt{d}} \begin{bmatrix} \mathbf{q}_1 \\ \mathbf{q}_2 \\ \mathbf{q}_3 \end{bmatrix} \begin{bmatrix} (\mathbf{k}_1)^T & (\mathbf{k}_2)^T & (\mathbf{k}_3)^T \end{bmatrix} = \frac{QK^T}{\sqrt{d}}$$

## 6.3.1 自注意力机制 Self-attention

➤ Self-attention  $\hat{a}_{i,j} = \text{Softmax}\left(\frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d}}\right), i, j = 1, 2, \dots$   $a_{i,j} = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d}}, i, j = 1, 2, \dots$

$$Q = \begin{bmatrix} \mathbf{q}_1 \\ \mathbf{q}_2 \\ \vdots \end{bmatrix}, \quad K = \begin{bmatrix} \mathbf{k}_1 \\ \mathbf{k}_2 \\ \vdots \end{bmatrix}, \quad V = \begin{bmatrix} \mathbf{v}_1 \\ \mathbf{v}_2 \\ \vdots \end{bmatrix}$$



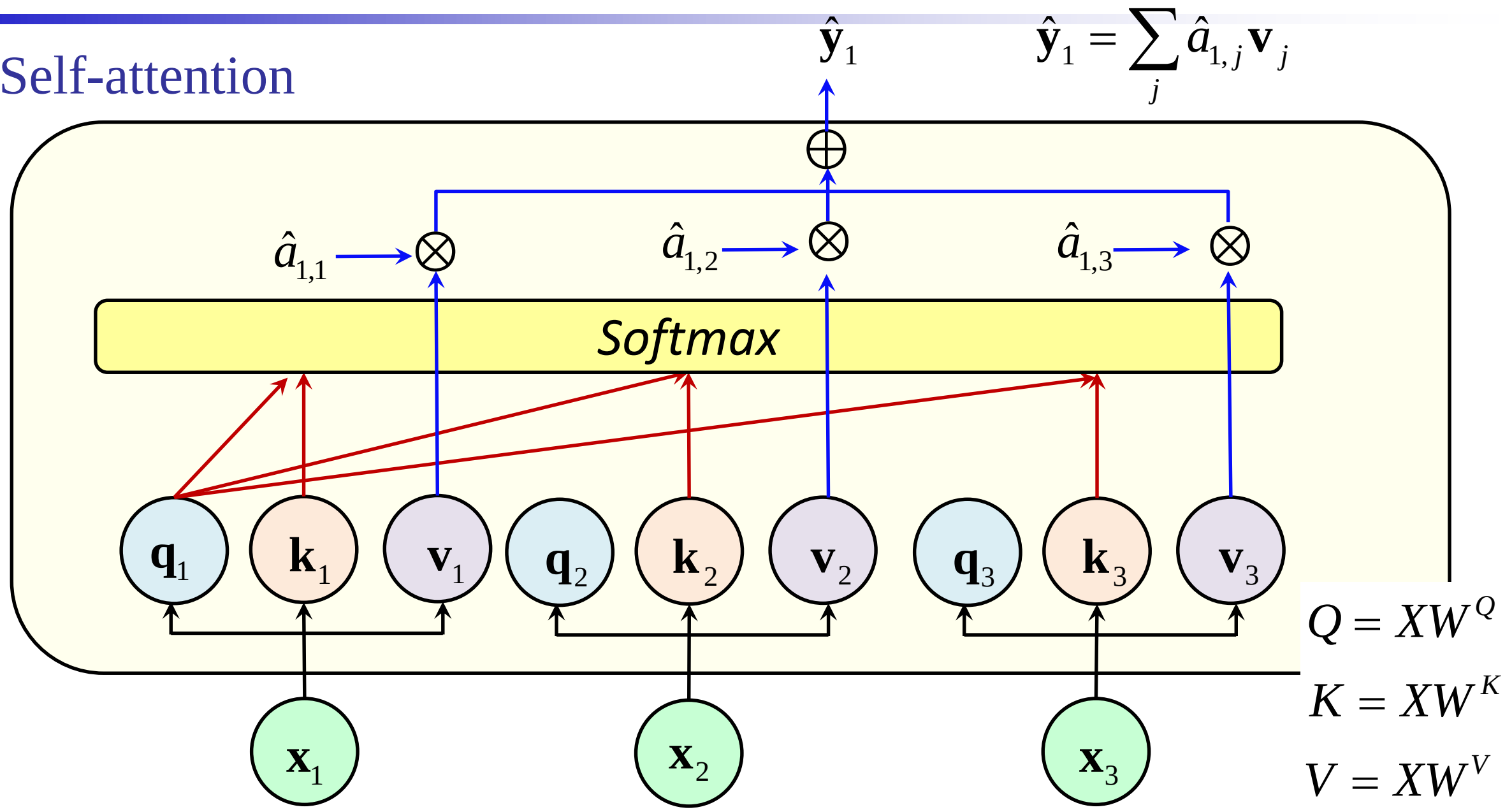
$$\hat{A} = \{\hat{a}_{i,j}\} = \begin{bmatrix} \hat{a}_{1,1} & \hat{a}_{1,2} & \hat{a}_{1,3} \\ \hat{a}_{2,1} & \hat{a}_{2,2} & \hat{a}_{2,3} \\ \hat{a}_{3,1} & \hat{a}_{3,2} & \hat{a}_{3,3} \end{bmatrix}$$

$$= \text{Softmax}\left(\frac{QK^T}{\sqrt{d}}\right)$$

每个行向量进行Softmax运算

## 6.3.1 自注意力机制 Self-attention

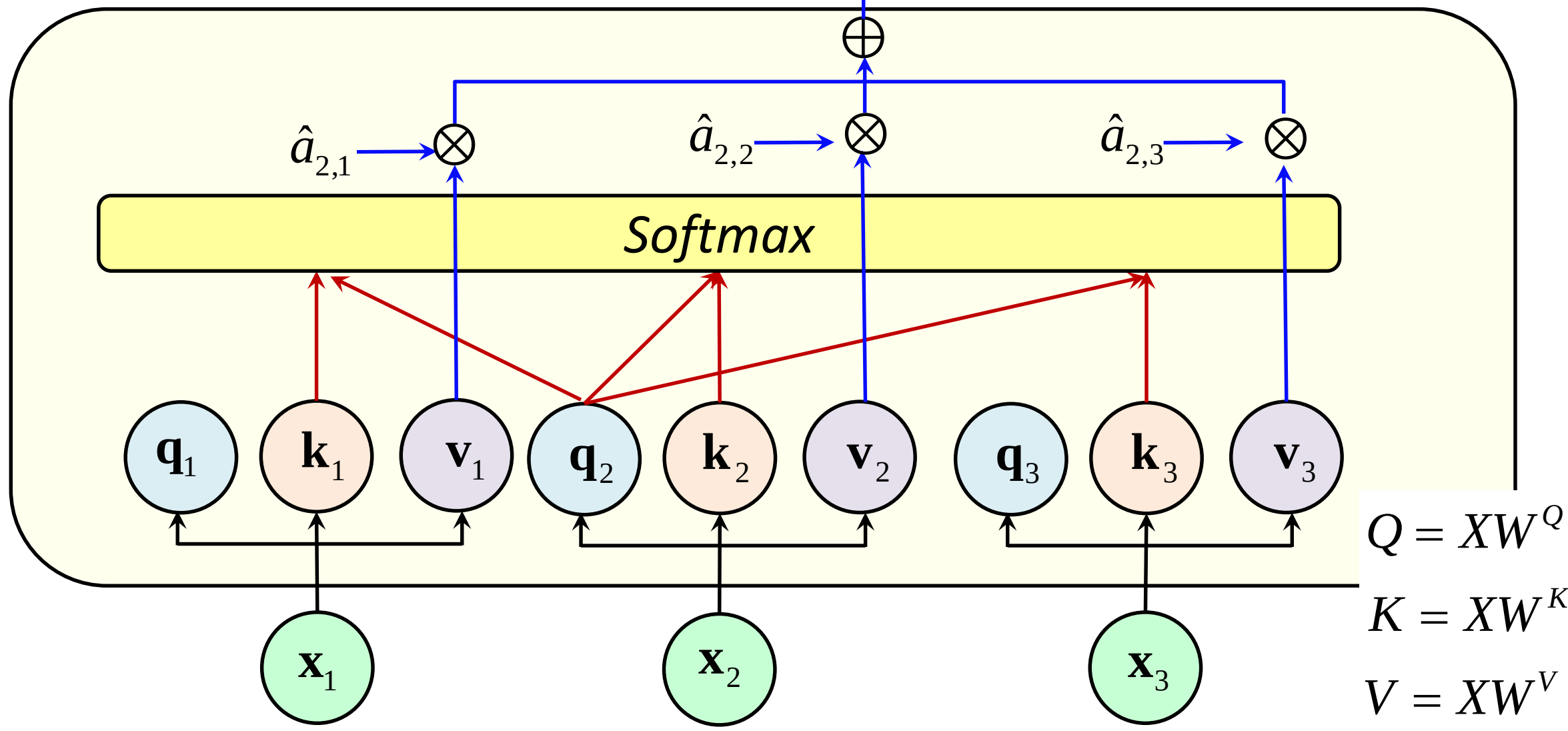
### ➤ Self-attention



## 6.3.1 自注意力机制 Self-attention

### ➤ Self-attention

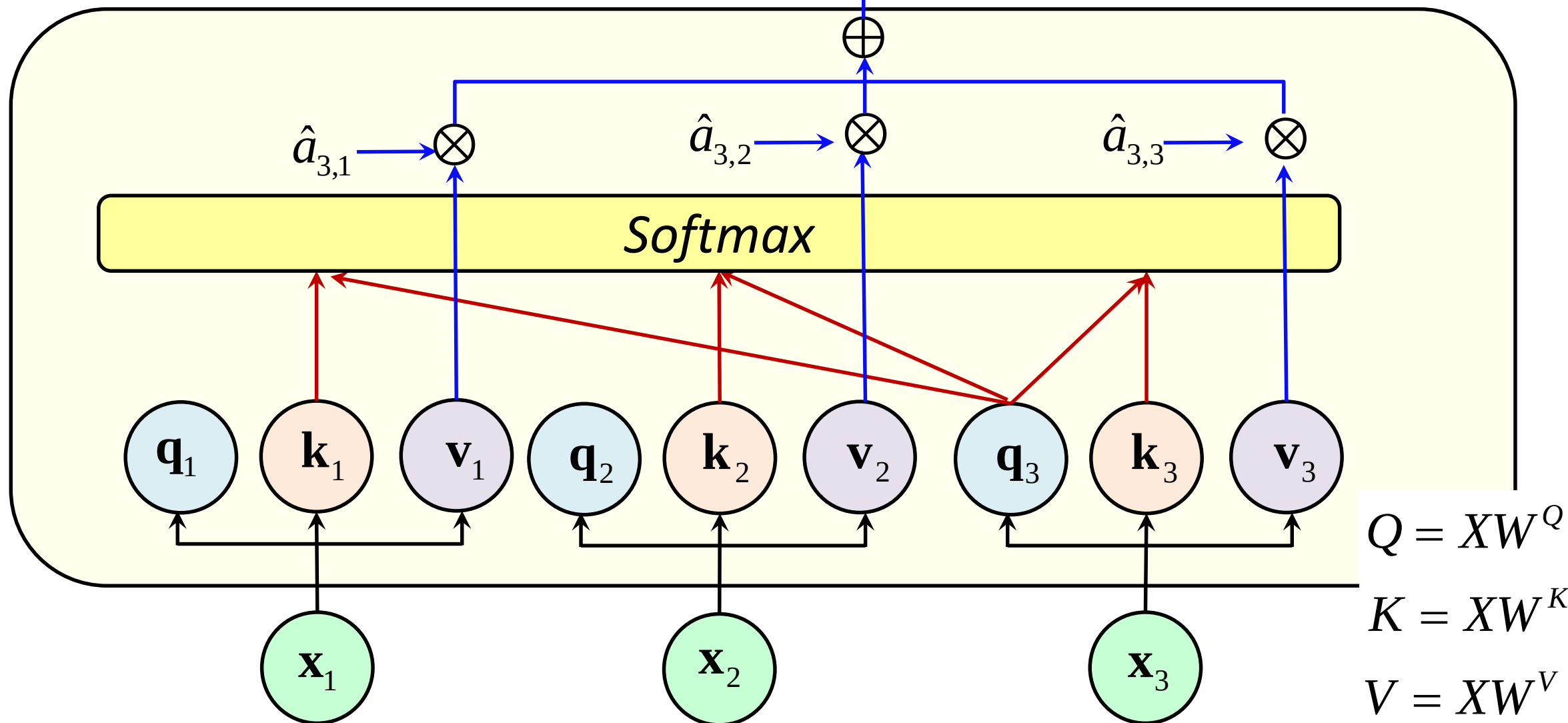
$$\hat{\mathbf{y}}_2 = \sum_j \hat{a}_{2,j} \mathbf{v}_j$$



## 6.3.1 自注意力机制 Self-attention

### ➤ Self-attention

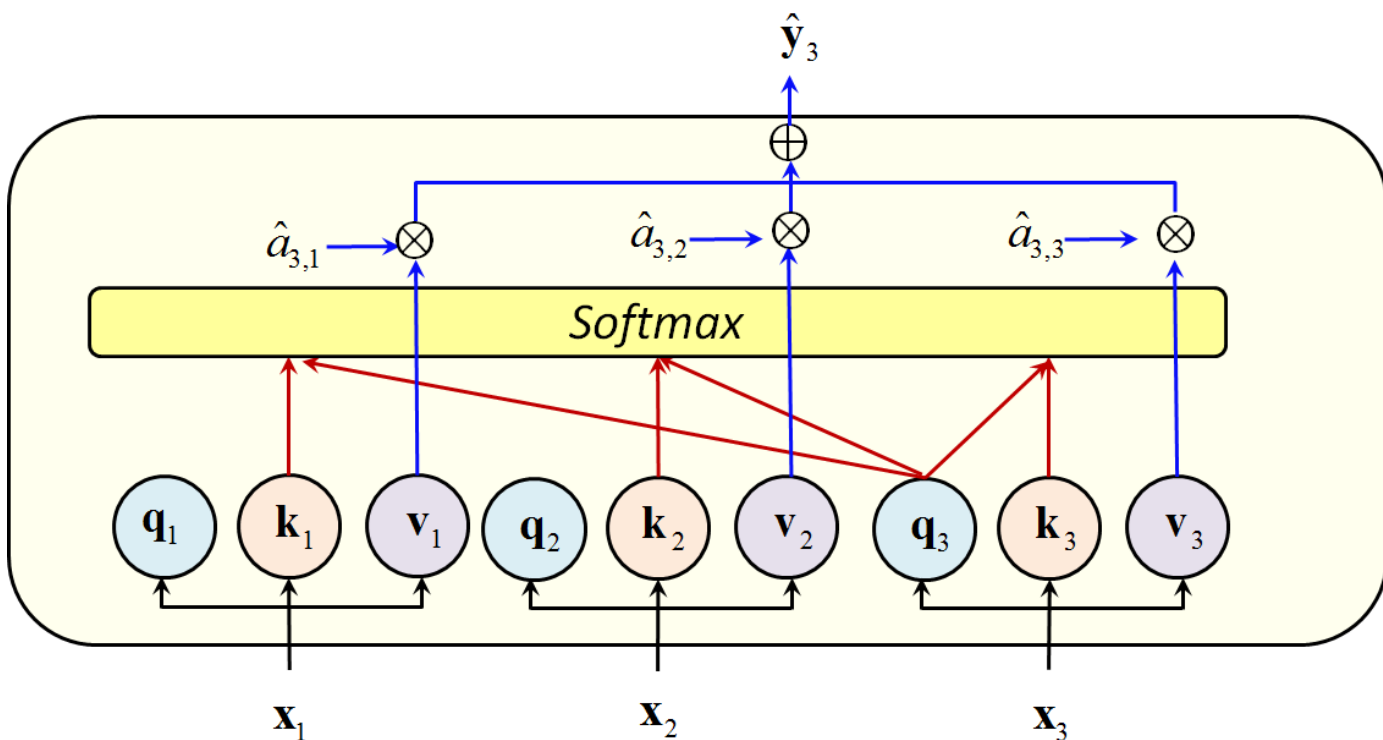
$$\hat{\mathbf{y}}_3 = \sum_j \hat{a}_{3,j} \mathbf{v}_j$$



## 6.3.1 自注意力机制 Self-attention

### ➤ Self-attention

$$\hat{\mathbf{y}}_i = \sum_j \hat{a}_{i,j} \mathbf{v}_j \quad Q = \begin{bmatrix} \mathbf{q}_1 \\ \mathbf{q}_2 \\ \vdots \end{bmatrix}, \quad K = \begin{bmatrix} \mathbf{k}_1 \\ \mathbf{k}_2 \\ \vdots \end{bmatrix}, \quad V = \begin{bmatrix} \mathbf{v}_1 \\ \mathbf{v}_2 \\ \vdots \end{bmatrix}, \quad \hat{Y} = \begin{bmatrix} \hat{\mathbf{y}}_1 \\ \hat{\mathbf{y}}_2 \\ \vdots \end{bmatrix}$$

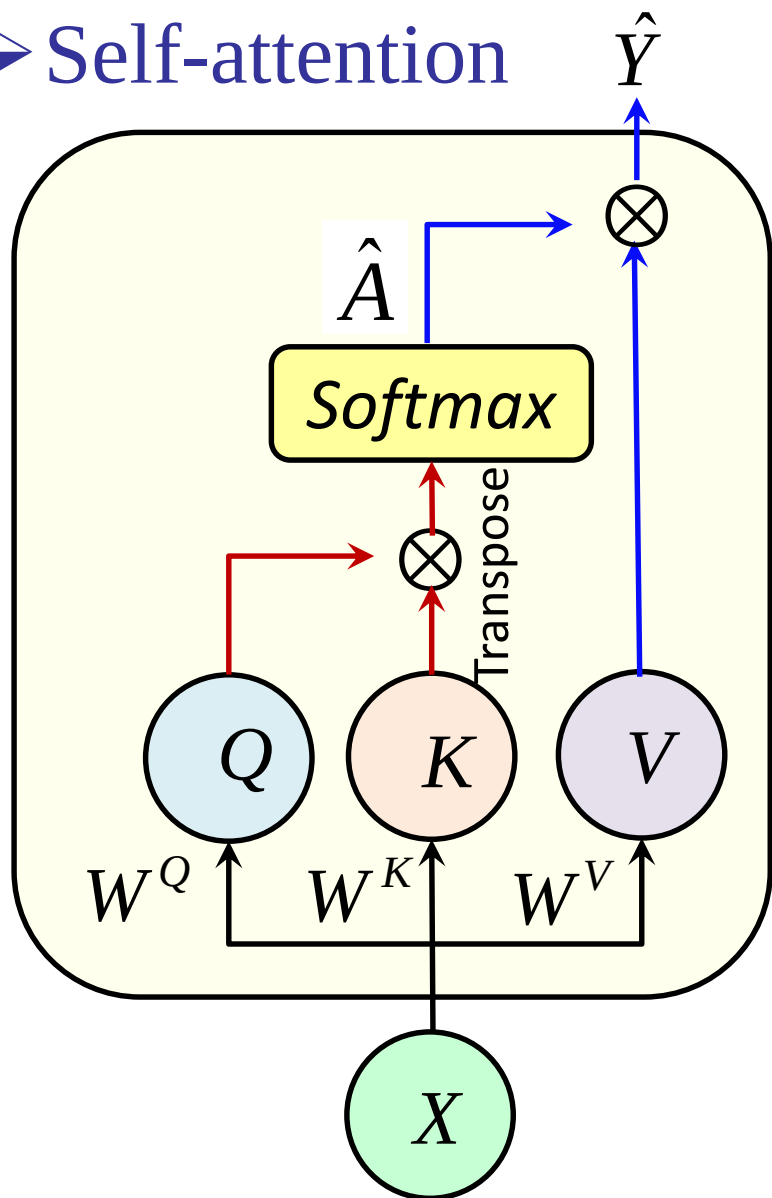


$$\hat{A} = \{\hat{a}_{i,j}\} = \text{Softmax}\left(\frac{QK^T}{\sqrt{d}}\right)$$

$$\hat{Y} = \hat{A}V = \text{Softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

## 6.3.1 自注意力机制 Self-attention

### ➤ Self-attention



$$\hat{Y} = \text{Softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

$$\hat{A} = \text{Softmax}\left(\frac{QK^T}{\sqrt{d}}\right)$$

$$Q = XW^Q, K = XW^K, V = XW^V$$

$\text{Attention}(Q, K, V)$

$$= \text{Softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$



## 6.3.1 自注意力机制 Self-attention

input #1

|   |   |   |   |
|---|---|---|---|
| 1 | 0 | 1 | 0 |
|---|---|---|---|

input #2

|   |   |   |   |
|---|---|---|---|
| 0 | 2 | 0 | 2 |
|---|---|---|---|

input #3

|   |   |   |   |
|---|---|---|---|
| 1 | 1 | 1 | 1 |
|---|---|---|---|

Step 1: Prepare inputs

## 6.3.1 自注意力机制 Self-attention

weights for **query**:

|   |   |   |
|---|---|---|
| 1 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 0 | 1 |
| 0 | 1 | 1 |

weights for **key**:

|   |   |   |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 1 | 0 |
| 0 | 1 | 0 |
| 1 | 1 | 0 |

weights for **value**:

|   |   |   |
|---|---|---|
| 0 | 2 | 0 |
| 0 | 3 | 0 |
| 1 | 0 | 3 |
| 1 | 1 | 0 |

input #1

|   |   |   |   |
|---|---|---|---|
| 1 | 0 | 1 | 0 |
|---|---|---|---|

input #2

|   |   |   |   |
|---|---|---|---|
| 0 | 2 | 0 | 2 |
|---|---|---|---|

input #3

|   |   |   |   |
|---|---|---|---|
| 1 | 1 | 1 | 1 |
|---|---|---|---|

## 6.3.1 自注意力机制 Self-attention

weights for **query**:

|   |   |   |
|---|---|---|
| 1 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 0 | 1 |
| 0 | 1 | 1 |

weights for **key**:

|   |   |   |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 1 | 0 |
| 0 | 1 | 0 |
| 1 | 1 | 0 |

weights for **value**:

|   |   |   |
|---|---|---|
| 0 | 2 | 0 |
| 0 | 3 | 0 |
| 1 | 0 | 3 |
| 1 | 1 | 0 |

Query representation for input #1:

$$\begin{bmatrix} 1 & 0 & 1 & 0 \end{bmatrix} \times \begin{bmatrix} 1 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 2 \end{bmatrix}$$

input #1

|   |   |   |   |
|---|---|---|---|
| 1 | 0 | 1 | 0 |
|---|---|---|---|

input #2

|   |   |   |   |
|---|---|---|---|
| 0 | 2 | 0 | 2 |
|---|---|---|---|

input #3

|   |   |   |   |
|---|---|---|---|
| 1 | 1 | 1 | 1 |
|---|---|---|---|

Step 3: Derive key, query and value

## 6.3.1 自注意力机制 Self-attention

weights for **query**:

|   |   |   |
|---|---|---|
| 1 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 0 | 1 |
| 0 | 1 | 1 |

weights for **key**:

|   |   |   |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 1 | 0 |
| 0 | 1 | 0 |
| 1 | 1 | 0 |

weights for **value**:

|   |   |   |
|---|---|---|
| 0 | 2 | 0 |
| 0 | 3 | 0 |
| 1 | 0 | 3 |
| 1 | 1 | 0 |

Query representation for input #2:

$$\begin{bmatrix} 0 & 2 & 0 & 2 \end{bmatrix} \times \begin{bmatrix} 1 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 2 & 2 \end{bmatrix}$$

input #1

|   |   |   |   |
|---|---|---|---|
| 1 | 0 | 1 | 0 |
|---|---|---|---|

input #2

|   |   |   |   |
|---|---|---|---|
| 0 | 2 | 0 | 2 |
|---|---|---|---|

input #3

|   |   |   |   |
|---|---|---|---|
| 1 | 1 | 1 | 1 |
|---|---|---|---|

Step 3: Derive key, query and value

## 6.3.1 自注意力机制 Self-attention

weights for **query**:

|   |   |   |
|---|---|---|
| 1 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 0 | 1 |
| 0 | 1 | 1 |

weights for **key**:

|   |   |   |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 1 | 0 |
| 0 | 1 | 0 |
| 1 | 1 | 0 |

weights for **value**:

|   |   |   |
|---|---|---|
| 0 | 2 | 0 |
| 0 | 3 | 0 |
| 1 | 0 | 3 |
| 1 | 1 | 0 |

Query representation for input #3:

$$\begin{bmatrix} 1 & 1 & 1 & 1 \end{bmatrix} \times \begin{bmatrix} 1 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 1 & 3 \end{bmatrix}$$

input #1

|   |   |   |   |
|---|---|---|---|
| 1 | 0 | 1 | 0 |
|---|---|---|---|

input #2

|   |   |   |   |
|---|---|---|---|
| 0 | 2 | 0 | 2 |
|---|---|---|---|

input #3

|   |   |   |   |
|---|---|---|---|
| 1 | 1 | 1 | 1 |
|---|---|---|---|

Step 3: Derive key, query and value

## 6.3.1 自注意力机制 Self-attention

weights for **query**:

|   |   |   |
|---|---|---|
| 1 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 0 | 1 |
| 0 | 1 | 1 |

weights for **key**:

|   |   |   |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 1 | 0 |
| 0 | 1 | 0 |
| 1 | 1 | 0 |

weights for **value**:

|   |   |   |
|---|---|---|
| 0 | 2 | 0 |
| 0 | 3 | 0 |
| 1 | 0 | 3 |
| 1 | 1 | 0 |

Query representations for inputs:

|          |                                                                                                                                                                    |                                   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                                                             |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                      |   |   |   |   |   |   |   |   |   |                                   |
|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------|---|---|---|---|---|---|---|---|---|---|---|---|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|---|---|---|---|---|---|---|---|---|---|---|---|--------------------------------------------------------------------------------------------------------------------------------------|---|---|---|---|---|---|---|---|---|-----------------------------------|
| input #1 | <table><tr><td>1</td><td>0</td><td>1</td><td>0</td></tr><tr><td>0</td><td>2</td><td>0</td><td>2</td></tr><tr><td>1</td><td>1</td><td>1</td><td>1</td></tr></table> | 1                                 | 0 | 1 | 0 | 0 | 2 | 0 | 2 | 1 | 1 | 1 | 1 | × | <table><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr></table> | 1 | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 1 | = | <table><tr><td>1</td><td>0</td><td>2</td></tr><tr><td>2</td><td>2</td><td>2</td></tr><tr><td>2</td><td>1</td><td>3</td></tr></table> | 1 | 0 | 2 | 2 | 2 | 2 | 2 | 1 | 3 | query representation for input #1 |
| 1        | 0                                                                                                                                                                  | 1                                 | 0 |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                                                             |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                      |   |   |   |   |   |   |   |   |   |                                   |
| 0        | 2                                                                                                                                                                  | 0                                 | 2 |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                                                             |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                      |   |   |   |   |   |   |   |   |   |                                   |
| 1        | 1                                                                                                                                                                  | 1                                 | 1 |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                                                             |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                      |   |   |   |   |   |   |   |   |   |                                   |
| 1        | 0                                                                                                                                                                  | 1                                 |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                                                             |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                      |   |   |   |   |   |   |   |   |   |                                   |
| 1        | 0                                                                                                                                                                  | 0                                 |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                                                             |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                      |   |   |   |   |   |   |   |   |   |                                   |
| 0        | 0                                                                                                                                                                  | 1                                 |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                                                             |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                      |   |   |   |   |   |   |   |   |   |                                   |
| 0        | 1                                                                                                                                                                  | 1                                 |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                                                             |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                      |   |   |   |   |   |   |   |   |   |                                   |
| 1        | 0                                                                                                                                                                  | 2                                 |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                                                             |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                      |   |   |   |   |   |   |   |   |   |                                   |
| 2        | 2                                                                                                                                                                  | 2                                 |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                                                             |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                      |   |   |   |   |   |   |   |   |   |                                   |
| 2        | 1                                                                                                                                                                  | 3                                 |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                                                             |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                      |   |   |   |   |   |   |   |   |   |                                   |
| input #2 |                                                                                                                                                                    | query representation for input #2 |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                                                             |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                      |   |   |   |   |   |   |   |   |   |                                   |
| input #3 |                                                                                                                                                                    | query representation for input #3 |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                                                             |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                      |   |   |   |   |   |   |   |   |   |                                   |

input #1

|   |   |   |   |
|---|---|---|---|
| 1 | 0 | 1 | 0 |
|---|---|---|---|

input #2

|   |   |   |   |
|---|---|---|---|
| 0 | 2 | 0 | 2 |
|---|---|---|---|

input #3

|   |   |   |   |
|---|---|---|---|
| 1 | 1 | 1 | 1 |
|---|---|---|---|

Step 3: Derive key, query and value

## 6.3.1 自注意力机制 Self-attention

weights for **query**:

|   |   |   |
|---|---|---|
| 1 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 0 | 1 |
| 0 | 1 | 1 |

weights for **key**:

|   |   |   |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 1 | 0 |
| 0 | 1 | 0 |
| 1 | 1 | 0 |

weights for **value**:

|   |   |   |
|---|---|---|
| 0 | 2 | 0 |
| 0 | 3 | 0 |
| 1 | 0 | 3 |
| 1 | 1 | 0 |

Key representations for inputs:

|          |                                                                                                                                                                    |   |   |                                 |   |   |   |   |   |   |   |   |   |   |                                                                                                                                                                             |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                      |   |   |   |   |   |   |   |   |   |                                 |
|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|---|---------------------------------|---|---|---|---|---|---|---|---|---|---|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|---|---|---|---|---|---|---|---|---|---|---|---|--------------------------------------------------------------------------------------------------------------------------------------|---|---|---|---|---|---|---|---|---|---------------------------------|
| input #1 | <table><tr><td>1</td><td>0</td><td>1</td><td>0</td></tr><tr><td>0</td><td>2</td><td>0</td><td>2</td></tr><tr><td>1</td><td>1</td><td>1</td><td>1</td></tr></table> | 1 | 0 | 1                               | 0 | 0 | 2 | 0 | 2 | 1 | 1 | 1 | 1 | × | <table><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table> | 0 | 0 | 1 | 1 | 1 | 0 | 0 | 1 | 0 | 1 | 1 | 0 | = | <table><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>4</td><td>4</td><td>0</td></tr><tr><td>2</td><td>3</td><td>1</td></tr></table> | 0 | 1 | 1 | 4 | 4 | 0 | 2 | 3 | 1 | key representation for input #1 |
| 1        | 0                                                                                                                                                                  | 1 | 0 |                                 |   |   |   |   |   |   |   |   |   |   |                                                                                                                                                                             |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                      |   |   |   |   |   |   |   |   |   |                                 |
| 0        | 2                                                                                                                                                                  | 0 | 2 |                                 |   |   |   |   |   |   |   |   |   |   |                                                                                                                                                                             |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                      |   |   |   |   |   |   |   |   |   |                                 |
| 1        | 1                                                                                                                                                                  | 1 | 1 |                                 |   |   |   |   |   |   |   |   |   |   |                                                                                                                                                                             |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                      |   |   |   |   |   |   |   |   |   |                                 |
| 0        | 0                                                                                                                                                                  | 1 |   |                                 |   |   |   |   |   |   |   |   |   |   |                                                                                                                                                                             |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                      |   |   |   |   |   |   |   |   |   |                                 |
| 1        | 1                                                                                                                                                                  | 0 |   |                                 |   |   |   |   |   |   |   |   |   |   |                                                                                                                                                                             |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                      |   |   |   |   |   |   |   |   |   |                                 |
| 0        | 1                                                                                                                                                                  | 0 |   |                                 |   |   |   |   |   |   |   |   |   |   |                                                                                                                                                                             |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                      |   |   |   |   |   |   |   |   |   |                                 |
| 1        | 1                                                                                                                                                                  | 0 |   |                                 |   |   |   |   |   |   |   |   |   |   |                                                                                                                                                                             |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                      |   |   |   |   |   |   |   |   |   |                                 |
| 0        | 1                                                                                                                                                                  | 1 |   |                                 |   |   |   |   |   |   |   |   |   |   |                                                                                                                                                                             |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                      |   |   |   |   |   |   |   |   |   |                                 |
| 4        | 4                                                                                                                                                                  | 0 |   |                                 |   |   |   |   |   |   |   |   |   |   |                                                                                                                                                                             |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                      |   |   |   |   |   |   |   |   |   |                                 |
| 2        | 3                                                                                                                                                                  | 1 |   |                                 |   |   |   |   |   |   |   |   |   |   |                                                                                                                                                                             |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                      |   |   |   |   |   |   |   |   |   |                                 |
| input #2 |                                                                                                                                                                    |   |   | key representation for input #2 |   |   |   |   |   |   |   |   |   |   |                                                                                                                                                                             |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                      |   |   |   |   |   |   |   |   |   |                                 |
| input #3 |                                                                                                                                                                    |   |   | key representation for input #3 |   |   |   |   |   |   |   |   |   |   |                                                                                                                                                                             |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                                                                      |   |   |   |   |   |   |   |   |   |                                 |

input #1

|   |   |   |   |
|---|---|---|---|
| 1 | 0 | 1 | 0 |
|---|---|---|---|

input #2

|   |   |   |   |
|---|---|---|---|
| 0 | 2 | 0 | 2 |
|---|---|---|---|

input #3

|   |   |   |   |
|---|---|---|---|
| 1 | 1 | 1 | 1 |
|---|---|---|---|

Step 3: Derive key, query and value

# 6.3.1 自注意力机制 Self-attention

weights for **query**:

|   |   |   |
|---|---|---|
| 1 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 0 | 1 |
| 0 | 1 | 1 |

weights for **key**:

|   |   |   |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 1 | 0 |
| 0 | 1 | 0 |
| 1 | 1 | 0 |

weights for **value**:

|   |   |   |
|---|---|---|
| 0 | 2 | 0 |
| 0 | 3 | 0 |
| 1 | 0 | 3 |
| 1 | 1 | 0 |

Value representations for inputs:

input #1  
input #2  
input #3

|   |   |   |   |
|---|---|---|---|
| 1 | 0 | 1 | 0 |
| 0 | 2 | 0 | 2 |
| 1 | 1 | 1 | 1 |

×

|   |   |   |
|---|---|---|
| 0 | 2 | 0 |
| 0 | 3 | 0 |
| 1 | 0 | 3 |
| 1 | 1 | 0 |

=

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 2 | 8 | 0 |
| 2 | 6 | 3 |

value representation for input #1  
value representation for input #2  
value representation for input #3

input #1

|   |   |   |   |
|---|---|---|---|
| 1 | 0 | 1 | 0 |
|---|---|---|---|

input #2

|   |   |   |   |
|---|---|---|---|
| 0 | 2 | 0 | 2 |
|---|---|---|---|

input #3

|   |   |   |   |
|---|---|---|---|
| 1 | 1 | 1 | 1 |
|---|---|---|---|



# 6.3.1 自注意力机制 Self-attention

weights for query:

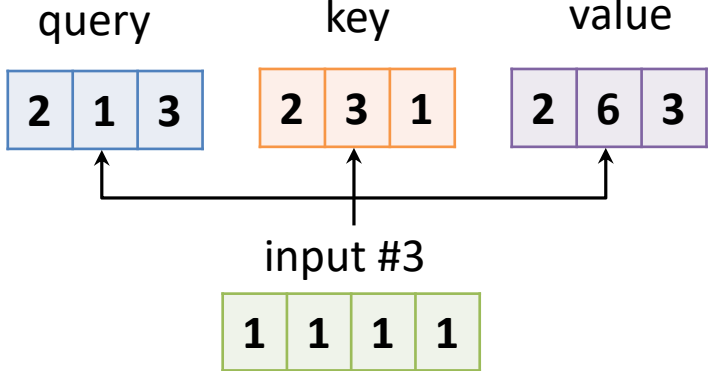
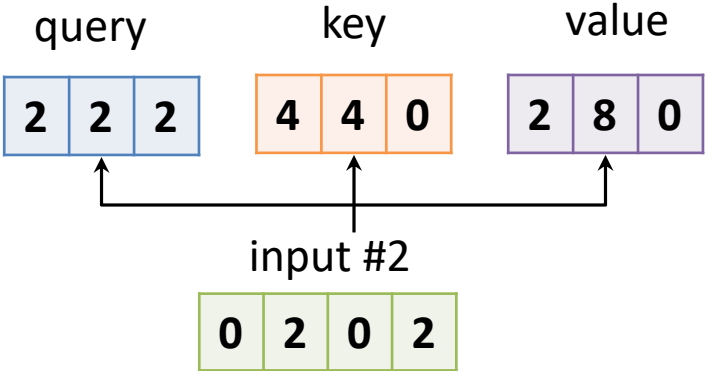
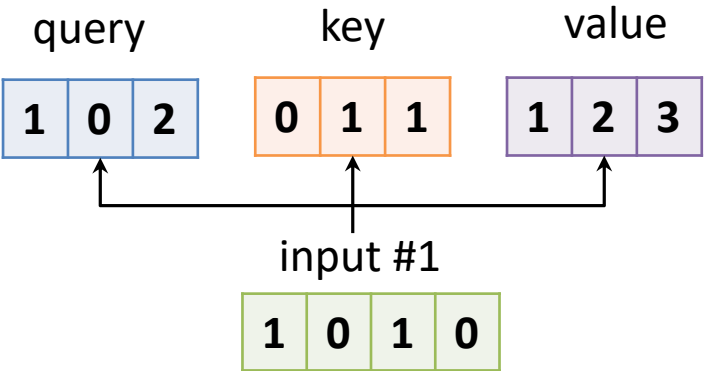
|   |   |   |
|---|---|---|
| 1 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 0 | 1 |
| 0 | 1 | 1 |

weights for key:

|   |   |   |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 1 | 0 |
| 0 | 1 | 0 |
| 1 | 1 | 0 |

weights for value:

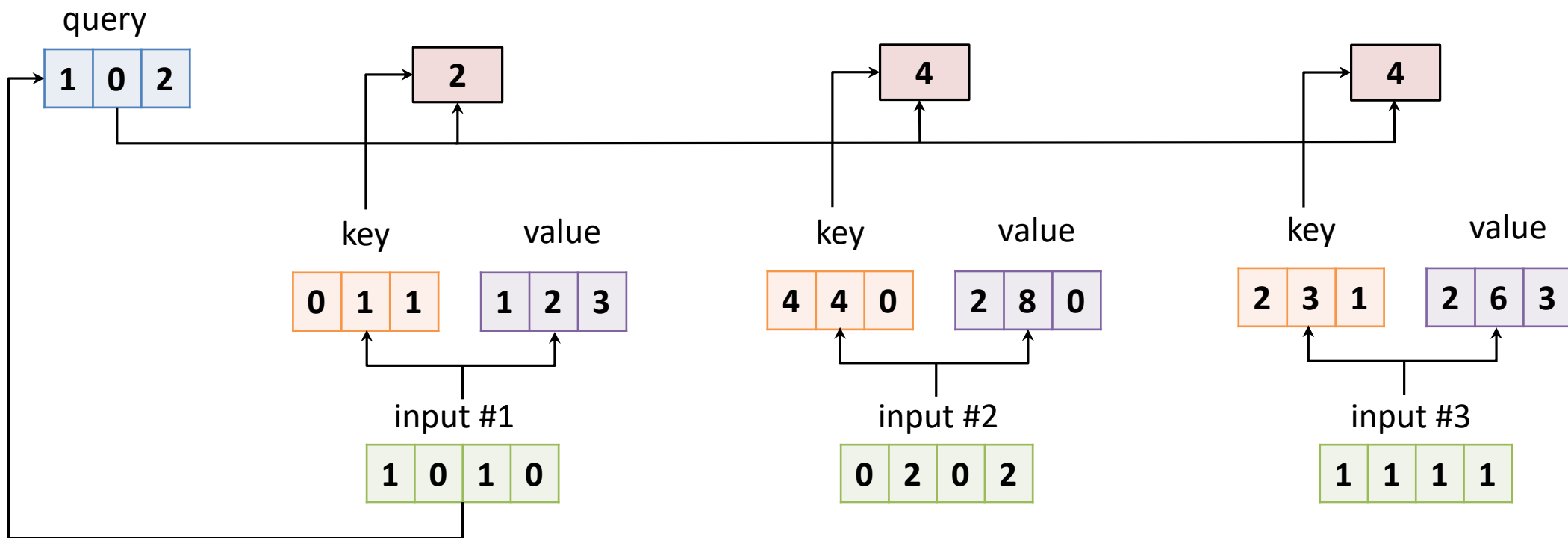
|   |   |   |
|---|---|---|
| 0 | 2 | 0 |
| 0 | 3 | 0 |
| 1 | 0 | 3 |
| 1 | 1 | 0 |



Step 3: Derive key, query and value

## 6.3.1 自注意力机制 Self-attention

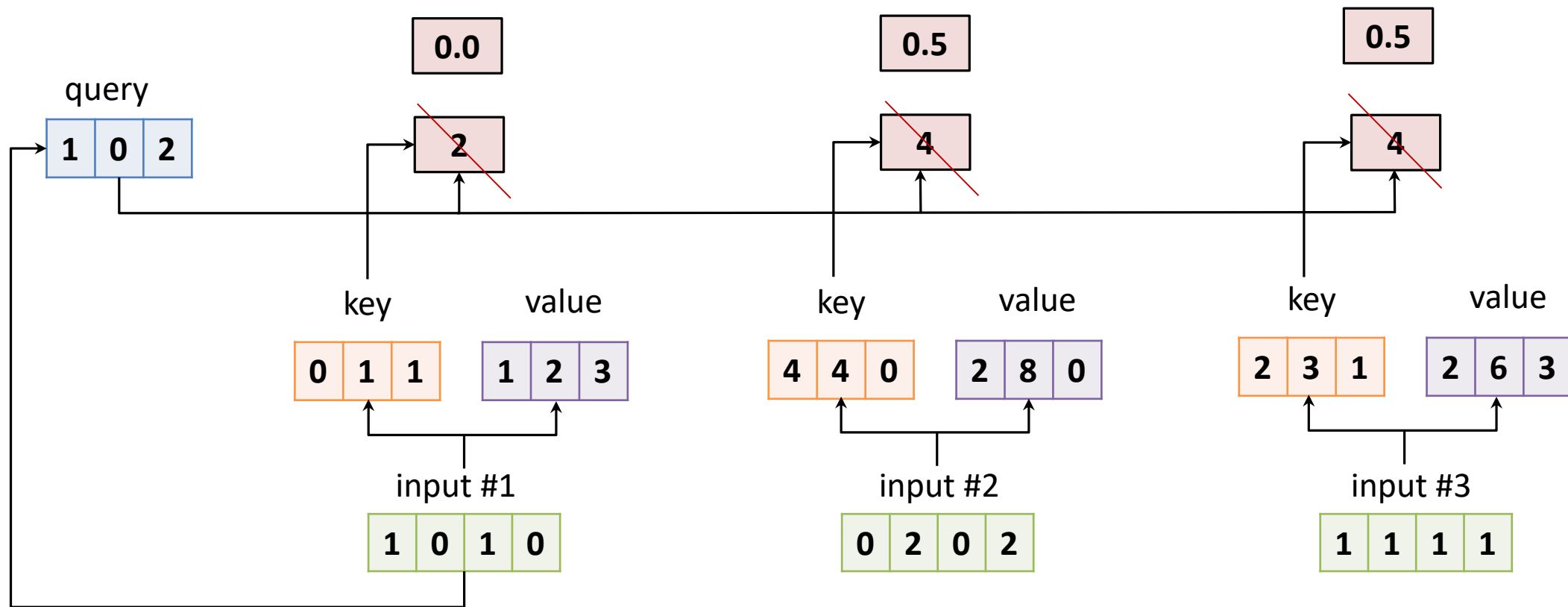
$$\begin{bmatrix} 1 & 0 & 2 \end{bmatrix} \times \begin{bmatrix} 0 & 1 & 1 \\ 4 & 4 & 0 \\ 2 & 3 & 1 \end{bmatrix}^T = \begin{bmatrix} 1 & 0 & 2 \end{bmatrix} \times \begin{bmatrix} 0 & 4 & 2 \\ 1 & 4 & 3 \\ 1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 4 & 4 \end{bmatrix}$$



Step 4: Calculate attention scores

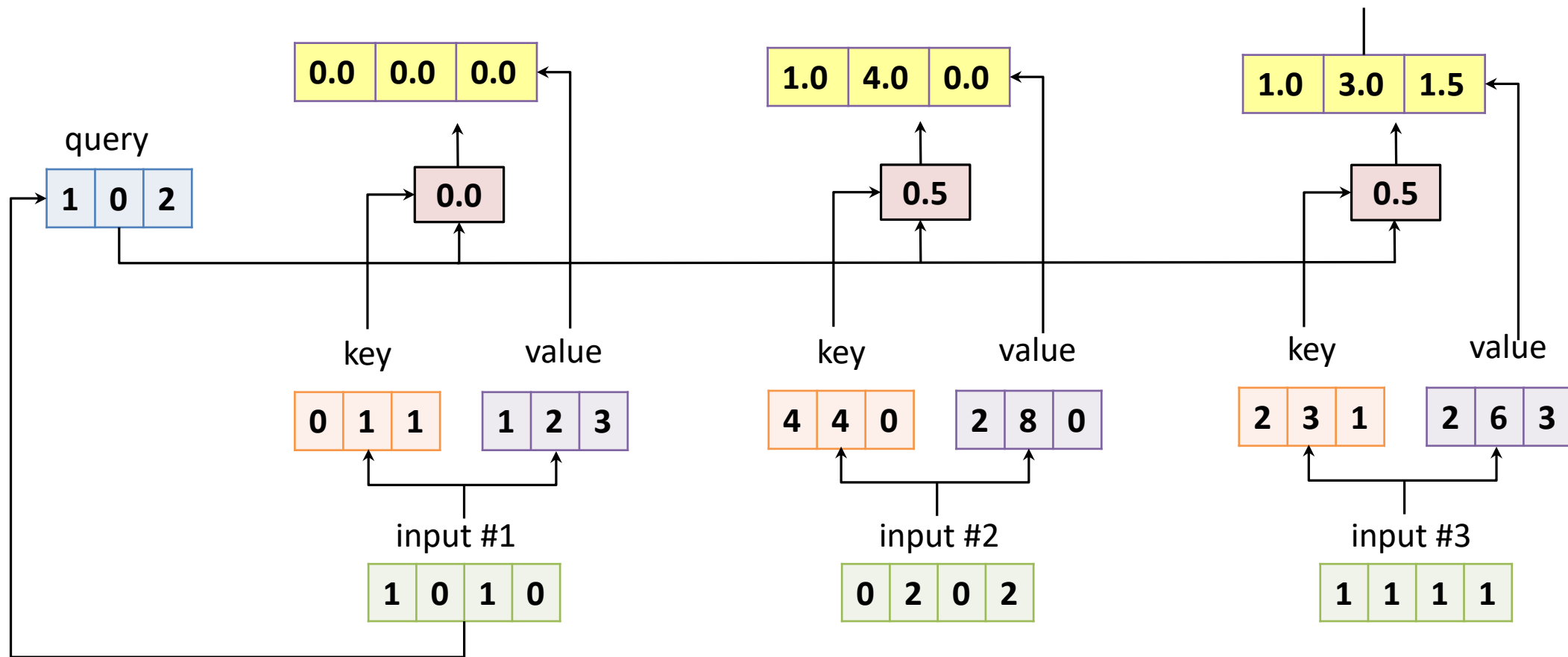
## 6.3.1 自注意力机制 Self-attention

$$\text{Softmax}(\begin{bmatrix} 2 & 4 & 4 \end{bmatrix}) \approx \begin{bmatrix} 0.0 & 0.5 & 0.5 \end{bmatrix}$$



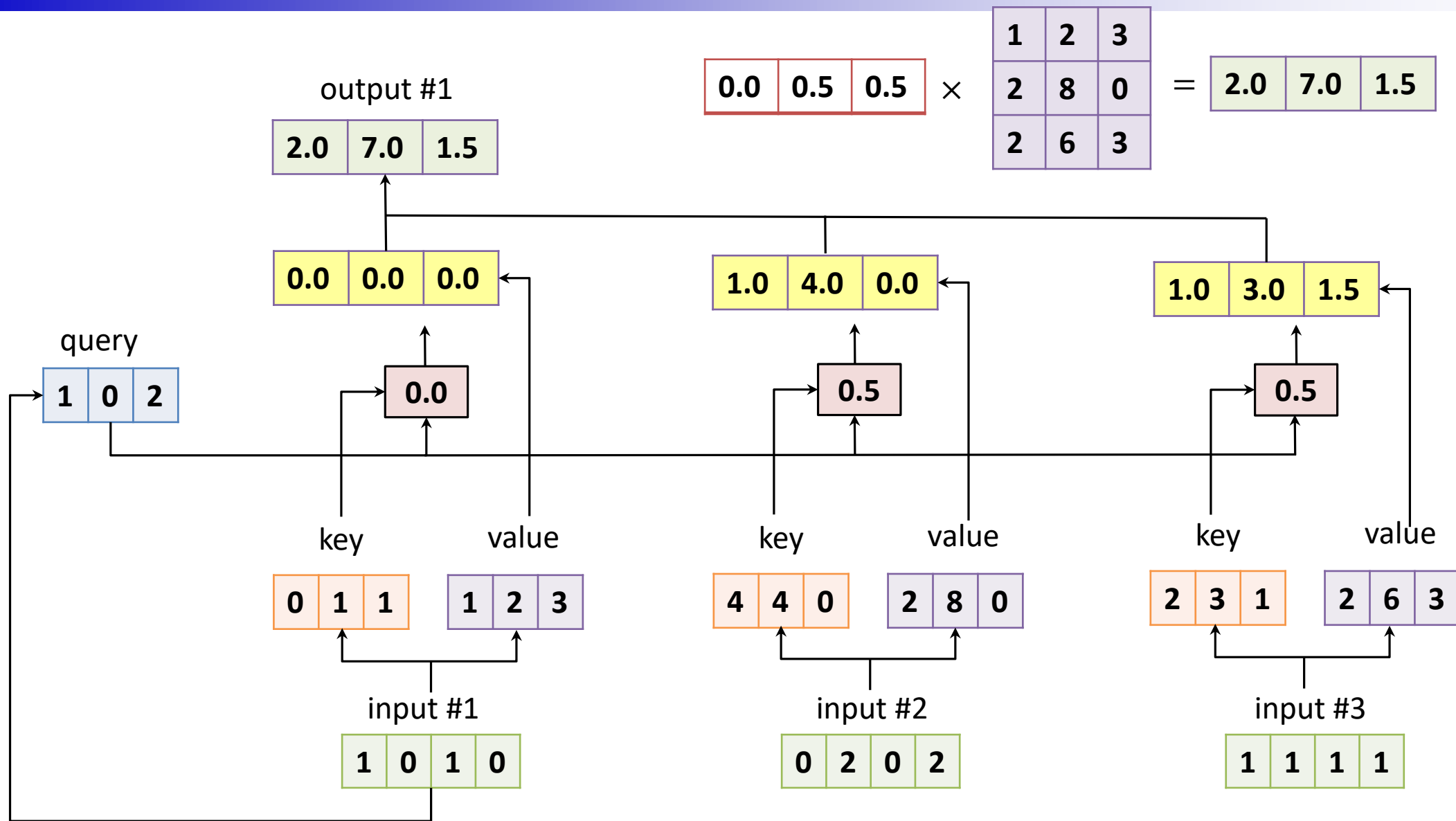
Step 4: Calculate attention scores

## 6.3.1 自注意力机制 Self-attention



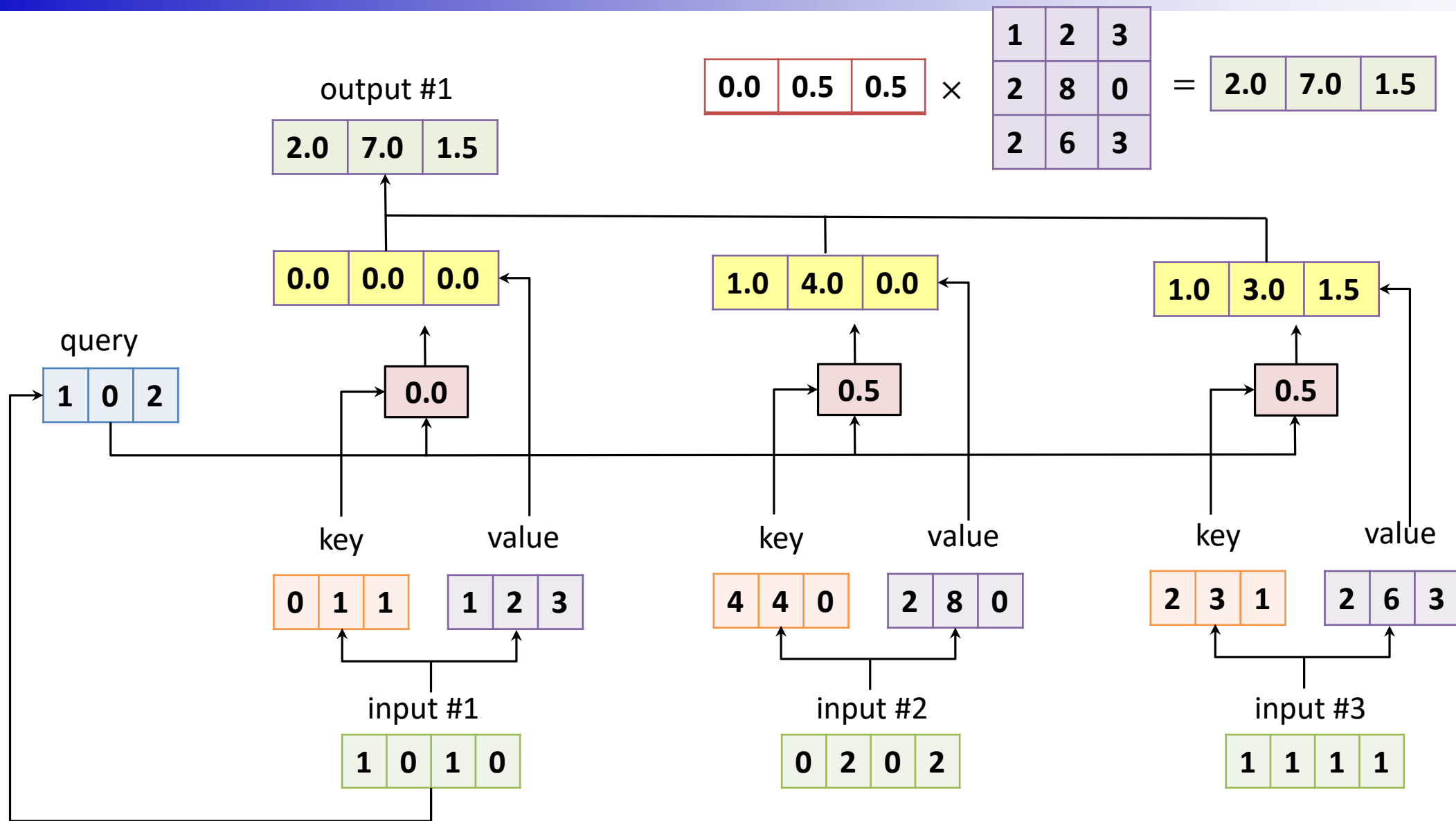
Step 5: **Multiply scores with values**

## 6.3.1 自注意力机制 Self-attention



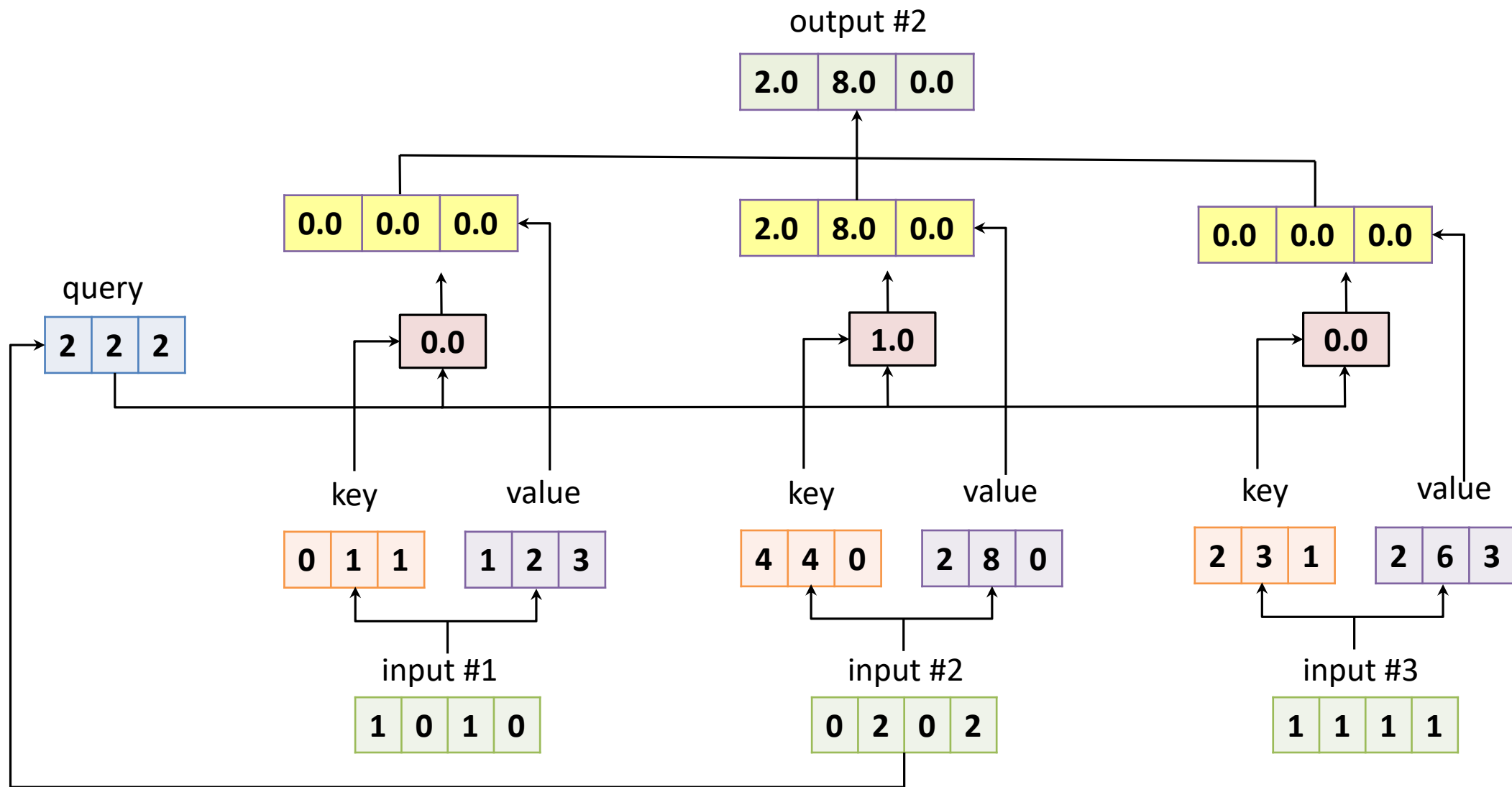
Step 6: Sum weighted values to get Output

## 6.3.1 自注意力机制 Self-attention



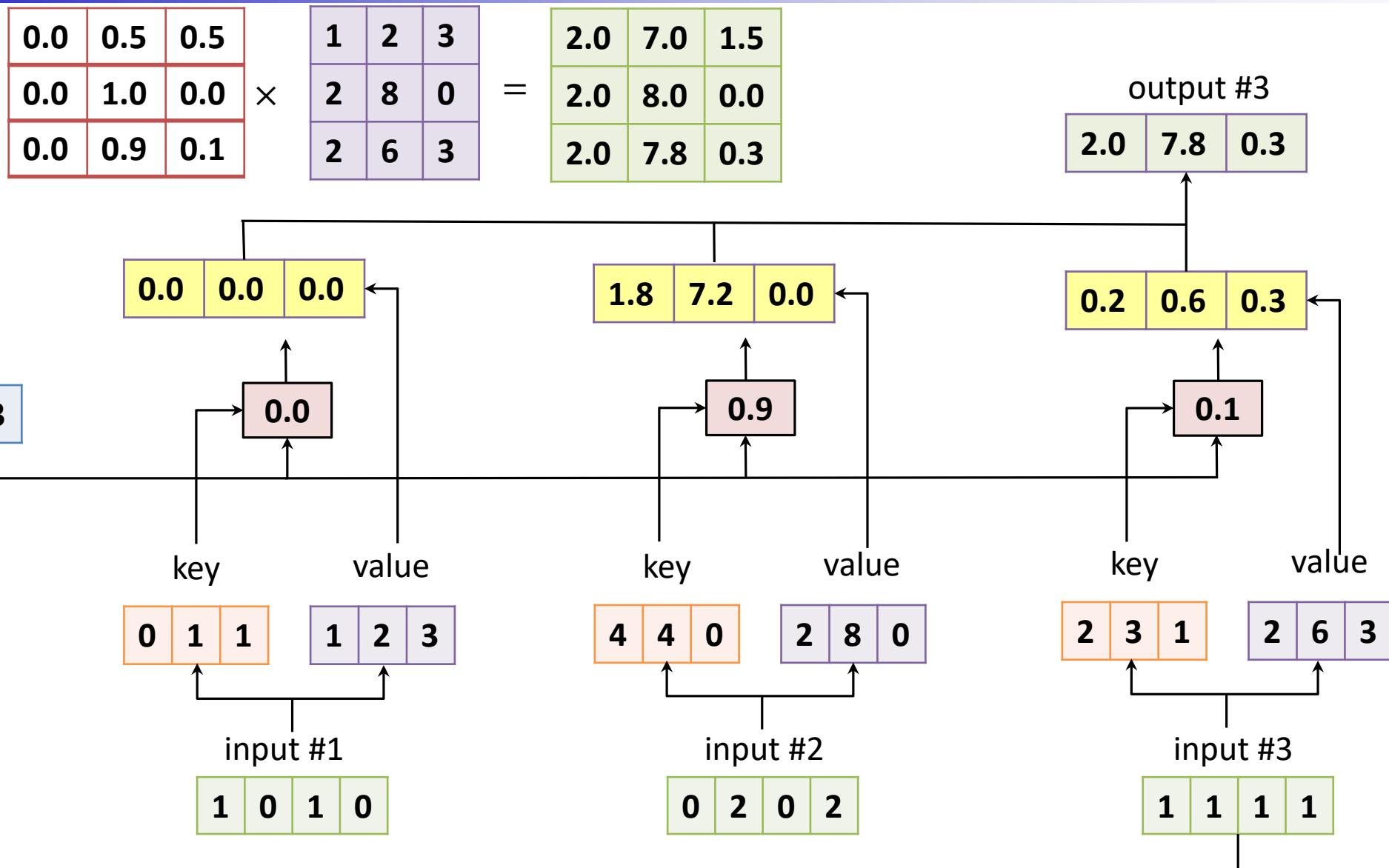
Step 6: Sum weighted values to get Output

## 6.3.1 自注意力机制 Self-attention



Step 6: Sum weighted values to get Output

## 6.3.1 自注意力机制 Self-attention

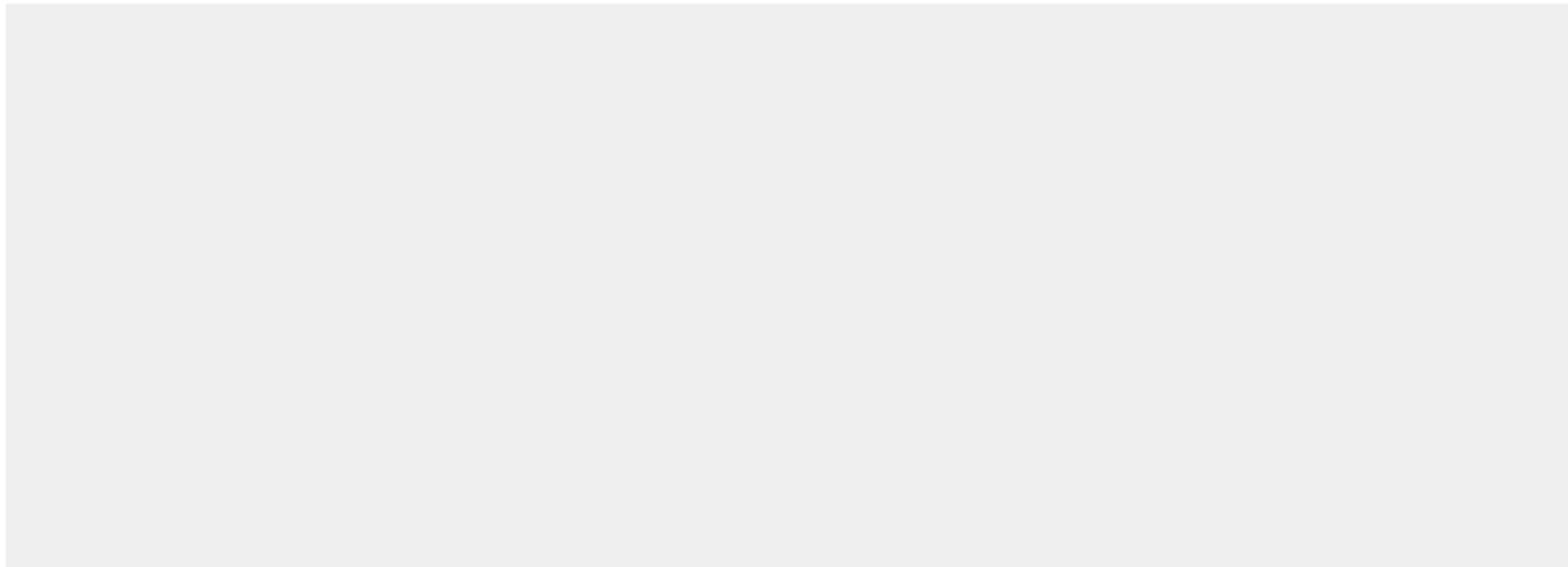


Step 6: Sum weighted values to get Output



## 6.3.1 自注意力机制 Self-attention

Self-attention



input #1

|   |   |   |   |
|---|---|---|---|
| 1 | 0 | 1 | 0 |
|---|---|---|---|

input #2

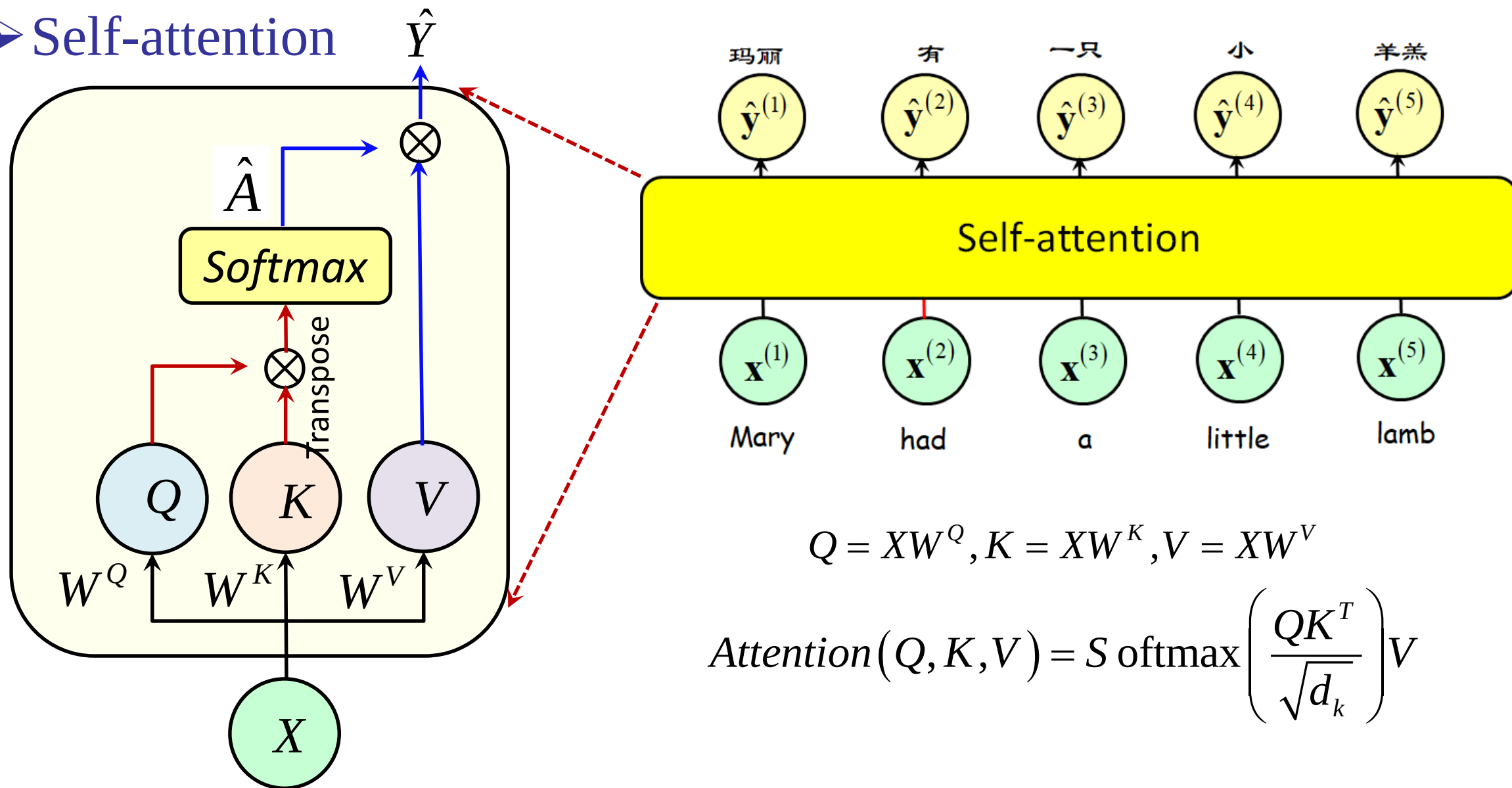
|   |   |   |   |
|---|---|---|---|
| 0 | 2 | 0 | 2 |
|---|---|---|---|

input #3

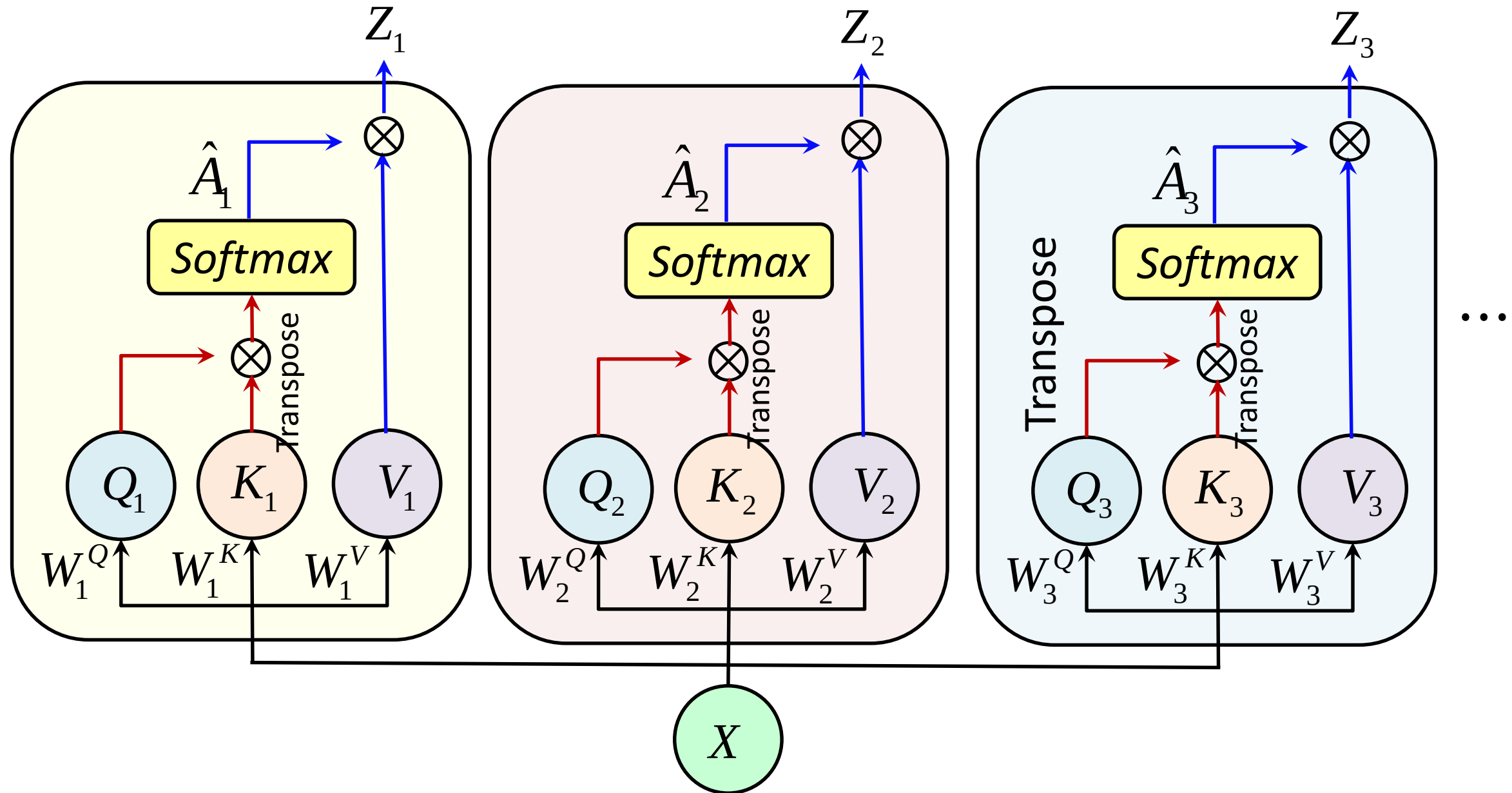
|   |   |   |   |
|---|---|---|---|
| 1 | 1 | 1 | 1 |
|---|---|---|---|

## 6.3.1 自注意力机制 Self-attention

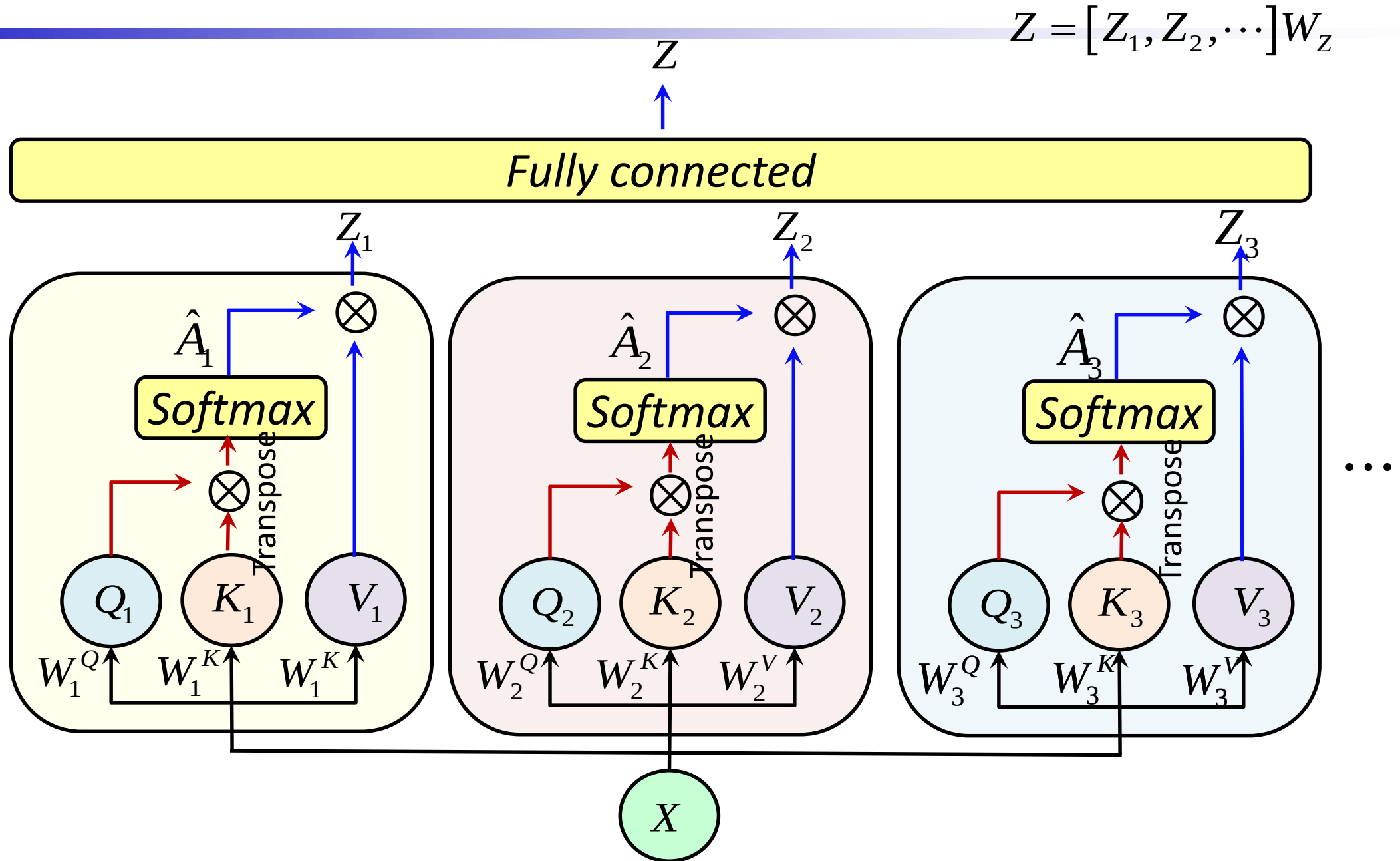
### ➤ Self-attention



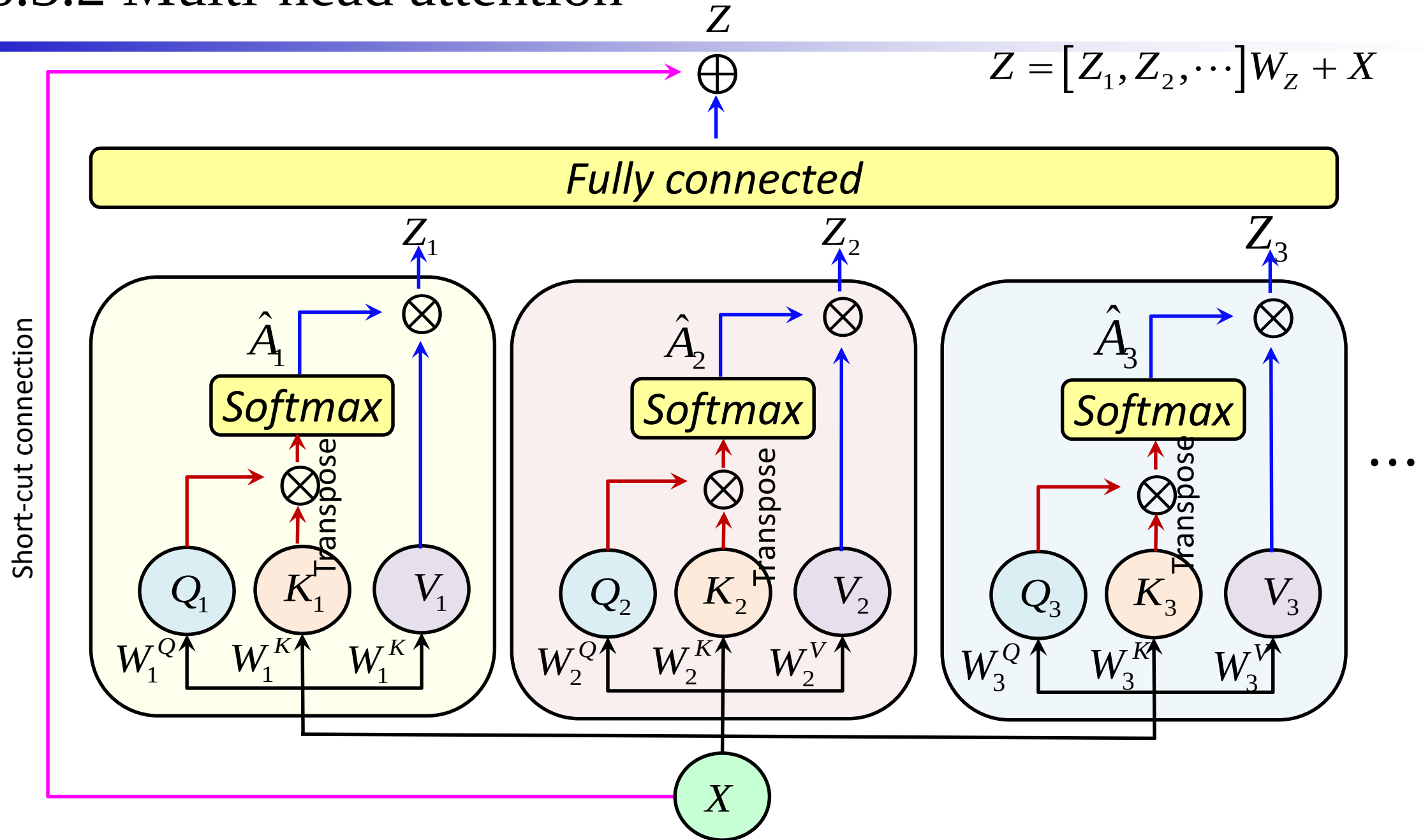
## 6.3.2 Multi-head attention



## 6.3.2 Multi-head attention



## 6.3.2 Multi-head attention



## 6.3.2 Multi-head attention (pytorch)

```
CLASS torch.nn.MultiheadAttention(embed_dim, num_heads, dropout=0.0,  
bias=True, add_bias_kv=False, add_zero_attn=False, kdim=None, [SOURCE]  
vdim=None, batch_first=False, device=None, dtype=None)
```

Allows the model to jointly attend to information from different representation subspaces. See [Attention Is All You Need](#)

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

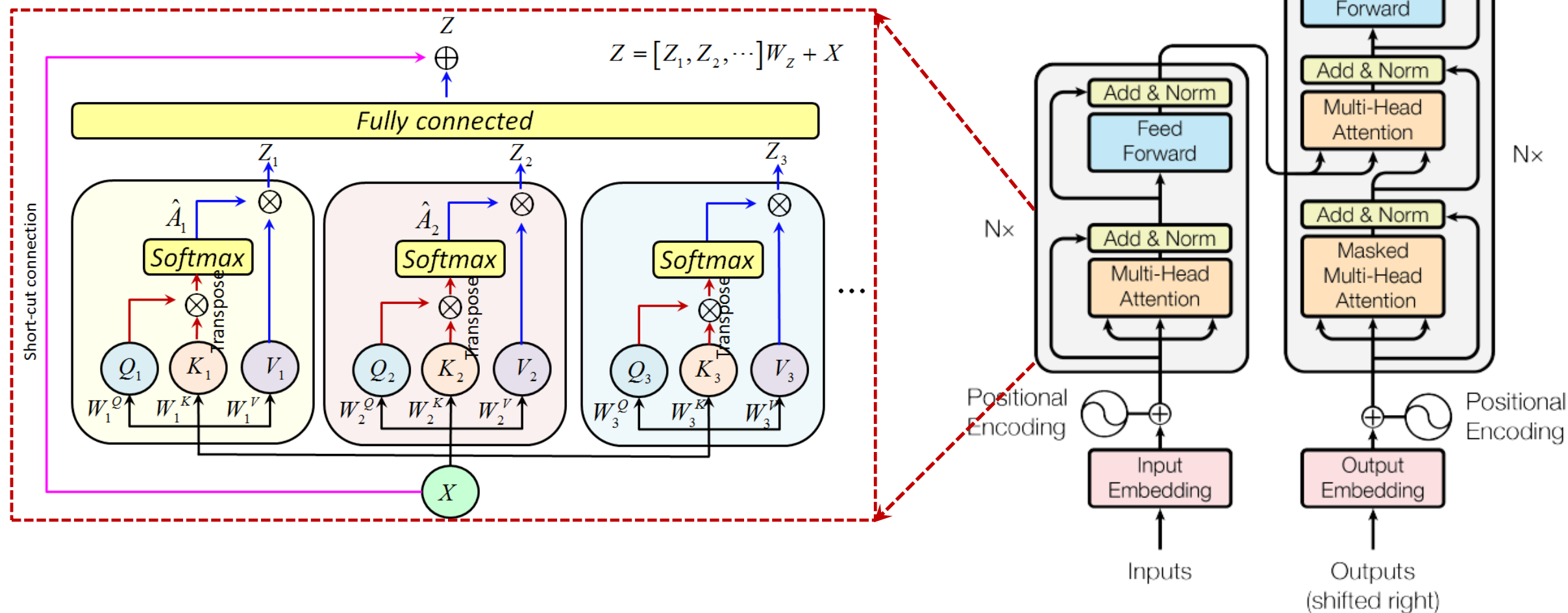
where  $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$ .

### Parameters

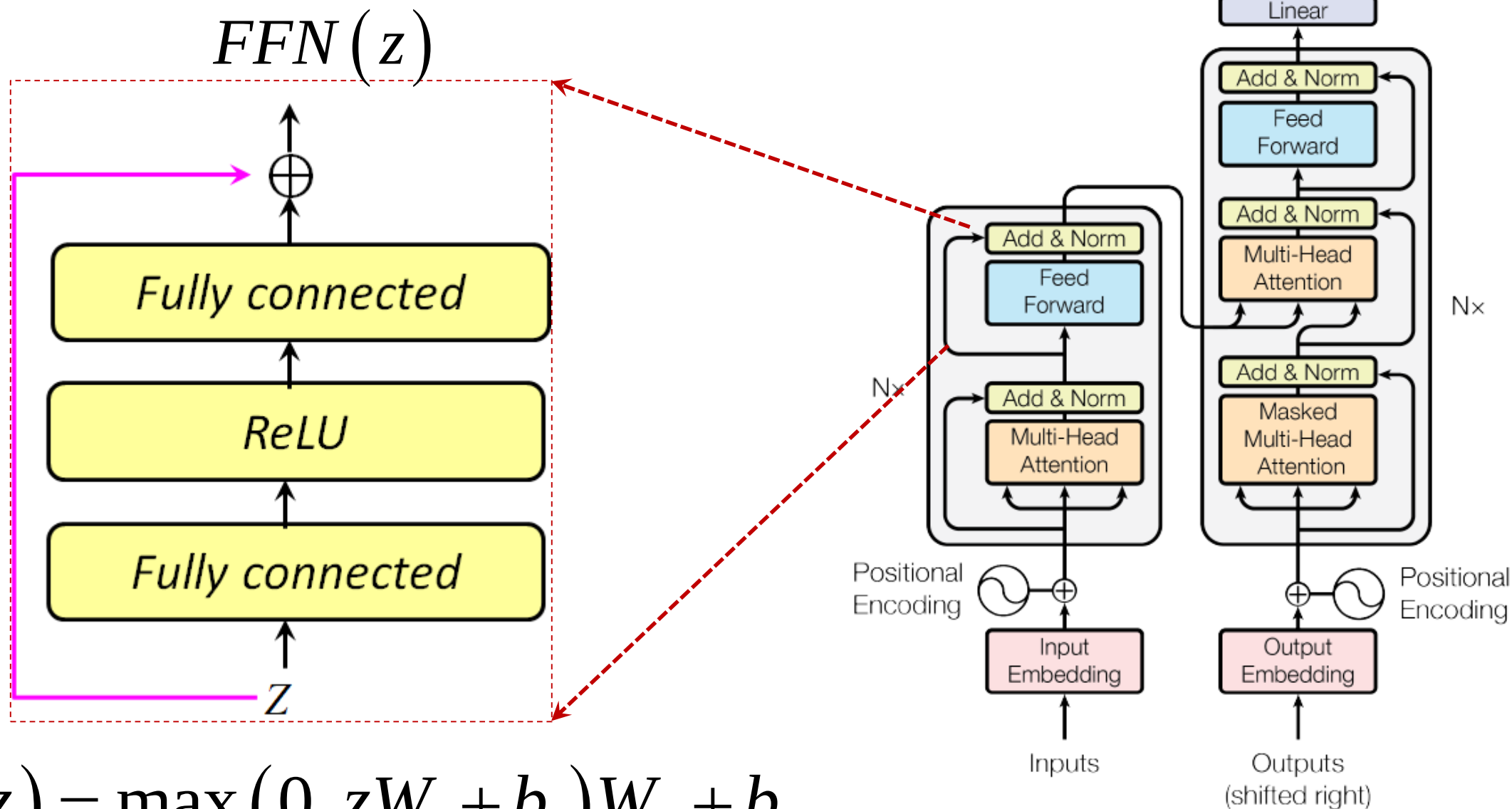
- **embed\_dim** – total dimension of the model.
- **num\_heads** – parallel attention heads.
- **dropout** – a Dropout layer on attn\_output\_weights. Default: 0.0.
- **bias** – add bias as module parameter. Default: True.
- **add\_bias\_kv** – add bias to the key and value sequences at dim=0.

<https://pytorch.org/docs/master/generated/torch.nn.MultiheadAttention.html?highlight=multihead> / and value sequences at dim=1.

## 6.3.3 Transformer



## 6.3.3 Transformer



$$FFN(z) = \max(0, zW_1 + b_1)W_2 + b_2$$



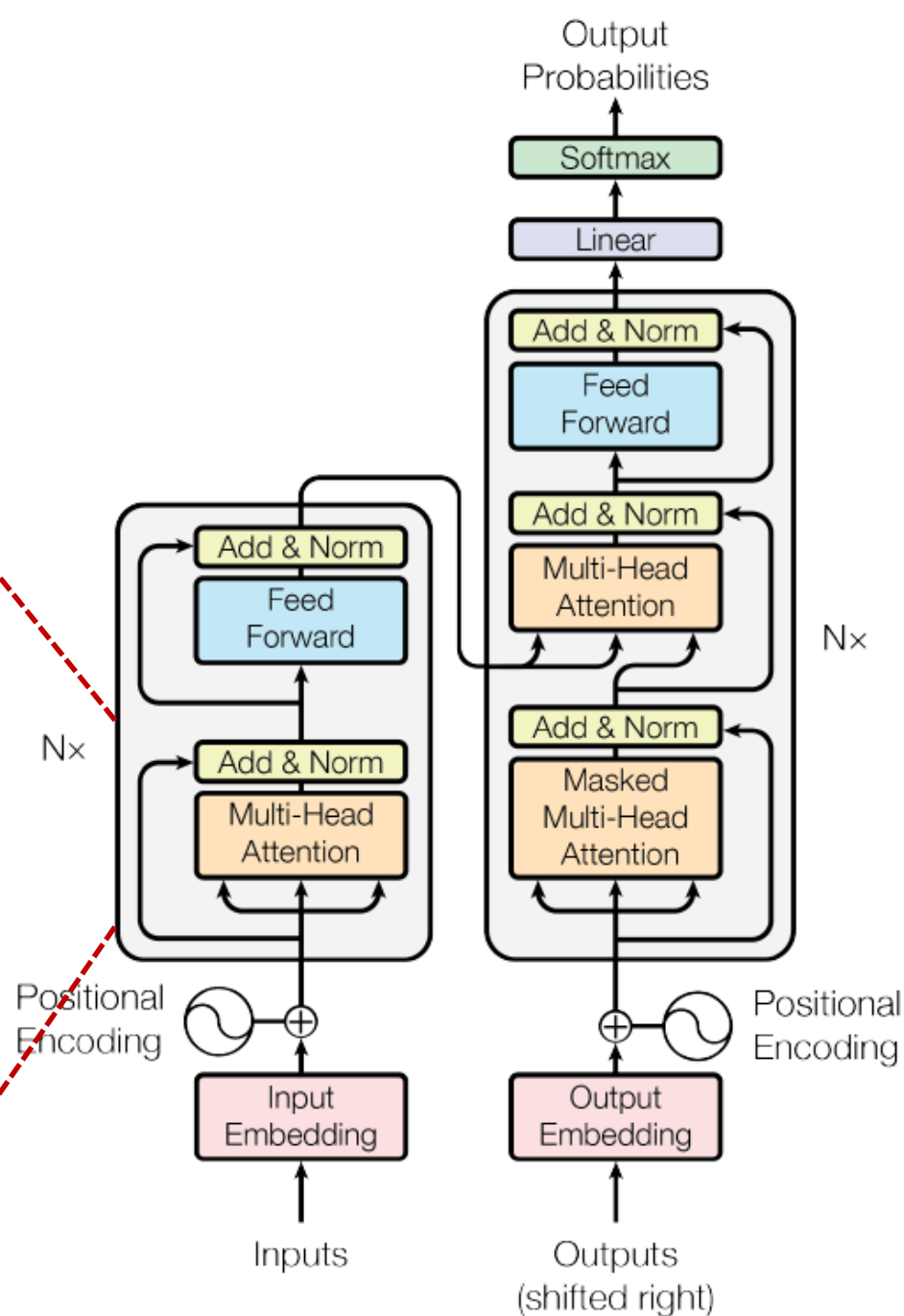
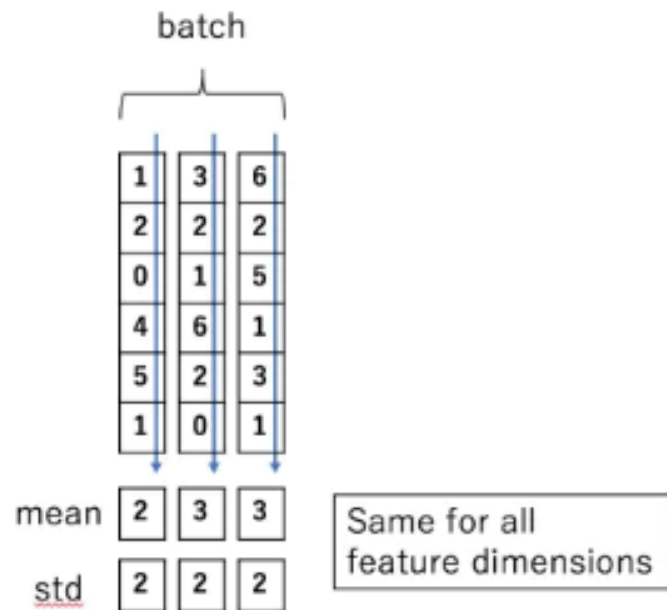
## 6.3.3 Transformer

### ► Layer Normalization (层归一化)

Batch Normalization



Layer Normalization

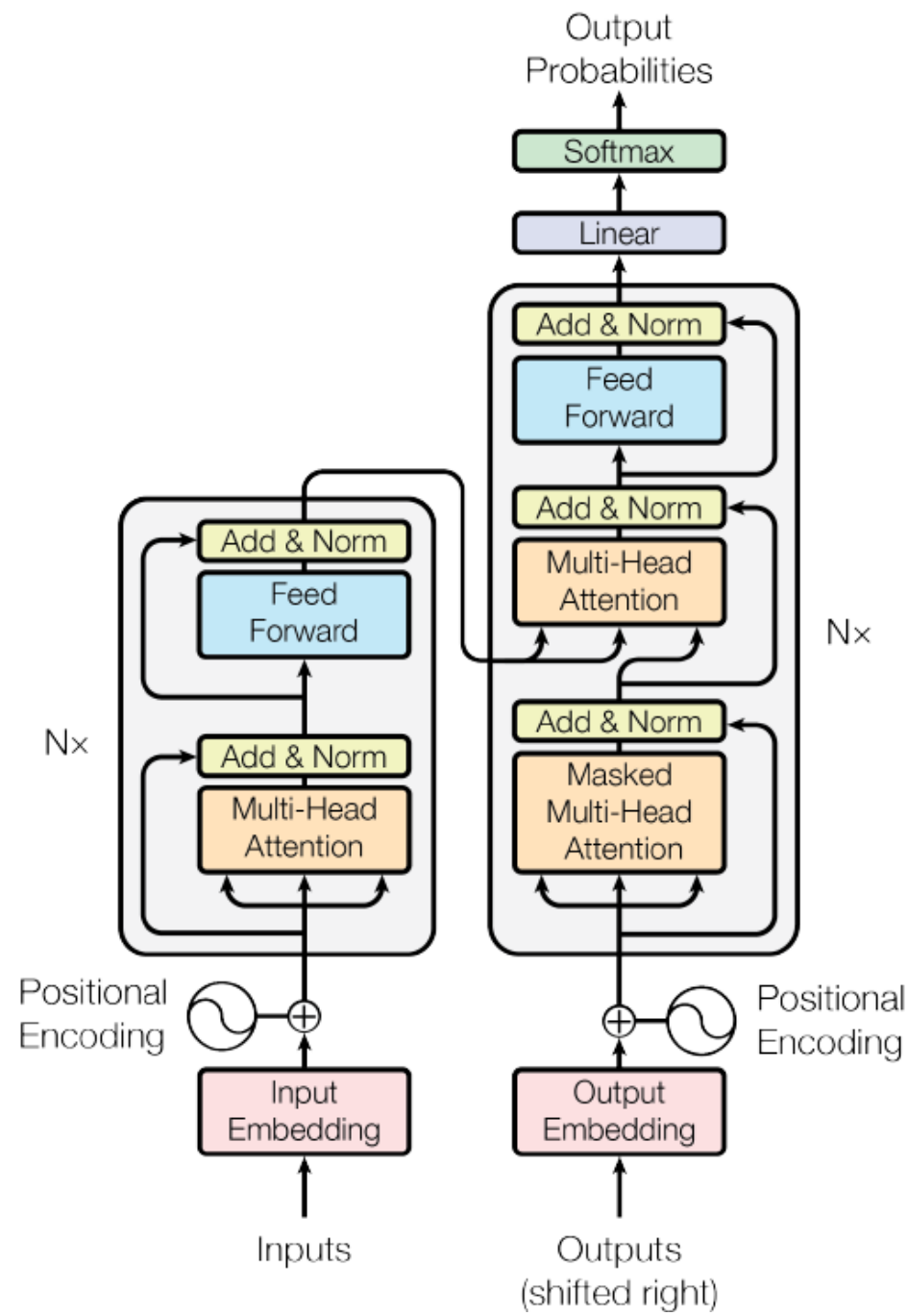
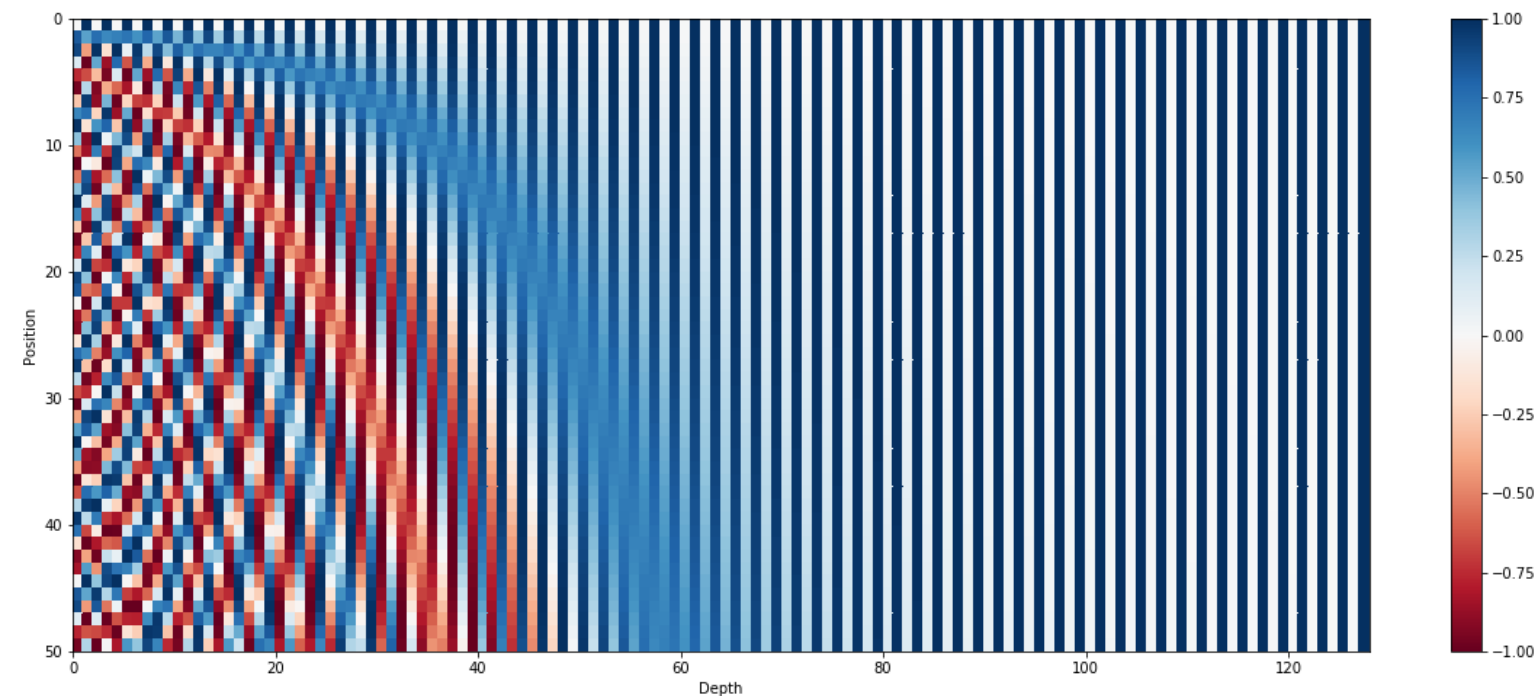


## 6.3.3 Transformer

### ► Positional Encoding

$$PE(pos, 2i) = \sin(pos/1000^{2i/d_{\text{model}}})$$

$$PE(pos, 2i+1) = \cos(pos/1000^{2i/d_{\text{model}}})$$



## 6.3.3 Transformer (Pytorch)

### TRANSFORMER

```
CLASS torch.nn.Transformer(d_model=512, nhead=8, num_encoder_layers=6,  
num_decoder_layers=6, dim_feedforward=2048, dropout=0.1, [SOURCE]  
activation='relu', custom_encoder=None, custom_decoder=None)
```

A transformer model. User is able to modify the attributes as needed. The architecture is based on the paper “Attention Is All You Need”. Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In Advances in Neural Information Processing Systems, pages 6000-6010. Users can build the BERT(<https://arxiv.org/abs/1810.04805>) model with corresponding parameters.

#### Parameters

- **d\_model** – the number of expected features in the encoder/decoder inputs (default=512).
- **nhead** – the number of heads in the multiheadattention models (default=8).
- **num\_encoder\_layers** – the number of sub-encoder-layers in the encoder

# 第6讲 序列建模：

## 循环神经网络与TRANSFORMER

### 6.1 序列建模概述

### 6.2 循环神经网络

### 6.3 TRANSFORMER

### 6.4 应用与展望

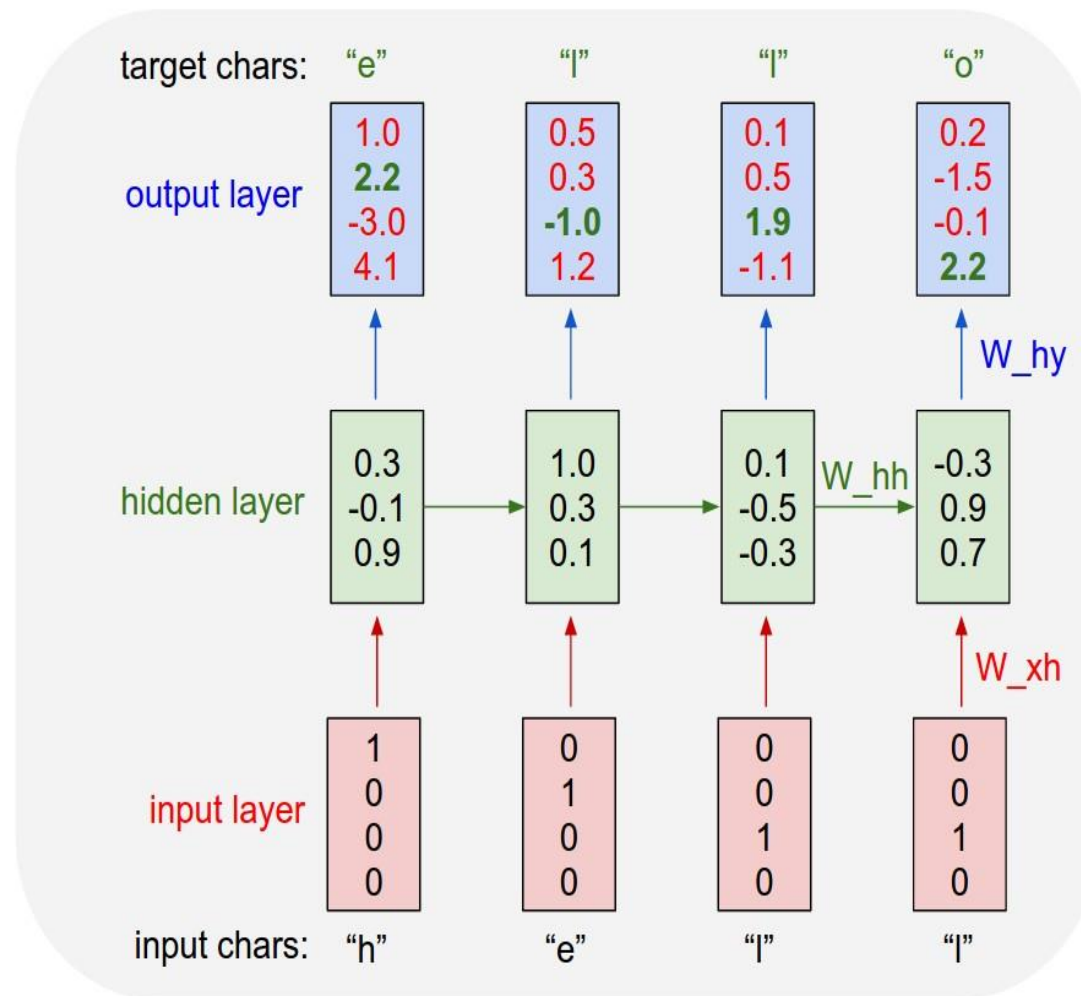
## 6.4.1 循环神经网络：应用示例

### ➤ 英文单词生成

Vocabulary:  
[h,e,l,o]

Example training  
sequence:  
“hello”

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$



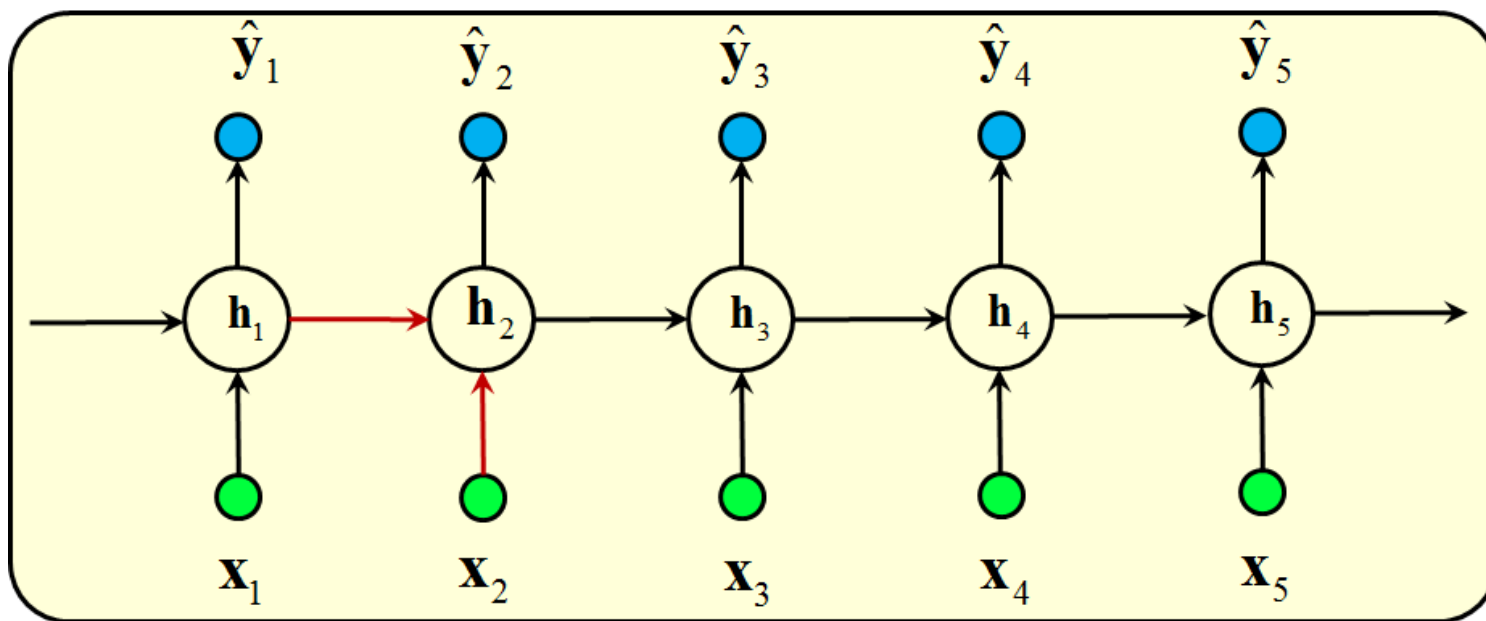
## 6.4.1 循环神经网络：应用示例

### ➤ 语言模型

Mary helped old MacDonald and had a lamb.

$$p(\text{Mary}) \quad p(\text{helped}|\text{Mary}) \cdots p(a|\text{Mary helped old} \cdots \text{had})$$

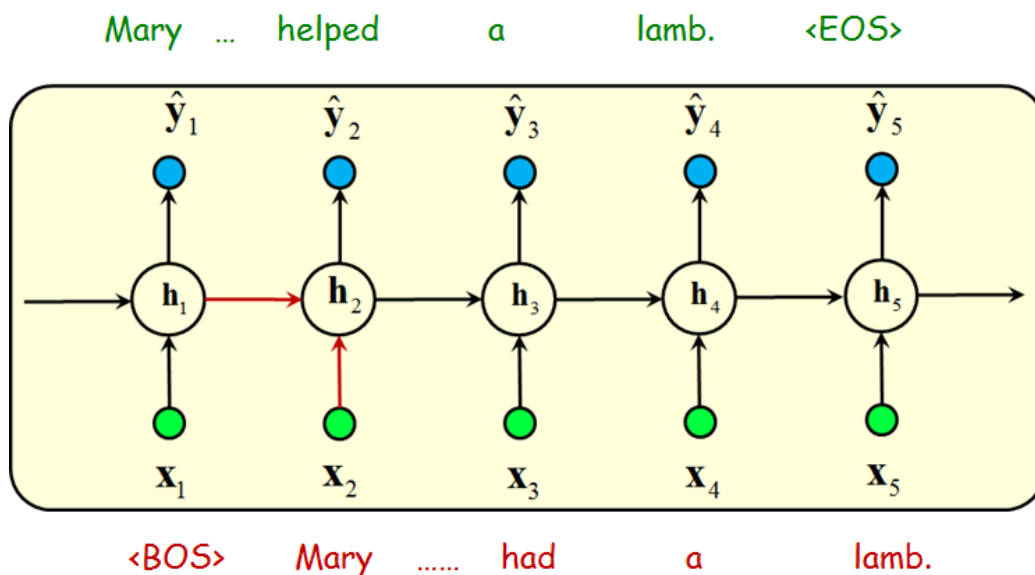
Mary ... helped a lamb. <EOS>



<BOS> Mary ..... had a lamb.

## 6.4.1 循环神经网络：应用示例

### ➤ 语言模型 language modeling



Step1: 准备语料库corpus，包括训练集和测试集

Step2: 设计循环神经网络

Step3: 确定损失函数，选择优化策略

Step4: 在训练集上训练学习神经网络参数

Step5: 在测试集上测试神经网络性能

## 6.4.1 循环神经网络：应用示例

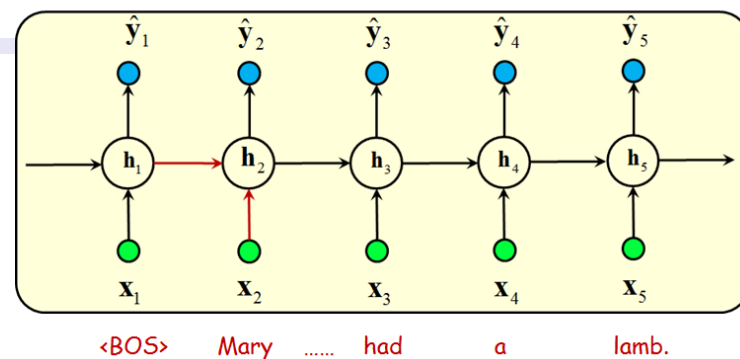
### ➤ 语言模型 language modeling

Step1: 准备语料库corpus，包括训练集和测试集

```
#####  
# Load data  
#####  
  
corpus = data.Corpus(args.data)  
  
# Starting from sequential data, batchify arranges the dataset into columns.  
# For instance, with the alphabet as the sequence and batch size 4, we'd get  
# [ a g m s ]  
# [ b h n t ]  
# [ c i o u ]  
# [ d j p v ]  
# [ e k q w ]  
# [ f l r x ].  
# These columns are treated as independent by the model, which means that the  
# dependence of e. g. 'g' on 'f' can not be learned, but allows more efficient  
# batch processing.  
  
def batchify(data, bsz):  
    # Work out how cleanly we can divide the dataset into bsz parts.  
    nbatch = data.size(0) // bsz  
    # Trim off any extra elements that wouldn't cleanly fit (remainders).  
    data = data.narrow(0, 0, nbatch * bsz)  
    # Evenly divide the data across the bsz batches.  
    data = data.view(bsz, -1).t().contiguous()  
    return data.to(device)  
  
eval_batch_size = 10  
train_data = batchify(corpus.train, args.batch_size)  
val_data = batchify(corpus.valid, eval_batch_size)  
test_data = batchify(corpus.test, eval_batch_size)
```

$$p(\text{Mary}) \quad p(\text{helped}|\text{Mary}) \quad \cdots p(a|\text{Mary helped old} \cdots \text{had})$$

Mary ... helped a lamb. <EOS>





## 6.4.1 循环神经网络：应用示例

### ➤ 语言模型 language modeling

#### Step2:设计循环神经网络

```
#####
# Build the model
#####

ntokens = len(corpus.dictionary)
model = model.RNNModel(args.model, ntokens, args.emsize, args.nhid, args.nlayers, args.dropout, args.tied).to(device)

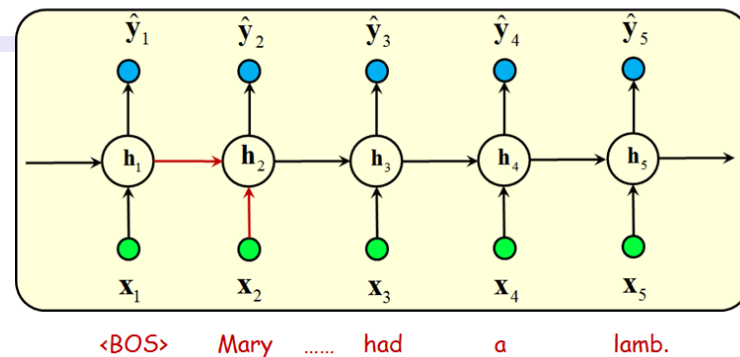
class RNNModel(nn.Module):
    """Container module with an encoder, a recurrent module, and a decoder."""

    def __init__(self, rnn_type, ntoken, ninp, nhid, nlayers, dropout=0.5, tie_weights=False):
        super(RNNModel, self).__init__()
        self.drop = nn.Dropout(dropout)
        self.encoder = nn.Embedding(ntoken, ninp)
        if rnn_type in ['LSTM', 'GRU']:
            self.rnn = getattr(nn, rnn_type)(ninp, nhid, nlayers, dropout=dropout)
        else:
            try:
                nonlinearity = {'RNN_TANH': 'tanh', 'RNN_RELU': 'relu'}[rnn_type]
            except KeyError:
                raise ValueError( """An invalid option for `--model` was supplied,
                                   options are ['LSTM', 'GRU', 'RNN_TANH' or 'RNN_RELU']""")
            self.rnn = nn.RNN(ninp, nhid, nlayers, nonlinearity=nonlinearity, dropout=dropout)
        self.decoder = nn.Linear(nhid, ntoken)

# Example input sequence and hidden states
x1, x2, x3, x4, x5 = <BOS>, Mary, ....., had, a, lamb.
h1, h2, h3, h4, h5 = hidden states
y1, y2, y3, y4, y5 = predicted tokens
```

$$p(\text{Mary}) \quad p(\text{helped}|\text{Mary}) \quad \cdots p(a|\text{Mary helped old} \cdots \text{had})$$

Mary ... helped a lamb. <EOS>



## 6.4.1 循环神经网络：应用示例

### ➤ 语言模型 language modeling

#### Step2:设计循环神经网络

```
#####
# Build the model
#####

ntokens = len(corpus.dictionary)
model = model.RNNModel(args.model, ntokens, args.emsize, args.nhid, args.nlayers, args.dropout, args.tied).to(device)

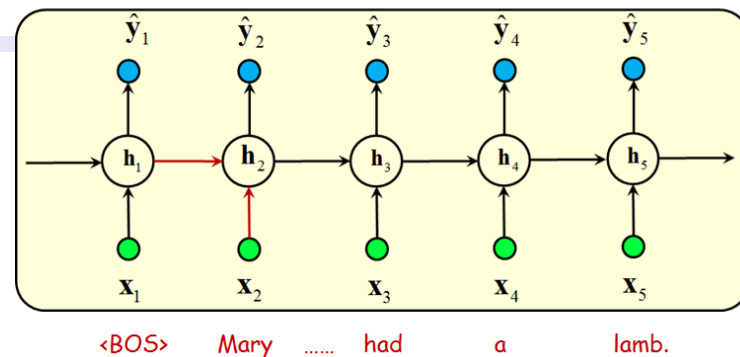
class RNNModel(nn.Module):
    """Container module with an encoder, a recurrent module, and a decoder."""

    def __init__(self, rnn_type, ntoken, ninp, nhid, nlayers, dropout=0.5, tie_weights=False):
        super(RNNModel, self).__init__()
        self.drop = nn.Dropout(dropout)
        self.encoder = nn.Embedding(ntoken, ninp)
        if rnn_type in ['LSTM', 'GRU']:
            self.rnn = getattr(nn, rnn_type)(ninp, nhid, nlayers, dropout=dropout)
        else:
            try:
                nonlinearity = {'RNN_TANH': 'tanh', 'RNN_RELU': 'relu'}[rnn_type]
            except KeyError:
                raise ValueError( "An invalid option for `--model` was supplied,
                                   options are ['LSTM', 'GRU', 'RNN_TANH' or 'RNN_RELU']")
        def forward(self, input, hidden):
            emb = self.drop(self.encoder(input))
            output, hidden = self.rnn(emb, hidden)
            output = self.drop(output)
            decoded = self.decoder(output.view(output.size(0)*output.size(1), output.size(2)))
            return decoded.view(output.size(0), output.size(1), decoded.size(1)), hidden

out)
```

$$p(\text{Mary}) \quad p(\text{helped}|\text{Mary}) \quad \cdots p(a|\text{Mary helped old} \cdots \text{had})$$

Mary ... helped a lamb. <EOS>



## 6.4.1 循环神经网络：应用示例

### ➤ 语言模型 language modeling

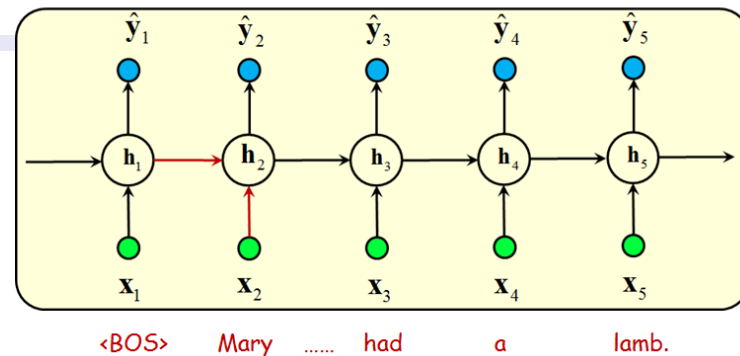
Step3:确定损失函数，选择优化策略

```
#####  
# Build the model  
#####  
  
ntokens = len(corpus.dictionary)  
model = model.RNNModel(args.model, ntokens, args.emsize, args.nhid, args.nlayers, args.dropout, args.tied).to(device)  
  
criterion = nn.CrossEntropyLoss()
```

损失函数

$p(\text{Mary}) \quad p(\text{helped}|\text{Mary}) \quad \cdots p(a|\text{Mary helped old} \cdots \text{had})$

Mary ... helped a lamb. <EOS>



## 6.4.1 循环神经网络：应用示例

### ➤ 语言模型 language modeling

#### Step4: 在训练集上训练学习神经网络参数

```
def train():
    # Turn on training mode which enables dropout.
    model.train()
    total_loss = 0.
    start_time = time.time()
    ntokens = len(corpus.dictionary)
    hidden = model.init_hidden(args.batch_size)
    for batch, i in enumerate(range(0, train_data.size(0) - 1, args.bptt)):
        data, targets = get_batch(train_data, i)
        # Starting each batch, we detach the hidden state from how it was previously produced.
        # If we didn't, the model would try backpropagating all the way to start of the dataset.
        hidden = repackage_hidden(hidden)
        model.zero_grad()
        output, hidden = model(data, hidden)
        loss = criterion(output.view(-1, ntokens), targets)
        loss.backward()

        # `clip_grad_norm` helps prevent the exploding gradient problem in RNNs / LSTMs.
        torch.nn.utils.clip_grad_norm_(model.parameters(), args.clip)
        for p in model.parameters():
            p.data.add_(-1r, p.grad.data)

        total_loss += loss.item()

    if batch % args.log_interval == 0 and batch > 0:
        cur_loss = total_loss / args.log_interval
        elapsed = time.time() - start_time
        print('| epoch {:3d} | {:5d}/{:5d} batches | lr {:02.2f} | ms/batch {:5.2f} | '
              'loss {:5.2f} | ppl {:8.2f}'.format(
            epoch, batch, len(train_data) // args.bptt, lr,
            elapsed * 1000 / args.log_interval, cur_loss, math.exp(cur_loss)))
        total_loss = 0
        start_time = time.time()
```

Gradient clipping

利用梯度下降更新参数

## 6.4.1 循环神经网络：有趣的应用

### ► 名著自动生成

#### Sonnet 116 - Let me not ...

*by William Shakespeare*

Let me not to the marriage of true minds  
Admit impediments. Love is not love  
Which alters when it alteration finds,  
Or bends with the remover to remove:  
O no! it is an ever-fixed mark  
That looks on tempests and is never shaken;  
It is the star to every wandering bark,  
Whose worth's unknown, although his height be taken.  
Love's not Time's fool, though rosy lips and cheeks  
Within his bending sickle's compass come:  
Love alters not with his brief hours and weeks,  
But bears it out even to the edge of doom.  
If this be error and upon me proved,  
I never writ, nor no man ever loved.

tyntd-iafhatawiaoihrdemot lytdws e ,tfti, astai f ogoh eoase rrranbyne 'nhthnee e  
plia tklrqd t o idoe ns,smtt h ne etie h,hregtrs nigtike,aoaenns lng

train more

"Tmont thithey" fomesscerliund  
Keushey. Thom here  
sheulke, anmerenith ol sivh I lalterthend Bleipile shuwly fil on aseterlome  
coaniogennc Phe lism thond hon at. MeiDimorotion in ther thize."

train more

Aftair fall unsuch that the hall for Prince Velzonski's that me of  
her hearly, and behs to so arwage fiving were to it beloge, pavu say falling misfort  
how, and Gogition is so overelical and ofter.

train more

"Why do what that day," replied Natasha, and wishing to himself the fact the  
princess, Princess Mary was easier, fed in had oftened him.  
Pierre aking his soul came to the packs and drove up his father-in-law women.



## 6.4.1 循环神经网络：有趣的应用

### ► 名著自动生成

PANDARUS:

Alas, I think he shall be come approached and the day  
When little srain would be attain'd into being never fed,  
And who is but a chain and subjects of his death,  
I should not sleep.

Second Senator:

They are away this miseries, produced upon my soul,  
Breaking and strongly should be buried, when I perish  
The earth and thoughts of many states.

DUKE VINCENTIO:

Well, your wit is in the care of side and that.

Second Lord:

They would be ruled after this chamber, and  
my fair nues begun out of the fact, to be conveyed,  
Whose noble souls I'll have the heart of the wars.

Clown:

Come, sir, I will make did behold your worship.

VIOLA:

I'll drink it.

VIOLA:

Why, Salisbury must find his flesh and thought  
That which I am not aps, not a man and in fire,  
To show the reining of the raven and the wars  
To grace my hand reproach within, and not a fair are hand,  
That Caesar and my goodly father's world;  
When I was heaven of presence and our fleets,  
We spare with hours, but cut thy council I am great,  
Murdered and by thy master's ready there  
My power to give thee but so much as hell:  
Some service in the noble bondman here,  
Would show him to her wine.

KING LEAR:

O, if you were a feeble sight, the courtesy of your law,  
Your sight and several breath, will wear the gods  
With his heads, and my hands are wonder'd at the deeds,  
So drop upon your lordship's head, and your opinion  
Shall be against your honour.

# 6.4.1 循环神经网络：有趣的应用

## ► 论文自动生成

For  $\bigoplus_{n=1, \dots, m} \mathcal{L}_{m_n} = 0$ , hence we can find a closed subset  $\mathcal{H}$  in  $\mathcal{H}$  and any sets  $\mathcal{F}$  on  $X$ ,  $U$  is a closed immersion of  $S$ , then  $U \rightarrow T$  is a separated algebraic space.

*Proof.* Proof of (1). It also start we get

$$S = \text{Spec}(R) = U \times_X U \times_X U$$

and the comparicoly in the fibre product covering we have to prove the lemma generated by  $\coprod Z \times_U U \rightarrow V$ . Consider the maps  $M$  along the set of points  $\text{Sch}_{fppf}$  and  $U \rightarrow U$  is the fibre category of  $S$  in  $U$  in Section, ?? and the fact that any  $U$  affine, see Morphisms, Lemma ?? . Hence we obtain a scheme  $S$  and any open subset  $W \subset U$  in  $\text{Sh}(G)$  such that  $\text{Spec}(R') \rightarrow S$  is smooth or an

$$U = \bigcup U_i \times_{S_i} U_i$$

which has a nonzero morphism we may assume that  $f_i$  is of finite presentation over  $S$ . We claim that  $\mathcal{O}_{X,x}$  is a scheme where  $x, x', s'' \in S'$  such that  $\mathcal{O}_{X,x'} \rightarrow \mathcal{O}'_{X',x'}$  is separated. By Algebra, Lemma ?? we can define a map of complexes  $\text{GL}_{S'}(x'/S'')$  and we win.  $\square$

To prove study we see that  $\mathcal{F}|_U$  is a covering of  $\mathcal{X}'$ , and  $\mathcal{T}_i$  is an object of  $\mathcal{F}_{X/S}$  for  $i > 0$  and  $\mathcal{F}_p$  exists and let  $\mathcal{F}_i$  be a presheaf of  $\mathcal{O}_X$ -modules on  $\mathcal{C}$  as a  $\mathcal{F}$ -module. In particular  $\mathcal{F} = U/\mathcal{F}$  we have to show that

$$\widetilde{M}^\bullet = \mathcal{I}^\bullet \otimes_{\text{Spec}(k)} \mathcal{O}_{S,s} - i_X^{-1} \mathcal{F}$$

is a unique morphism of algebraic stacks. Note that

$$\text{Arrows} = (\text{Sch}/S)_{fppf}^{opp}, (\text{Sch}/S)_{fppf}$$

and

$$V = \Gamma(S, \mathcal{O}) \mapsto (U, \text{Spec}(A))$$

is an open subset of  $X$ . Thus  $U$  is affine. This is a continuous map of  $X$  is the inverse, the groupoid scheme  $S$ .

*Proof.* See discussion of sheaves of sets.  $\square$

The result for prove any open covering follows from the less of Example ?? . It may replace  $S$  by  $X_{spaces, \acute{e}tale}$  which gives an open subspace of  $X$  and  $T$  equal to  $S_{Zar}$ , see Descent, Lemma ?? . Namely, by Lemma ?? we see that  $R$  is geometrically regular over  $S$ .

**Lemma 0.1.** Assume (3) and (3) by the construction in the description.

Suppose  $X = \lim |X|$  (by the formal open covering  $X$  and a single map  $\text{Proj}_X(\mathcal{A}) = \text{Spec}(B)$  over  $U$  compatible with the complex

$$\text{Set}(\mathcal{A}) = \Gamma(X, \mathcal{O}_{X, \mathcal{O}_X}).$$

When in this case of to show that  $\mathcal{Q} \rightarrow \mathcal{C}_{Z/X}$  is stable under the following result in the second conditions of (1), and (3). This finishes the proof. By Definition ?? (without element is when the closed subschemes are catenary. If  $T$  is surjective we may assume that  $T$  is connected with residue fields of  $S$ . Moreover there exists a closed subspace  $Z \subset X$  of  $X$  where  $U$  in  $X'$  is proper (some defining as a closed subset of the uniqueness it suffices to check the fact that the following theorem

(1)  $f$  is locally of finite type. Since  $S = \text{Spec}(R)$  and  $Y = \text{Spec}(R)$ .

*Proof.* This is form all sheaves of sheaves on  $X$ . But given a scheme  $U$  and a surjective étale morphism  $U \rightarrow X$ . Let  $U \cap U = \coprod_{i=1, \dots, n} U_i$  be the scheme  $X$  over  $S$  at the schemes  $X_i \rightarrow X$  and  $U = \lim_i X_i$ .  $\square$

The following lemma surjective restrocomposes of this implies that  $\mathcal{F}_{x_0} = \mathcal{F}_{x_0} = \mathcal{F}_{X, \dots, 0}$ .

**Lemma 0.2.** Let  $X$  be a locally Noetherian scheme over  $S$ ,  $E = \mathcal{F}_{X/S}$ . Set  $\mathcal{I} = \mathcal{I}_1 \subset \mathcal{I}'_n$ . Since  $\mathcal{I}^n \subset \mathcal{I}^n$  are nonzero over  $i_0 \leq \mathfrak{p}$  is a subset of  $\mathcal{I}_{n,0} \circ \bar{A}_2$  works.

**Lemma 0.3.** In Situation ?? . Hence we may assume  $\mathfrak{q}' = 0$ .

*Proof.* We will use the property we see that  $\mathfrak{p}$  is the next functor (??). On the other hand, by Lemma ?? we see that

$$D(\mathcal{O}_{X'}) = \mathcal{O}_X(D)$$

where  $K$  is an  $F$ -algebra where  $\delta_{n+1}$  is a scheme over  $S$ .  $\square$

## 6.4.1 循环神经网络：有趣的应用

### ► 论文自动生成

*Proof. Omitted.* □

**Lemma 0.1.** *Let  $\mathcal{C}$  be a set of the construction.*

*Let  $\mathcal{C}$  be a gerber covering. Let  $\mathcal{F}$  be a quasi-coherent sheaves of  $\mathcal{O}$ -modules. We have to show that*

$$\mathcal{O}_{\mathcal{O}_X} = \mathcal{O}_X(\mathcal{L})$$

*Proof.* This is an algebraic space with the composition of sheaves  $\mathcal{F}$  on  $X_{\text{étale}}$  we have

$$\mathcal{O}_X(\mathcal{F}) = \{\text{morph}_1 \times_{\mathcal{O}_X} (\mathcal{G}, \mathcal{F})\}$$

where  $\mathcal{G}$  defines an isomorphism  $\mathcal{F} \rightarrow \mathcal{F}$  of  $\mathcal{O}$ -modules. □

**Lemma 0.2.** *This is an integer  $Z$  is injective.*

*Proof.* See Spaces, Lemma ?? □

**Lemma 0.3.** *Let  $S$  be a scheme. Let  $X$  be a scheme and  $X$  is an affine open covering. Let  $\mathcal{U} \subset \mathcal{X}$  be a canonical and locally of finite type. Let  $X$  be a scheme. Let  $X$  be a scheme which is equal to the formal complex.*

*The following to the construction of the lemma follows.*

*Let  $X$  be a scheme. Let  $X$  be a scheme covering. Let*

$$b : X \rightarrow Y' \rightarrow Y \rightarrow Y \rightarrow Y' \times_X Y \rightarrow X.$$

*be a morphism of algebraic spaces over  $S$  and  $Y$ .*

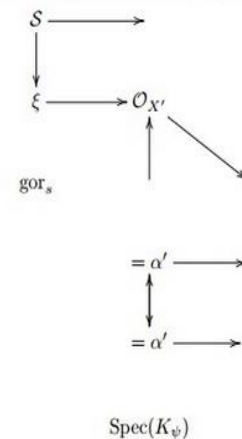
*Proof.* Let  $X$  be a nonzero scheme of  $X$ . Let  $X$  be an algebraic space. Let  $\mathcal{F}$  be a quasi-coherent sheaf of  $\mathcal{O}_X$ -modules. The following are equivalent

- (1)  $\mathcal{F}$  is an algebraic space over  $S$ .
- (2) If  $X$  is an affine open covering.

Consider a common structure on  $X$  and  $X$  the functor  $\mathcal{O}_X(U)$  which is locally of finite type. □

*Example from on A. Karpathy's cs231n notes.*

This since  $\mathcal{F} \in \mathcal{F}$  and  $x \in \mathcal{G}$  the diagram



is a limit. Then  $\mathcal{G}$  is a finite type and assume  $S$  is a flat and  $\mathcal{F}$  and  $\mathcal{G}$  is a finite type  $f_*$ . This is of finite type diagrams, and

- the composition of  $\mathcal{G}$  is a regular sequence,
- $\mathcal{O}_{X'}$  is a sheaf of rings.

□

*Proof.* We have see that  $X = \text{Spec}(R)$  and  $\mathcal{F}$  is a finite type representable by algebraic space. The property  $\mathcal{F}$  is a finite morphism of algebraic stacks. Then the cohomology of  $X$  is an open neighbourhood of  $U$ . □

*Proof.* This is clear that  $\mathcal{G}$  is a finite presentation, see Lemmas ??.

A reduced above we conclude that  $U$  is an open covering of  $\mathcal{C}$ . The functor  $\mathcal{F}$  is a “field”

$$\mathcal{O}_{X,x} \longrightarrow \mathcal{F}_x^{-1}(\mathcal{O}_{X_{\text{étale}}}) \longrightarrow \mathcal{O}_{X_t}^{-1} \mathcal{O}_{X_{\lambda}}(\mathcal{O}_{X_n}^{\vee})$$

is an isomorphism of covering of  $\mathcal{O}_{X_t}$ . If  $\mathcal{F}$  is the unique element of  $\mathcal{F}$  such that  $X$  is an isomorphism.

The property  $\mathcal{F}$  is a disjoint union of Proposition ?? and we can filtered set of presentations of a scheme  $\mathcal{O}_X$ -algebra with  $\mathcal{F}$  are opens of finite type over  $S$ .

If  $\mathcal{F}$  is a scheme theoretic image points. □

If  $\mathcal{F}$  is a finite direct sum  $\mathcal{O}_{X_{\lambda}}$  is a closed immersion, see Lemma ?? . This is a sequence of  $\mathcal{F}$  is a similar morphism.



## 6.4.1 循环神经网络：有趣的应用

### ➤ 代码自动生成

```
static void do_command(struct seq_file *m, void *v)
{
    int column = 32 << (cmd[2] & 0x80);
    if (state)
        cmd = (int)(int_state ^ (in_8(&ch->ch_flags) & Cmd) ? 2 : 1);
    else
        seq = 1;
    for (i = 0; i < 16; i++) {
        if (k & (1 << 1))
            pipe = (in_use & UMXTHREAD_UNCCA) +
                ((count & 0x00000000ffffffff) & 0x000000f) << 8;
        if (count == 0)
            sub(pid, ppc_md.kexec_handle, 0x20000000);
        pipe_set_bytes(i, 0);
    }
    /* Free our user pages pointer to place camera if all dash */
    subsystem_info = &of_changes[PAGE_SIZE];
    rek_controls(offset, idx, &soffset);
    /* Now we want to deliberately put it to device */
    control_check_polarity(&context, val, 0);
    for (i = 0; i < COUNTER; i++)
        seq_puts(s, "policy ");
}
```

Example from on A. Karpathy's cs231n notes.

## 6.4.2 Transformer: 应用示例

### ► 应用：语言模型

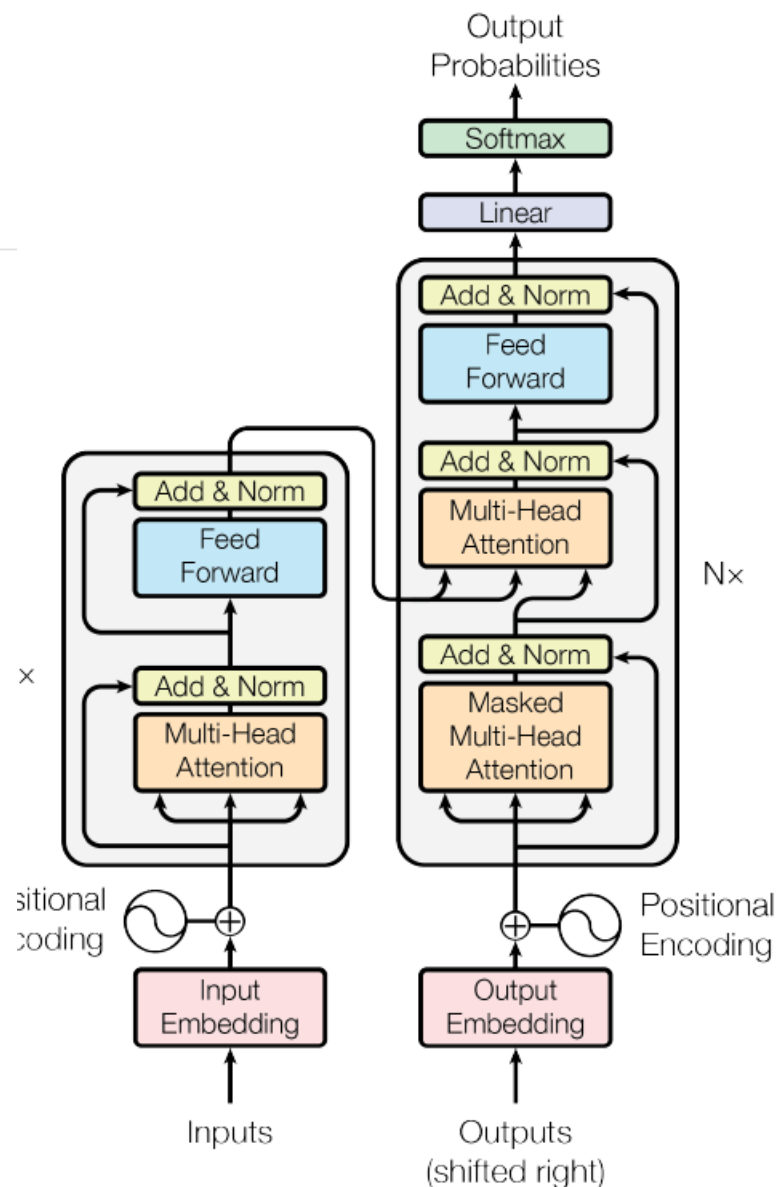
Step1: 定义Transformer模型

```
import torch
from torch import nn, Tensor
import torch.nn.functional as F
from torch.nn import TransformerEncoder, TransformerEncoderLayer
from torch.utils.data import dataset

class TransformerModel(nn.Module):

    def __init__(self, ntoken: int, d_model: int, nhead: int, d_hid: int,
                 nlayers: int, dropout: float = 0.5):
        super().__init__()
        self.model_type = 'Transformer'
        self.pos_encoder = PositionalEncoding(d_model, dropout)
        encoder_layers = TransformerEncoderLayer(d_model, nhead, d_hid, dropout)
        self.transformer_encoder = TransformerEncoder(encoder_layers, nlayers)
        self.encoder = nn.Embedding(ntoken, d_model)
        self.d_model = d_model
        self.decoder = nn.Linear(d_model, ntoken)

        self.init_weights()
```

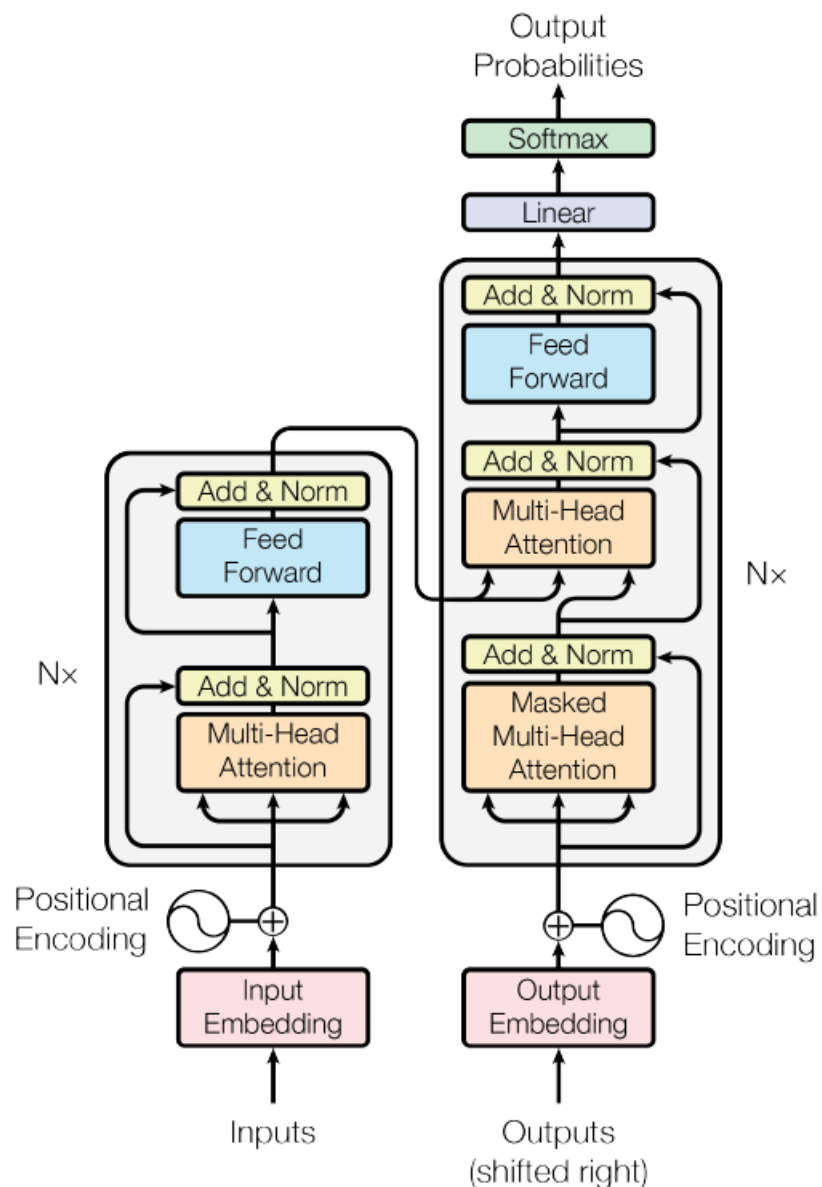


## 6.4.2 Transformer: 应用示例

### ► 应用：语言模型

Step1: 定义Transformer模型

```
class TransformerModel(nn.Module):  
  
    def __init__(self, ntoken: int, d_model: int, nhead: int, d_hid: int,  
                  nlayers: int, dropout: float = 0.5):  
        super().__init__()  
        self.model_type = 'Transformer'  
        self.pos_encoder = PositionalEncoding(d_model, dropout)  
        encoder_layers = TransformerEncoderLayer(d_model, nhead, d_hid, dropout)  
        self.transformer_encoder = TransformerEncoder(encoder_layers, nlayers)  
        self.encoder = nn.Embedding(ntoken, d_model)  
        self.d_model = d_model  
        self.decoder = nn.Linear(d_model, ntoken)  
  
    def forward(self, src: Tensor, src_mask: Tensor) -> Tensor:  
        """  
        Args:  
            src: Tensor, shape [seq_len, batch_size]  
            src_mask: Tensor, shape [seq_len, seq_len]  
  
        Returns:  
            output Tensor of shape [seq_len, batch_size, ntoken]  
        """  
        src = self.encoder(src) * math.sqrt(self.d_model)  
        src = self.pos_encoder(src)  
        output = self.transformer_encoder(src, src_mask)  
        output = self.decoder(output)  
        return output
```



## 6.4.2 Transformer: 应用示例

### ► 应用：语言模型

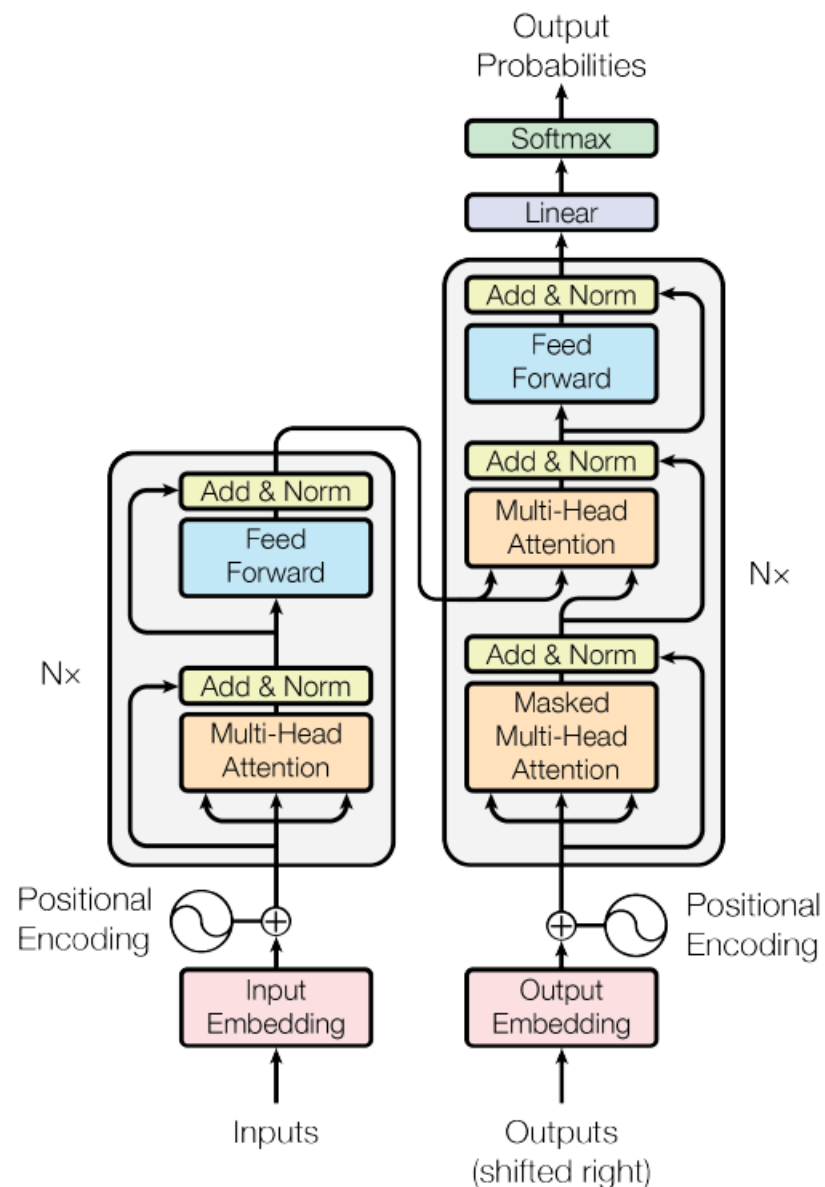
Step1: 定义位置编码函数

```
class PositionalEncoding(nn.Module):

    def __init__(self, d_model: int, dropout: float = 0.1, max_len: int = 5000):
        super().__init__()
        self.dropout = nn.Dropout(p=dropout)

        position = torch.arange(max_len).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2) * (-math.log(10000.0) / d_model))
        pe = torch.zeros(max_len, 1, d_model)
        pe[:, 0, 0::2] = torch.sin(position * div_term)
        pe[:, 0, 1::2] = torch.cos(position * div_term)
        self.register_buffer('pe', pe)

    def forward(self, x: Tensor) -> Tensor:
        """
        Args:
            x: Tensor, shape [seq_len, batch_size, embedding_dim]
        """
        x = x + self.pe[:x.size(0)]
        return self.dropout(x)
```



```

train_iter = WikiText2(split='train')
tokenizer = get_tokenizer('basic_english')
vocab = build_vocab_from_iterator(map(tokenizer, train_iter), specials=['<unk>'])
vocab.set_default_index(vocab['<unk>'])

def data_process(raw_text_iter: dataset.IterableDataset) -> Tensor:
    """Converts raw text into a flat Tensor."""
    data = [torch.tensor(vocab(tokenizer(item)), dtype=torch.long) for item in raw_text_iter]
    return torch.cat(torch.tensor(filter(lambda t: t.numel() > 0, data)))

# train_iter was "consumed" by the process of building the vocab,
# so we have to create it again
train_iter, val_iter, test_iter = WikiText2()
train_data = data_process(train_iter)
val_data = data_process(val_iter)
test_data = data_process(test_iter)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

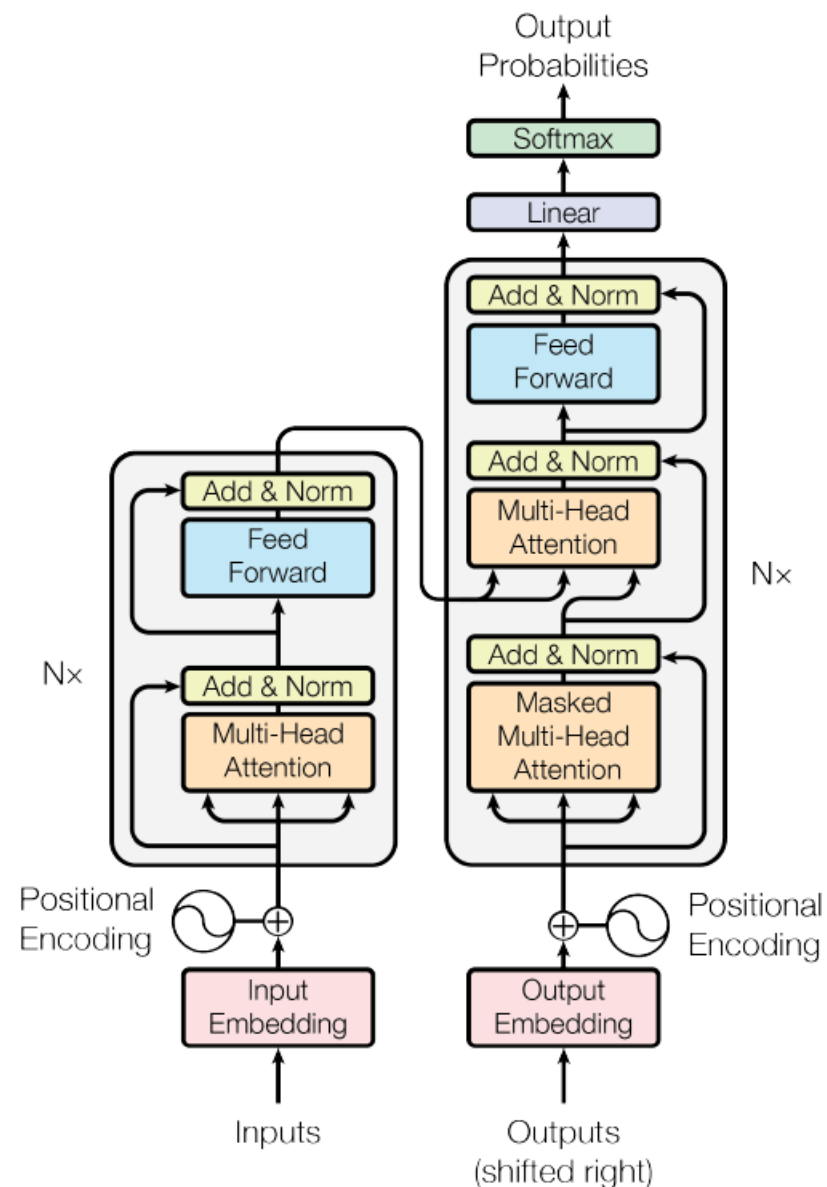
def batchify(data: Tensor, bsz: int) -> Tensor:
    """Divides the data into bsz separate sequences, removing extra elements
    that wouldn't cleanly fit.

    Args:
        data: Tensor, shape [N]
        bsz: int, batch size

    Returns:
        Tensor of shape [N // bsz, bsz]
    """
    seq_len = data.size(0) // bsz
    data = data[:seq_len * bsz]
    data = data.view(bsz, seq_len).t().contiguous()
    return data.to(device)

```

Step2: 准备语料库corpus，包括训练集和测试集

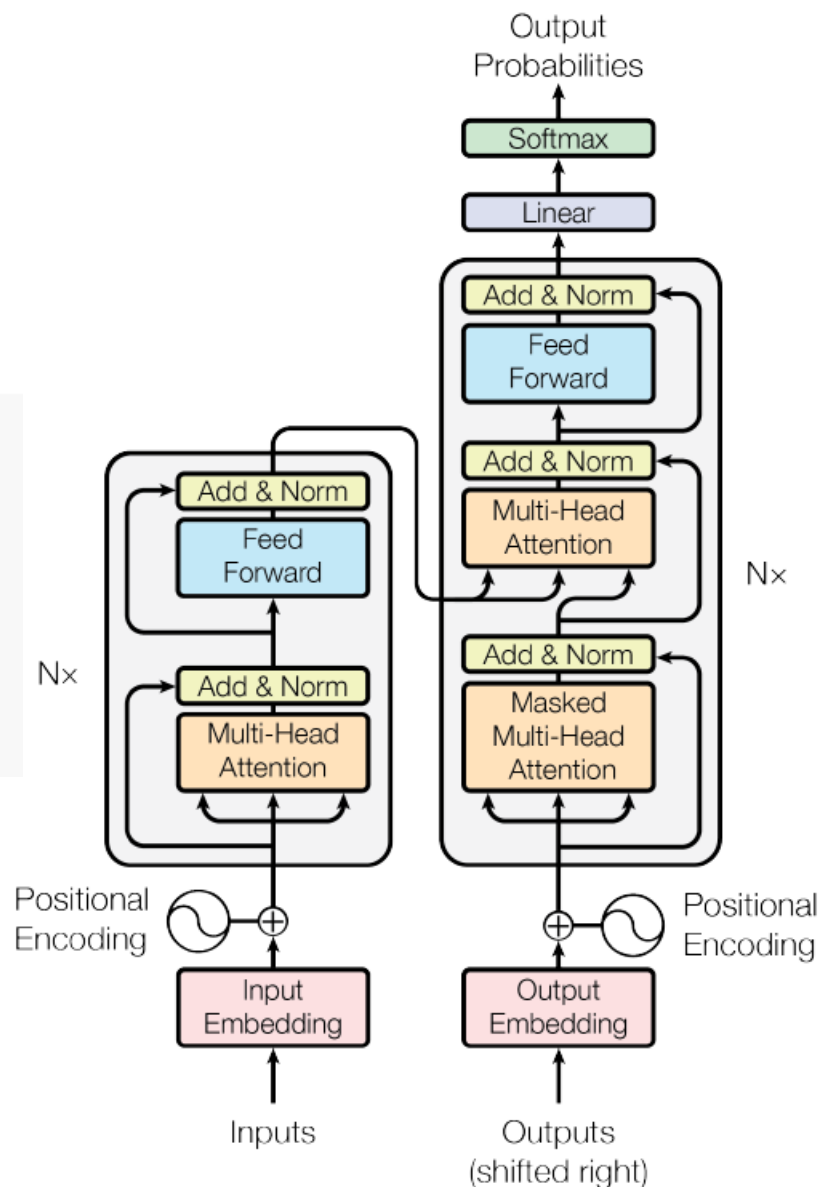


## 6.4.2 Transformer: 应用示例

### ➤ 应用：语言模型

Step3: 实例化模型

```
ntokens = len(vocab) # size of vocabulary
emsize = 200 # embedding dimension
d_hid = 200 # dimension of the feedforward network model in nn.TransformerEncoder
nlayers = 2 # number of nn.TransformerEncoderLayer in nn.TransformerEncoder
nhead = 2 # number of heads in nn.MultiheadAttention
dropout = 0.2 # dropout probability
model = TransformerModel(ntokens, emsize, nhead, d_hid, nlayers, dropout).to(device)
```



```

import copy
import time

criterion = nn.CrossEntropyLoss()
lr = 5.0 # learning rate
optimizer = torch.optim.SGD(model.parameters(), lr=lr)
scheduler = torch.optim.lr_scheduler.StepLR(optimizer, 1.0, gamma=0.95)

def train(model: nn.Module) -> None:
    model.train() # turn on train mode
    total_loss = 0.
    log_interval = 200
    start_time = time.time()
    src_mask = generate_square_subsequent_mask(bptt).to(device)

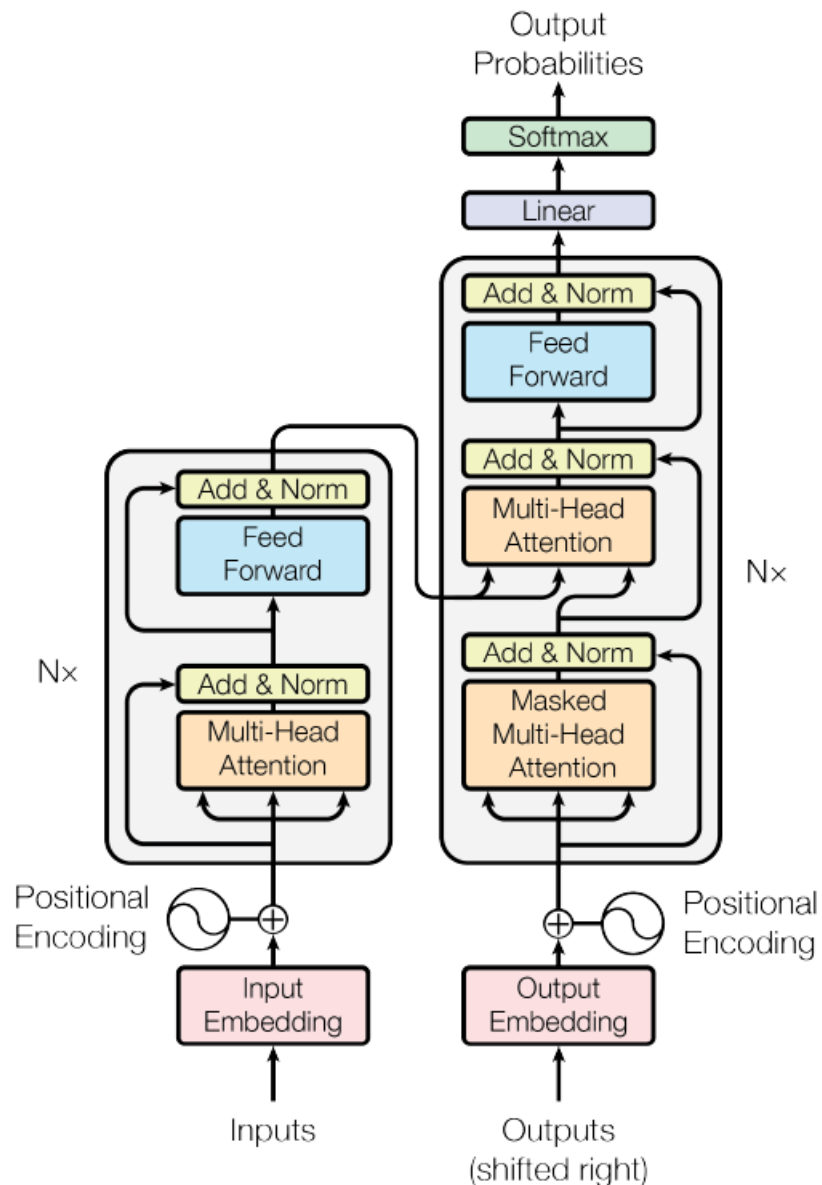
    num_batches = len(train_data) // bptt
    for batch, i in enumerate(range(0, train_data.size(0) - 1, bptt)):
        data, targets = get_batch(train_data, i)
        batch_size = data.size(0)
        if batch_size != bptt: # only on last batch
            src_mask = src_mask[:batch_size, :batch_size]
        output = model(data, src_mask)
        loss = criterion(output.view(-1, ntokens), targets)

        optimizer.zero_grad()
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 0.5)
        optimizer.step()

        total_loss += loss.item()
        if batch % log_interval == 0 and batch > 0:
            lr = scheduler.get_last_lr()[0]
            ms_per_batch = (time.time() - start_time) * 1000 / log_interval
            cur_loss = total_loss / log_interval
            ppl = math.exp(cur_loss)
            print(f'| epoch {epoch:3d} | {batch:5d}/{num_batches:5d} batches | '
                  f'lr {lr:02.2f} | ms/batch {ms_per_batch:5.2f} | '
                  f'loss {cur_loss:5.2f} | ppl {ppl:8.2f}')
            total_loss = 0
            start_time = time.time()

```

## Step4: 训练和评测模型



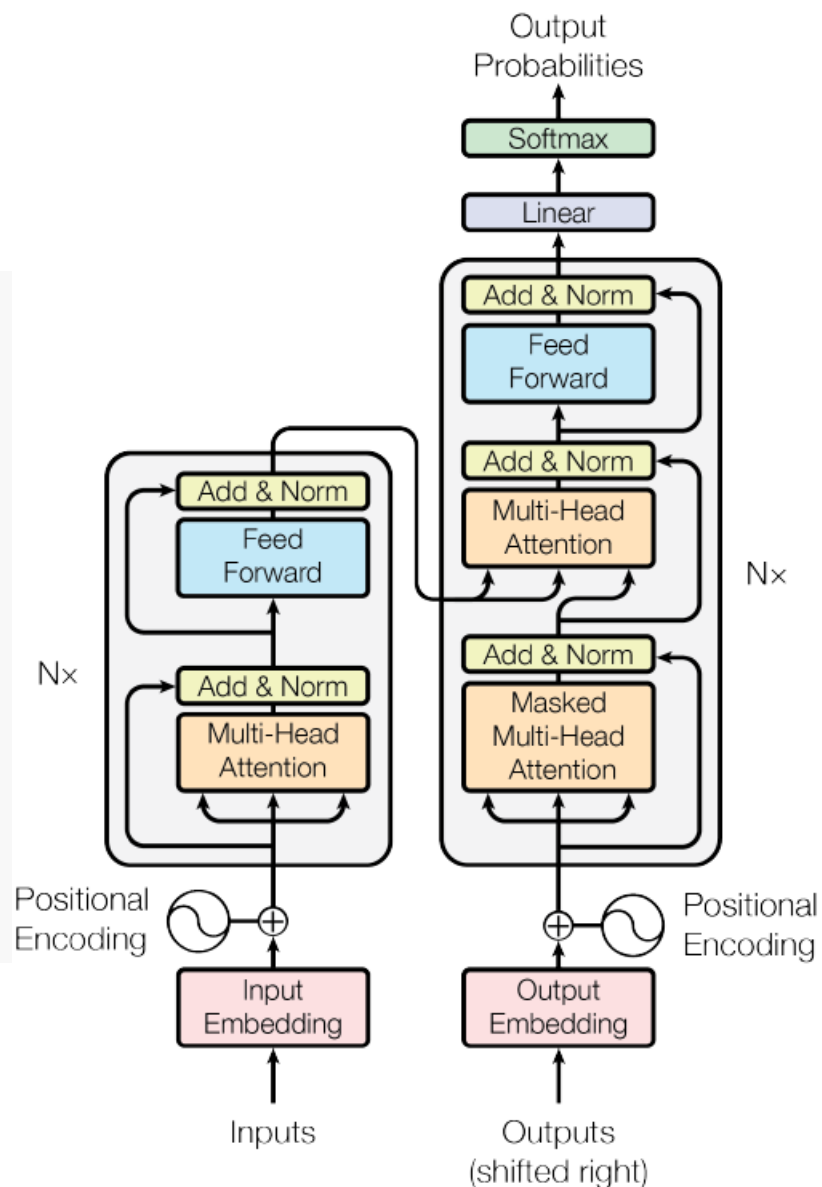


## 6.4.2 Transformer: 应用示例

### ➤ 应用：语言模型

Step4: 训练和评测模型

```
def evaluate(model: nn.Module, eval_data: Tensor) -> float:
    model.eval() # turn on evaluation mode
    total_loss = 0.
    src_mask = generate_square_subsequent_mask(bptt).to(device)
    with torch.no_grad():
        for i in range(0, eval_data.size(0) - 1, bptt):
            data, targets = get_batch(eval_data, i)
            batch_size = data.size(0)
            if batch_size != bptt:
                src_mask = src_mask[:batch_size, :batch_size]
            output = model(data, src_mask)
            output_flat = output.view(-1, ntokens)
            total_loss += batch_size * criterion(output_flat, targets).item()
    return total_loss / (len(eval_data) - 1)
```





## 6.4.2 Transformer: 应用示例

### ➤ 应用：语言模型

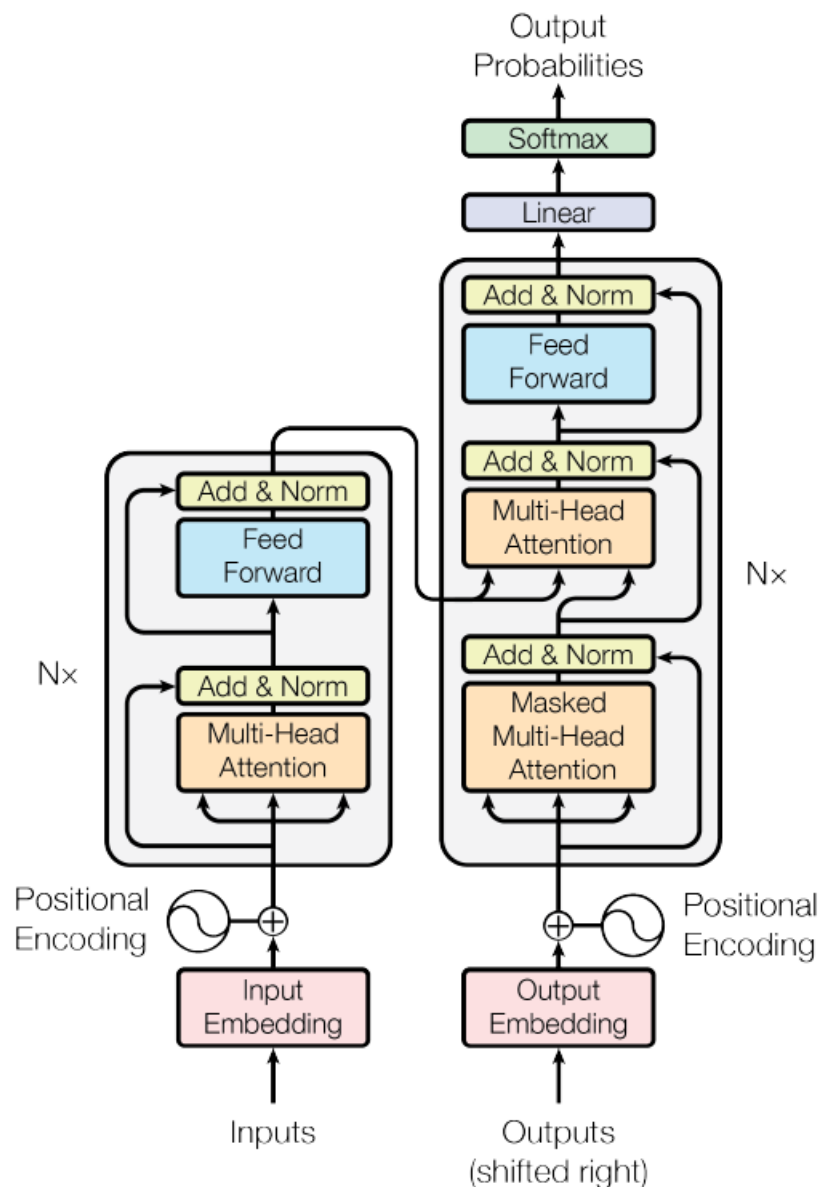
Step4: 训练和评测模型

```
best_val_loss = float('inf')
epochs = 3
best_model = None

for epoch in range(1, epochs + 1):
    epoch_start_time = time.time()
    train(model)
    val_loss = evaluate(model, val_data)
    val_ppl = math.exp(val_loss)
    elapsed = time.time() - epoch_start_time
    print('-' * 89)
    print(f'| end of epoch {epoch:3d} | time: {elapsed:5.2f}s | '
          f'valid loss {val_loss:5.2f} | valid ppl {val_ppl:8.2f}')
    print('-' * 89)

    if val_loss < best_val_loss:
        best_val_loss = val_loss
        best_model = copy.deepcopy(model)

    scheduler.step()
```

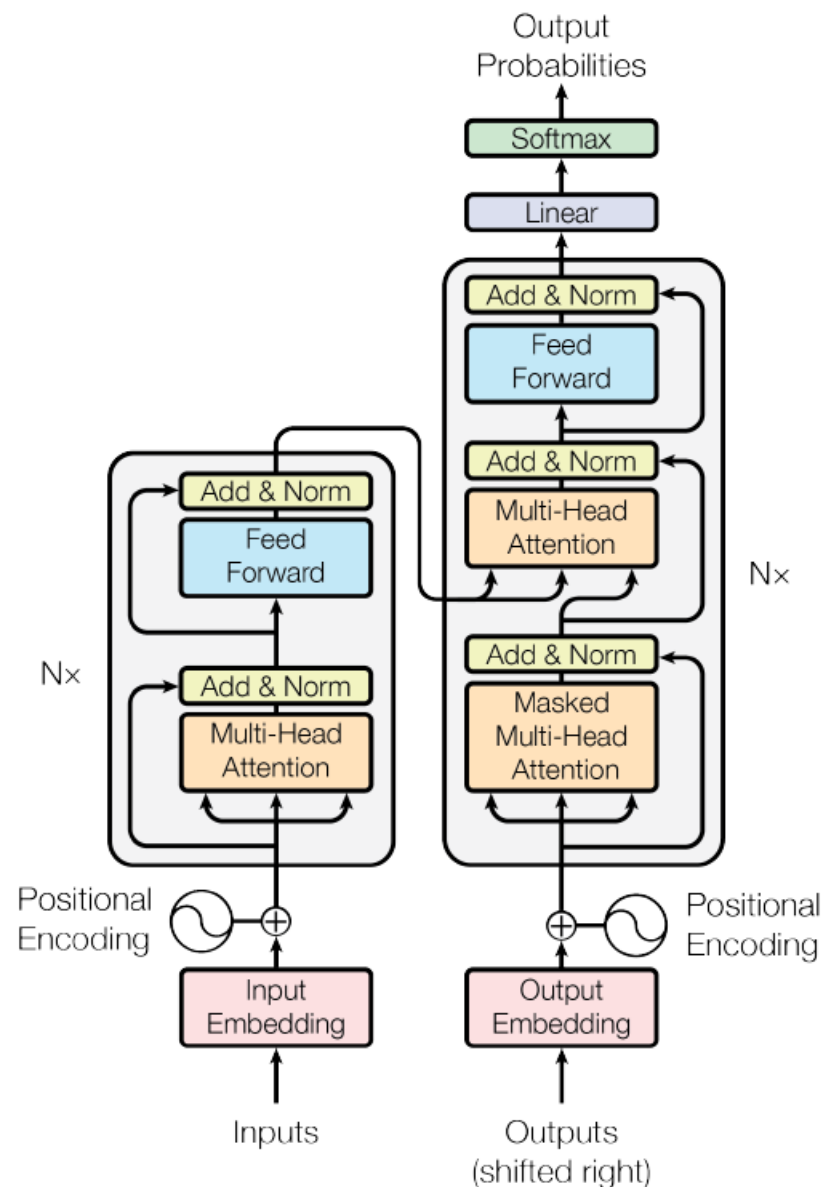


## 6.4.2 Transformer: 应用示例

### ➤ 应用：语言模型

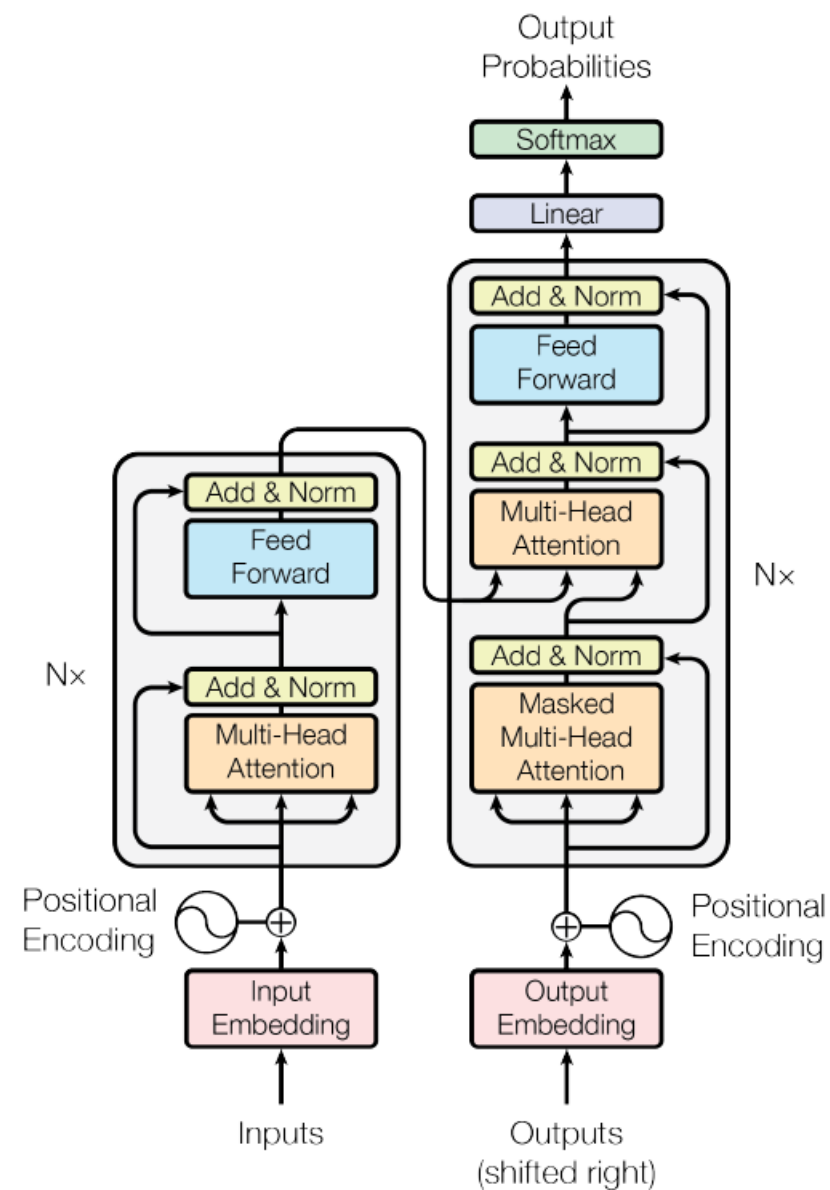
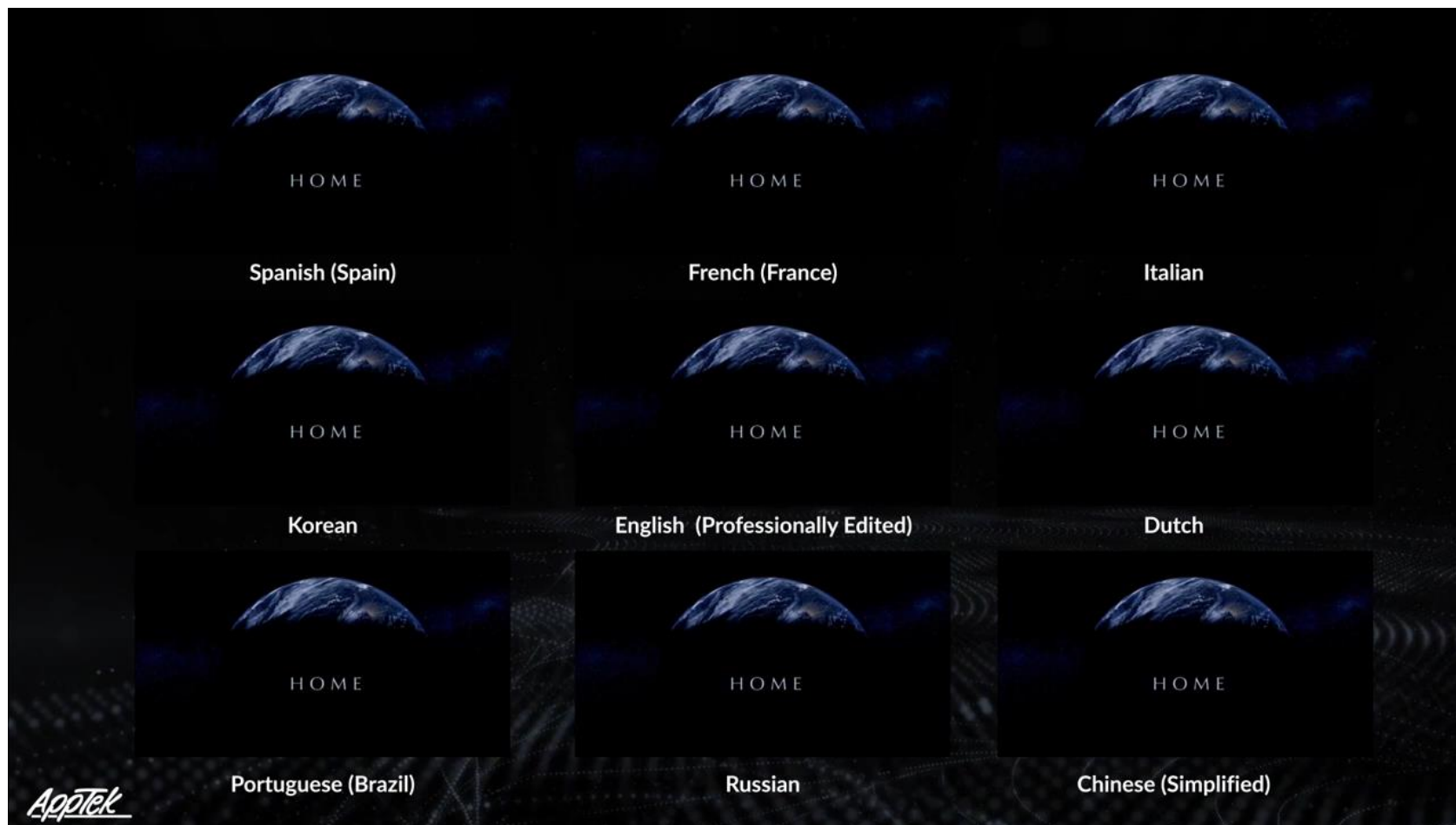
损失变化示意

|       |   |  |                    |  |         |  |                |  |           |  |             |
|-------|---|--|--------------------|--|---------|--|----------------|--|-----------|--|-------------|
| epoch | 1 |  | 200/ 2928 batches  |  | lr 5.00 |  | ms/batch 21.78 |  | loss 8.14 |  | ppl 3421.03 |
| epoch | 1 |  | 400/ 2928 batches  |  | lr 5.00 |  | ms/batch 21.38 |  | loss 6.85 |  | ppl 941.65  |
| epoch | 1 |  | 600/ 2928 batches  |  | lr 5.00 |  | ms/batch 21.36 |  | loss 6.42 |  | ppl 612.34  |
| epoch | 1 |  | 800/ 2928 batches  |  | lr 5.00 |  | ms/batch 21.39 |  | loss 6.28 |  | ppl 536.43  |
| epoch | 1 |  | 1000/ 2928 batches |  | lr 5.00 |  | ms/batch 21.39 |  | loss 6.18 |  | ppl 482.89  |
| epoch | 1 |  | 1200/ 2928 batches |  | lr 5.00 |  | ms/batch 21.38 |  | loss 6.15 |  | ppl 466.39  |
| epoch | 1 |  | 1400/ 2928 batches |  | lr 5.00 |  | ms/batch 21.38 |  | loss 6.10 |  | ppl 447.80  |
| epoch | 1 |  | 1600/ 2928 batches |  | lr 5.00 |  | ms/batch 21.37 |  | loss 6.10 |  | ppl 445.58  |
| epoch | 1 |  | 1800/ 2928 batches |  | lr 5.00 |  | ms/batch 21.38 |  | loss 6.02 |  | ppl 410.14  |
| epoch | 1 |  | 2000/ 2928 batches |  | lr 5.00 |  | ms/batch 21.35 |  | loss 6.01 |  | ppl 406.32  |
| epoch | 1 |  | 2200/ 2928 batches |  | lr 5.00 |  | ms/batch 21.36 |  | loss 5.88 |  | ppl 358.38  |
| epoch | 1 |  | 2400/ 2928 batches |  | lr 5.00 |  | ms/batch 21.44 |  | loss 5.95 |  | ppl 385.36  |
| epoch | 1 |  | 2600/ 2928 batches |  | lr 5.00 |  | ms/batch 21.40 |  | loss 5.94 |  | ppl 380.64  |
| epoch | 1 |  | 2800/ 2928 batches |  | lr 5.00 |  | ms/batch 21.36 |  | loss 5.87 |  | ppl 353.83  |



## 6.4.2 Transformer: 应用

### ► 应用：机器翻译



## 6.4.2 Transformer: 应用

### ➤应用：语言大模型 GPT系列 (Generative Pre-Training)

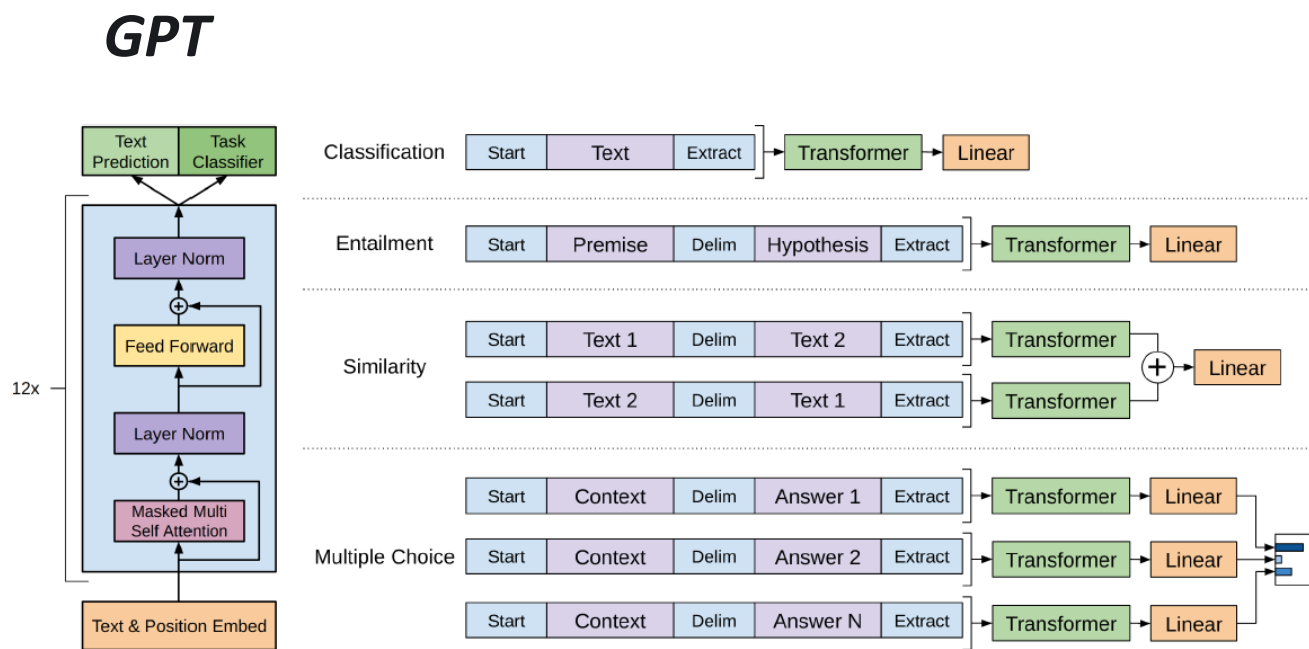
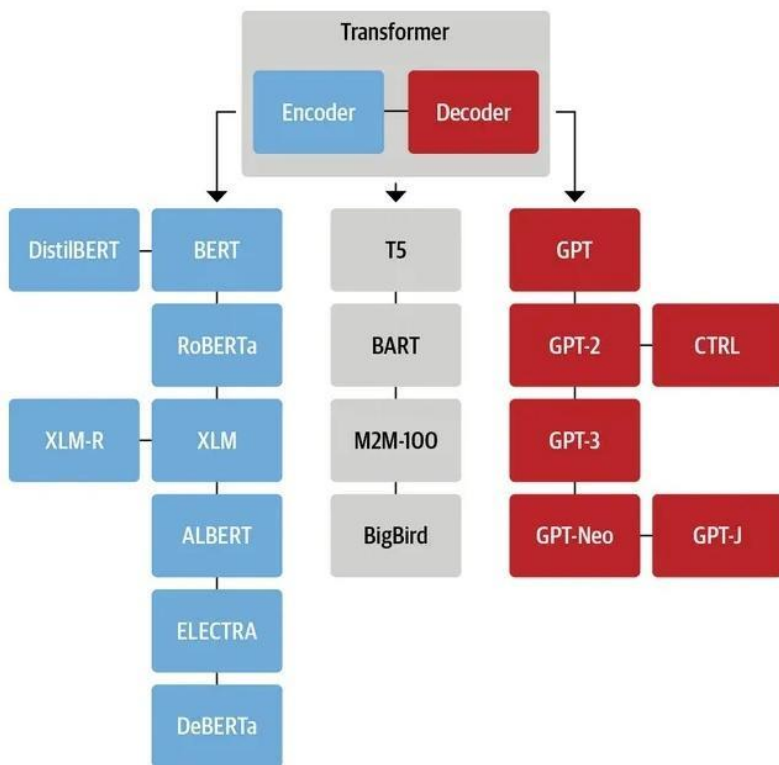


Figure 1: **(left)** Transformer architecture and training objectives used in this work. **(right)** Input transformations for fine-tuning on different tasks. We convert all structured inputs into token sequences to be processed by our pre-trained model, followed by a linear+softmax layer.

## 6.4.2 Transformer: 应用

### ➤ 应用：语言大模型 GPT 系列 (Generative Pre-Training)

**GPT-2** We use a Transformer (Vaswani et al., 2017) based architecture for our LMs. The model largely follows the details of the OpenAI GPT model (Radford et al., 2018) with a few modifications. Layer normalization (Ba et al., 2016) was moved to the input of each sub-block, similar to a pre-activation residual network (He et al., 2016) and an additional layer normalization was added after the final self-attention block. A modified initialization which accounts for the accumulation on the residual path with model depth is used. We scale the weights of residual layers at initialization by a factor of  $1/\sqrt{N}$  where  $N$  is the number of residual layers. The vocabulary is expanded to 50,257. We also increase the context size from 512 to 1024 tokens and a larger batchsize of 512 is used.

## 6.4.2 Transformer: 应用

### ➤ 应用：语言大模型 GPT 系列

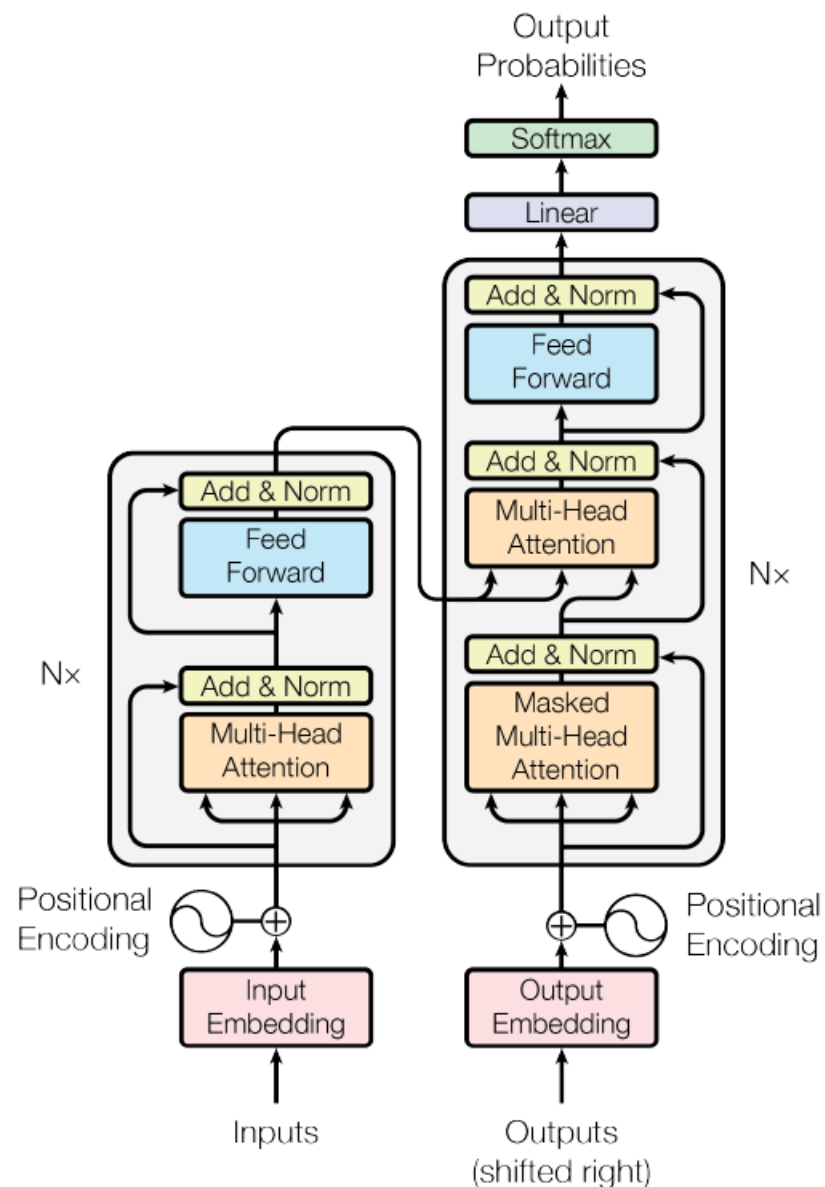
#### GPT-3

##### Describe a layout.

Just describe any layout you want, and it'll try to render below!

A div that contains 3 buttons each with a random color.

Generate





## 6.4.2 Transformer: 应用

### ➤ 应用：语言大模型 GPT 系列

**GPT-3**

Describe a layout.

Just describe any layout you want!

I

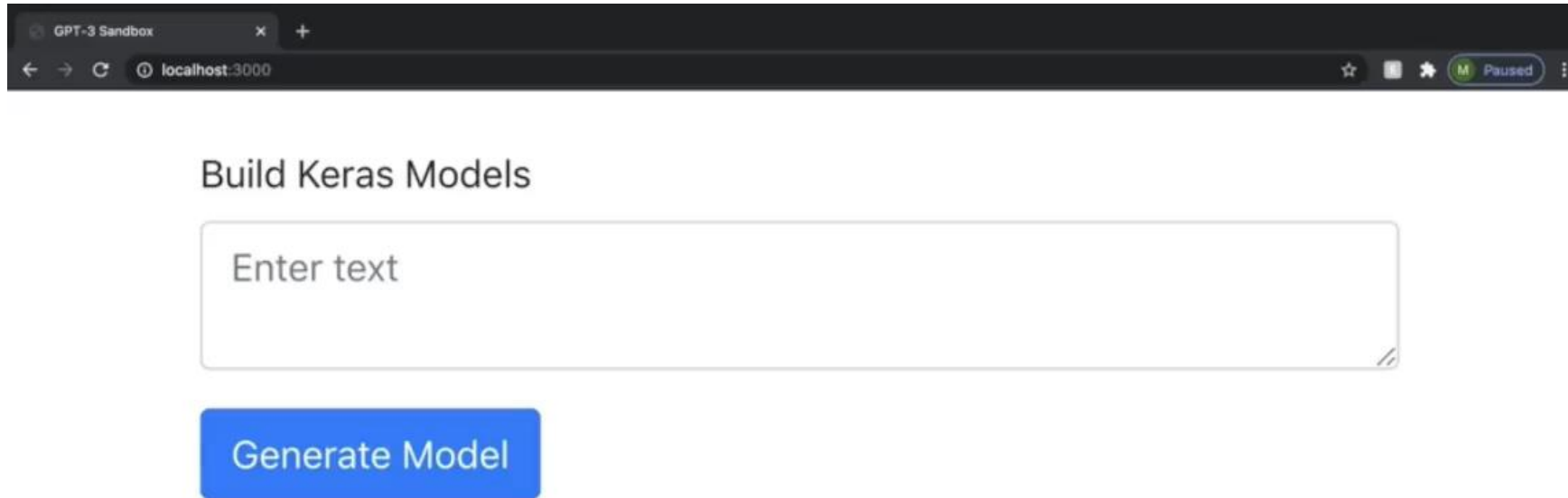
Generate

Mega-mode is  
on

## 6.4.2 Transformer: 应用

### ➤ 应用：语言大模型 GPT 系列

**GPT-3**





## 6.4.2 Transformer: 应用-大语言模型



User this code is not working like i expect — how do i fix it?

```
resultWorkerErr := make(chan error)
defer close(resultWorkerErr)
go func() {
    defer cancel()
    resultWorkerErr <- b.resultWorker(ctx)
}()

err := b.worker(ctx)
cancel()
if err == nil {
    return <-resultWorkerErr
}
return multierror.Append(err, <-resultWorkerErr)
```

ChatGPT It's difficult to say what's wrong with the code without more context information about what the code is supposed to do and what isn't this the entire code or just a part of it?

User just part of it — the error never surfaces. I think there is something

ChatGPT It's difficult to say without more information about what the code is happening when it's executed. One potential issue with the code is that the `resultWorkerErr` channel is never closed, which means that the code could potentially hang if

User what is the fermat's little theorem

ChatGPT Fermat's Little Theorem states that if  $p$  is a prime then  $a^p \equiv a \pmod{p}$  ( $a$  to the power  $p$ ) is congruent to  $a$  remainder is always  $a$ .

For example, if  $p = 5$  and  $a = 2$ , then  $2^5 = 32$  is congruent to 2 modulo 5, and Fermat's Little

Fermat's Little Theorem is often used in cryptography necessary to perform modular arithmetic or as a tool for proving other theorems in number theory.

User how is it used in cryptography?

ChatGPT Fermat's Little Theorem is used in cryptography applications in the generation of so-called pseudo-random numbers to securely transmit messages over the internet and other networks.

In a public-key cryptography system, each user has a pair of keys: a public key, which is widely known and can be used by anyone to encrypt a message intended for that user, and a private key, which is known only to the user and is used to decrypt messages that have been encrypted with the corresponding public key.

One way to generate these keys is to use prime numbers and Fermat's Little Theorem. For

User help me write a short note to introduce myself to my neighbor

ChatGPT Hi there,

My name is [Your Name] and I just moved in next door. I wanted to reach out and introduce myself. I'm looking forward to getting to know you and the neighborhood. If you ever need anything or just want to chat, please don't hesitate to stop by or give me a call.

Best,  
[Your Name]

User can you make it more formal?

ChatGPT Dear Neighbor,

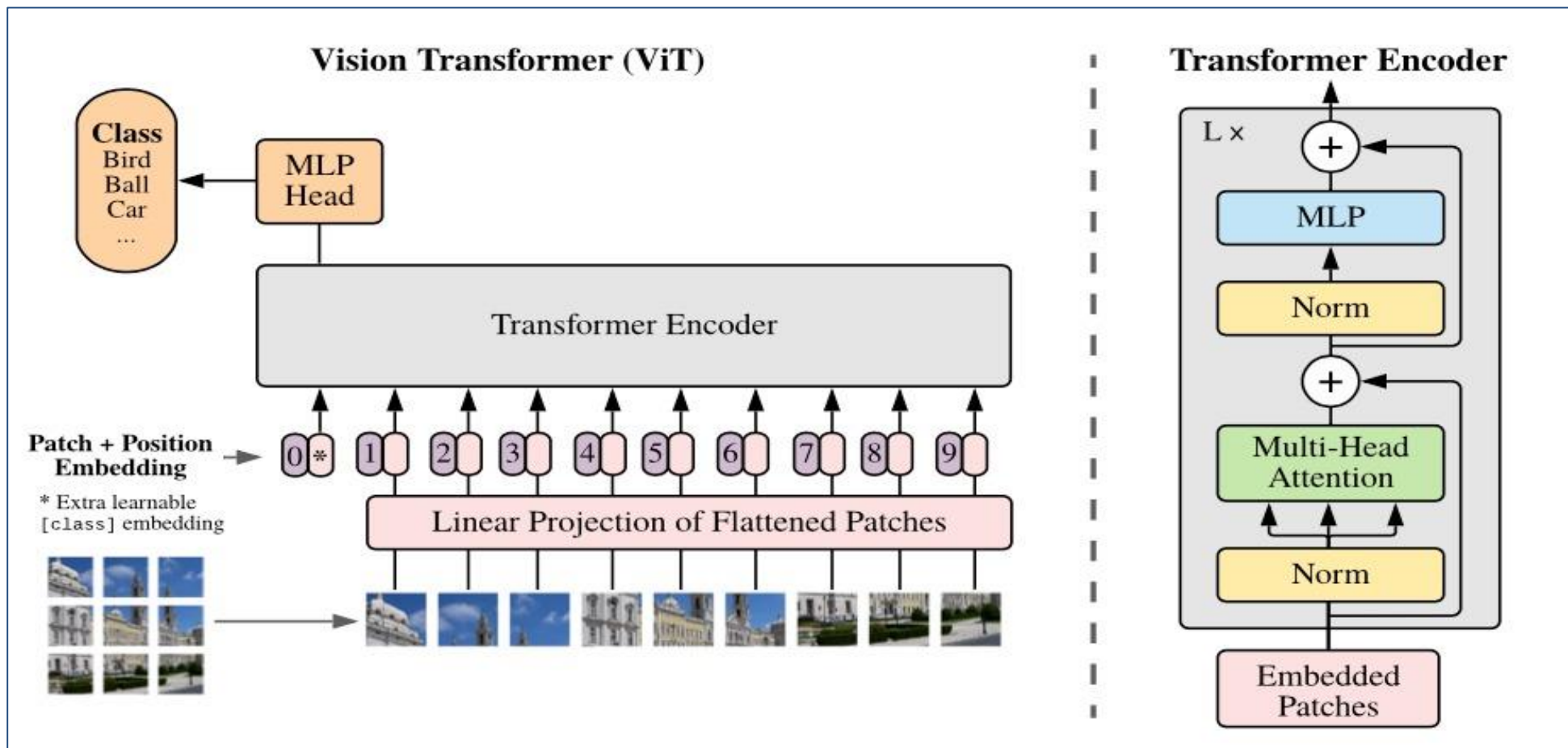
I am writing to introduce myself. My name is [Your Name] and I have recently moved in next door. I wanted to take a moment to reach out and say hello. I am looking forward to getting to know you and the neighborhood.

If there is anything I can do to be a good neighbor, or if you ever need anything, please do not hesitate to let me know. I am happy to help in any way that I can.

Sincerely,  
[Your Name]

## 6.4.2 Transformer: 应用--视觉识别

### ➤ Vision Transformer (ViT)



## 6.4.2 Transformer: 应用--视觉识别

### ➤ Segmentation Transformer

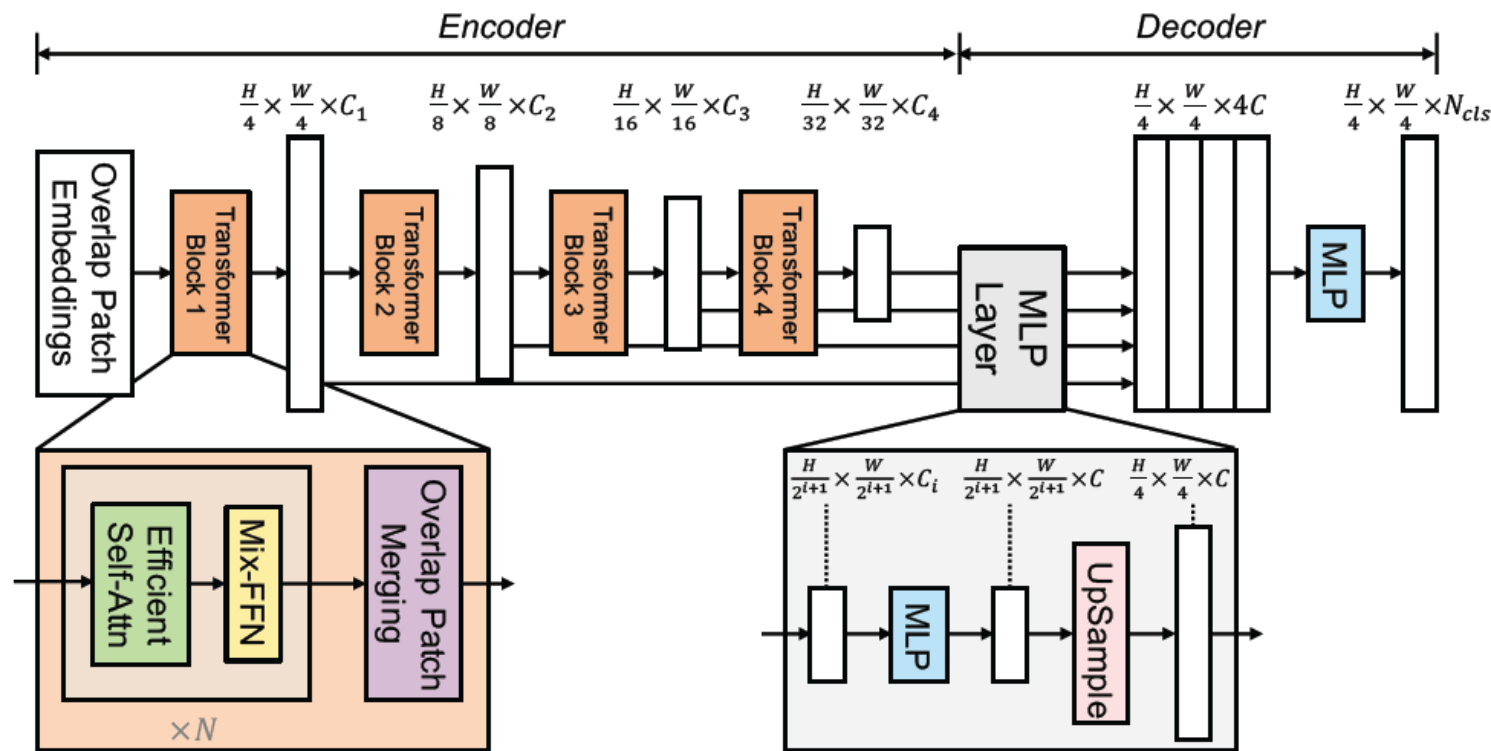


Figure 2: **The proposed SegFormer framework** consists of two main modules: A hierarchical Transformer encoder to extract coarse and fine features; and a lightweight All-MLP decoder to directly fuse these multi-level features and predict the semantic segmentation mask. “FFN” indicates feed-forward network.

## 6.4.2 Transformer: 应用--视觉识别

### ➤ Segmentation Transformer - demo

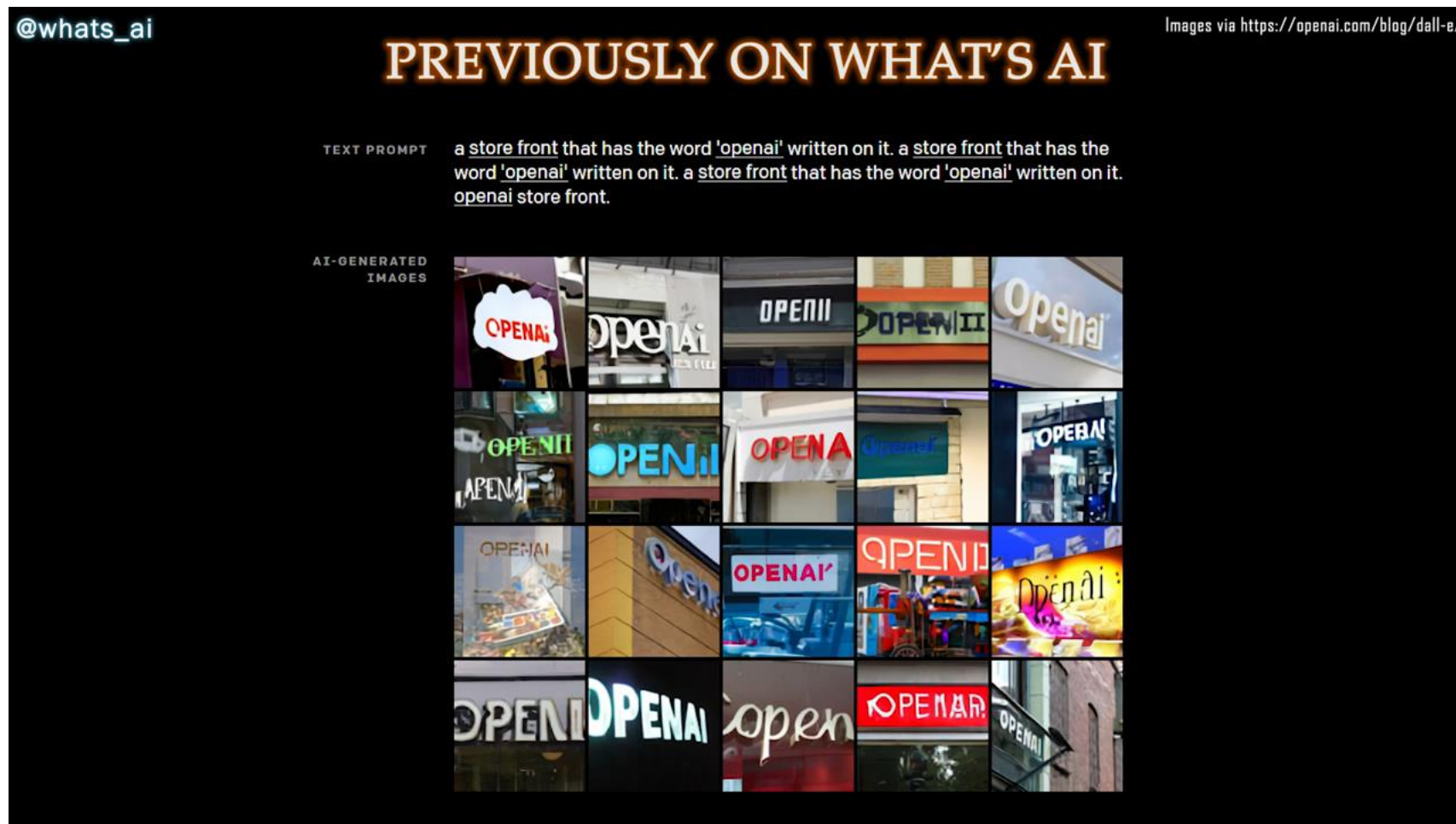
#### SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers

Enze Xie, Wenhai Wang, Zhiding Yu, Anima Anandkumar, Jose M. Alvarez, Ping Luo

The University of Hong Kong Nanjing University NVIDIA Caltech

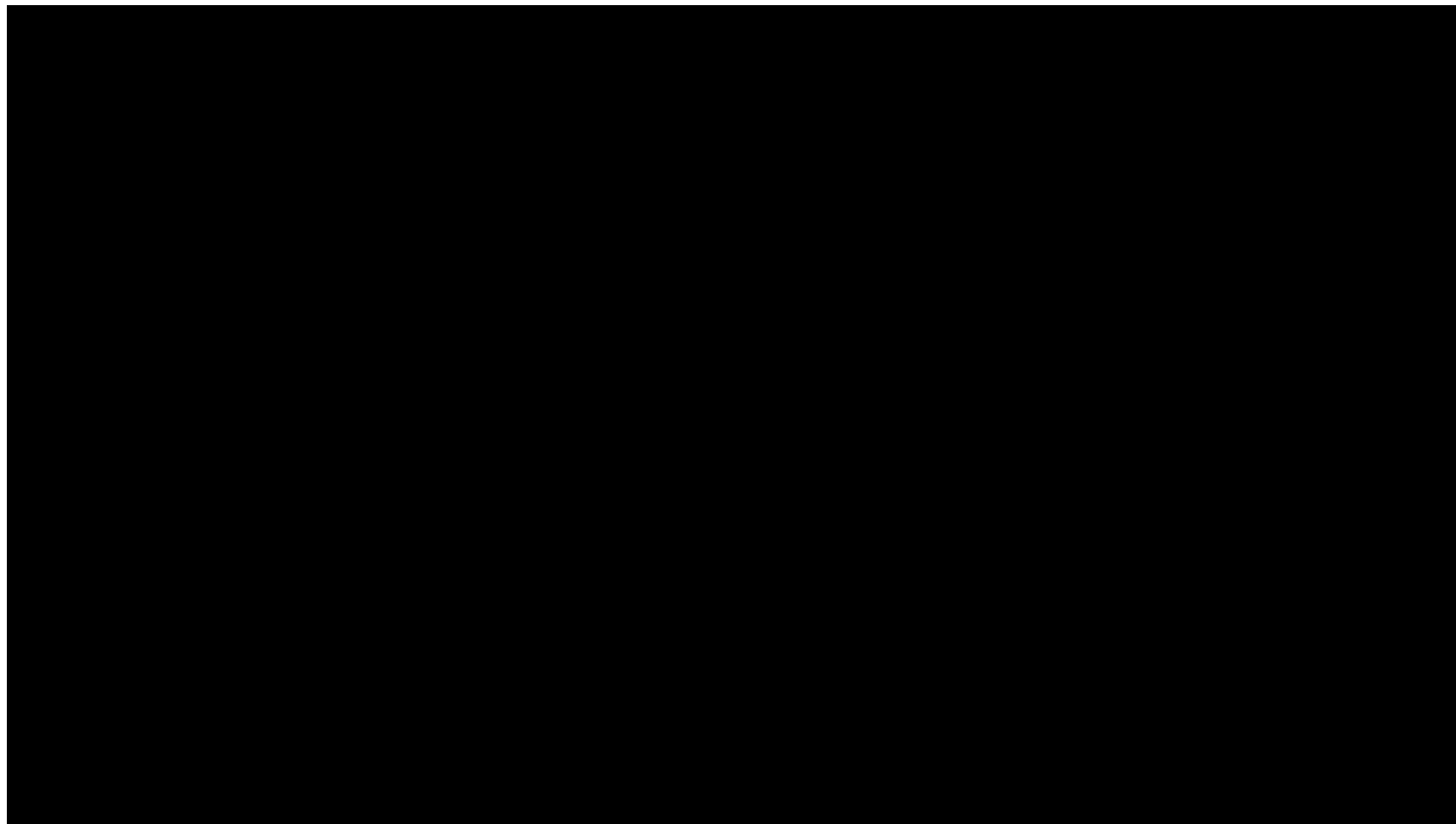
## 6.4.2 Transformer: 应用--多模态学习

### ➤ Text-to-image - demo (DALL E)



## 6.4.2 Transformer: 应用--多模态学习

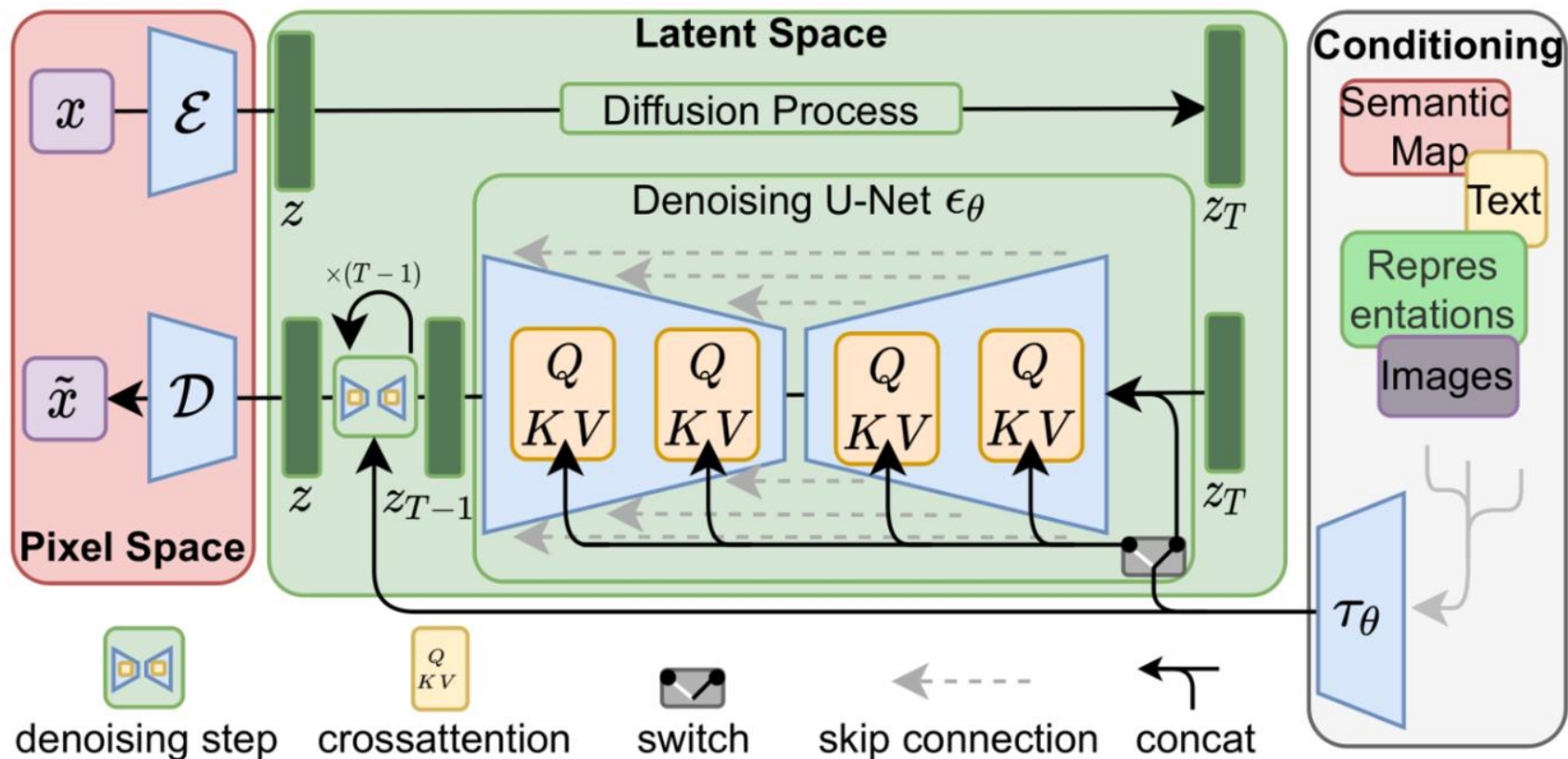
### ➤ Text-to-image - demo (DALL E 2)





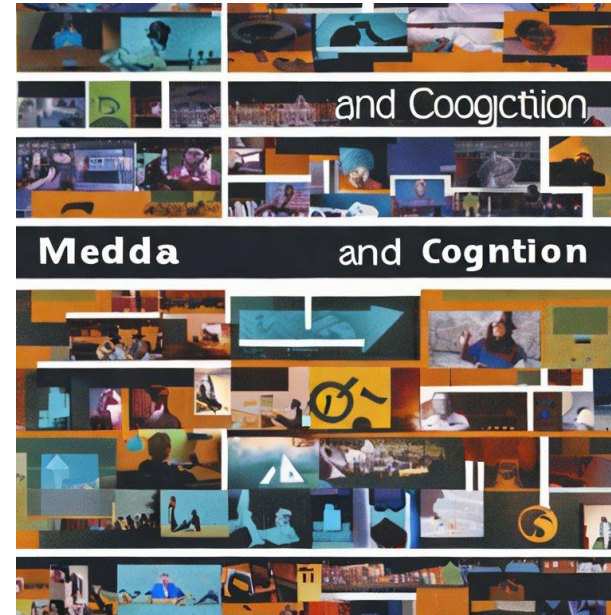
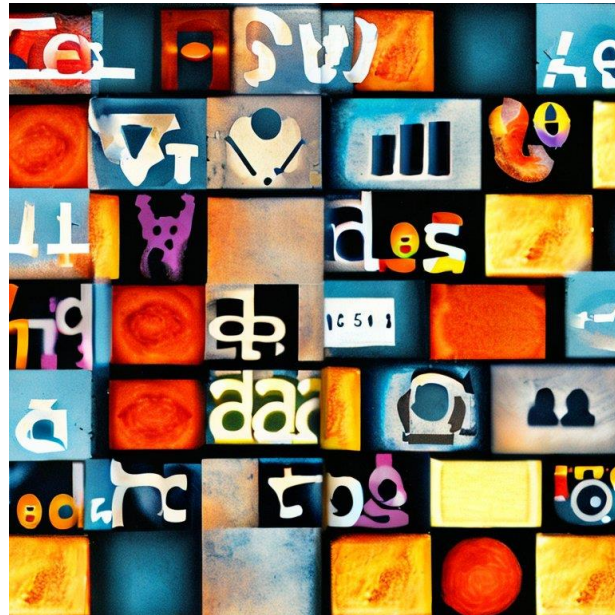
## 6.4.2 Transformer: 应用--多模态学习

### ➤ Text-to-image - demo (stable diffusion)



## 6.4.2 Transformer: 应用--多模态学习

### ➤ Text-to-image - demo (stable diffusion)





谢谢大家！

欢迎提出宝贵意见！